University of Sannio

Department of Engineering

Master's Degree in

Electronics Engineering for Automation and Sensing

OPTIMAL ELECTRIC VEHICLE BATTERY MANAGEMENT FOR VEHICLE-TO-GRID: MODEL PREDICTIVE CONTROL AND REINFORCEMENT LEARNING APPROACHES

Supervisor:
Prof. Carmela Bernardo

Co-Supervisor:
Dr. Antonio Pepiciello

Candidate:
Angelo Caravella
Student ID 389000016

Academic Year 2024–2025

Abstract in italian

L'adozione crescente dei **Veicoli Elettrici (EV)** in concomitanza con la sempre maggiore diffusione di **Fonti di Energia Rinnovabile (RES)** non programmabili, presenta sfide significative alla **stabilità** e all'**efficienza della rete elettrica**. La tecnologia **Vehicle-to-Grid (V2G)** emerge come soluzione fondamentale, trasformando gli EV da carichi passivi a **risorse energetiche flessibili** capaci di fornire vari **servizi di rete**. Questa tesi affronta il complesso **problema di ottimizzazione multi-obiettivo** della gestione intelligente di carica e scarica degli EV, che intrinsecamente implica un equilibrio tra **benefici economici**, **esigenze di mobilità dell'utente**, **preservazione della salute della batteria** e **stabilità della rete** in condizioni stocastiche.

Di fronte alla complessa sfida di ottimizzare la ricarica degli EV in scenari V2G, un approccio che si limita a un singolo modello di controllo, come il Deep Q-Networks (DQN), risulterebbe inadeguato. La natura del problema, caratterizzata da molteplici obiettivi contrastanti e da una profonda incertezza, richiede un'analisi comparativa e rigorosa di un'ampia gamma di strategie di controllo. Per questo motivo, la ricerca si concentra sulla valutazione di un portafoglio diversificato di algoritmi, che include numerosi modelli di Deep Reinforcement Learning (DRL), approcci euristici e il Model Predictive Control (MPC). Questo approccio consente di mappare in modo completo il panorama delle soluzioni, identificando i punti di forza e di debolezza di ciascun approccio in relazione alle diverse sfaccettature del problema V2G.

Il lavoro di tesi non si focalizza su un singolo metodo, ma adotta un approccio comparativo su larga scala perché **non esiste una soluzione unica** data la complessità del problema V2G, che rende improbabile che un solo algoritmo sia ottimale in tutte le condizioni. **Si ricercano i compromessi** poiché l'obiettivo è comprendere i trade-off tra l'efficienza dei dati, la stabilità dell'addestramento, la robustezza all'incertezza e la complessità computazionale delle diverse famiglie di algoritmi. **La validazione è più rigorosa** in quanto confrontare i modelli di DRL non solo tra loro ma anche con benchmark consolidati come le euristiche e l'MPC fornisce una misura molto più credibile del loro reale valore aggiunto.

Abstract

The growing adoption of **Electric Vehicles (EVs)**, combined with the increasing penetration of intermittent **Renewable Energy Sources (RESs)**, presents significant challenges to the **stability** and **efficiency** of the power grid. The **Vehicle-to-Grid (V2G)** technology emerges as a key solution, transforming EVs from passive loads into **flexible energy resources** capable of providing various **grid services**. This thesis addresses the complex **multi-objective optimization problem** of smart EV charging and discharging, which requires balancing **economic benefits**, **user mobility needs**, **battery health preservation**, and **grid stability** under stochastic conditions.

Given the complexity of optimizing EV charging in V2G scenarios, relying on a single control model is insufficient. The nature of the problem, characterized by multiple conflicting objectives and profound uncertainty, demands a rigorous comparative analysis of a wide range of control strategies. For this reason, the research focuses on evaluating a diverse portfolio of algorithms, including numerous Deep Reinforcement Learning (DRL) models, heuristic approaches, and Model Predictive Control (MPC). This method allows for a complete mapping of the solution space, identifying the strengths and weaknesses of each approach in relation to the different facets of the V2G problem.

This thesis does not focus on a single method, but adopts a large-scale comparative approach because **there is no single solution** given the complexity of the V2G problem, which makes it unlikely that a single algorithm is optimal under all conditions.**Trade-offs are explored** because the goal is to understand the trade-offs between data efficiency, training stability, robustness to uncertainty, and computational complexity of different families of algorithms. **Validation is more rigorous** because comparing DRL models not only with each other but also with established benchmarks such as heuristics and MPC provides a much more credible measure of their true added value.

List of Acronyms

Acronym	Description
Artificial Intelligence & Control	
A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
AC	Actor-Critic
AI	Artificial Intelligence
ARS	Augmented Random Search
CL	Curriculum Learning
DDPG	Deep Deterministic Policy Gradient
DQN	Deep Q-Networks
DRL	Deep Reinforcement Learning
LQR	Linear Quadratic Regulator
MILP	Mixed-Integer Linear Programming
MPC	Model Predictive Control
NN	Neural Network
PER	Prioritized Experience Replay
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
SAC	Soft Actor-Critic
TD3	Twin-Delayed Deep Deterministic Policy Gradient
TQC	Truncated Quantile Critics
TRPO	Trust Region Policy Optimization
Electric Vehicles & Charging	
AFAP	As Fast As Possible (Heuristic)
ALAP	As Late As Possible (Heuristic)
CPO	Charge Point Operator
EV	Electric Vehicle
V2B	Vehicle-to-Building
V2G	Vehicle-to-Grid
V2H	Vehicle-to-Home
V2V	Vehicle-to-Vehicle
VPP	Virtual Power Plant
Metrics & Technical Parameters	
CC	Constant Current
SoC	State of Charge
SoH	State of Health
CV	Constant Voltage
DoD	Depth of Discharg
MSE	Mean Square Error
RMSE	Root Mean Square Error

Contents

Chapter 1

Introduction

The global strategy for decarbonizing transport heavily relies on the shift toward electric mobility. This thesis investigates the complex challenges and opportunities arising from the large-scale integration of electric vehicles (EVs), as illustrated in Figure 1.1, into existing power grids.

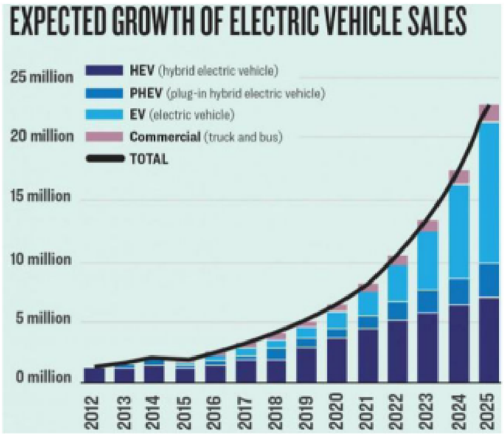


Figure 1.1: Expected growth of EV sales (Image from: [43]).

The chart [43] breaks down this growth into several categories:
1) HEV (hybrid electric vehicle)
2) PHEV (plug-in hybrid electric vehicle)
3) EV (electric vehicle)
4) Commercial (truck and bus)
It is stated that "EVs are growing very fast in the car market, and it is predicted to take a significant market share shortly."

1.1 Background and Relevance of Electric Vehicles and Vehicle-to-Grid

The rapidly expanding EV market is reshaping modern mobility, offering a path to reduced carbon emissions and enhanced energy efficiency [32]. This transition is fundamental to environmental sustainability, as it lessens dependence on fossil fuels, mitigates climate change by cutting greenhouse gas emissions, and improves urban air quality. However, integrating millions of EVs into the power system is a significant challenge, threatening to intensify peak demand, strain transmission and distribution networks, and cause technical issues like voltage irregularities or line losses [32, 37].

This challenge raises a critical question:

Can we transform this apparent liability into a foundational asset for grid stability? The answer may be found in the Vehicle-to-Grid (V2G) paradigm. V2G reimagines EVs not as passive loads, but as mobile, flexible energy assets capable of bidirectional power exchange [2]. This potential is significant, considering EVs remain parked for approximately 96% of the day, providing a vast window for grid interaction. Furthermore, the rapid responsiveness of EV batteries makes them ideal for providing ancillary services like frequency regulation [2]. A growing body of research, reviewed by Qiu et al. [34] and Xie [48], suggests that intelligent, bidirectional charging can offset the negative impacts of EV integration. The central proposition of this thesis is to test this hypothesis: that through advanced control methodologies, V2G can be proven not just theoretically sound, but practically indispensable, provided its economic and grid-stabilizing benefits demonstrably outweigh costs such as battery degradation and infrastructure investment.

Alongside V2G, other bidirectional power flow schemes, shown in Figure 1.2, have been proposed to enhance energy resilience:

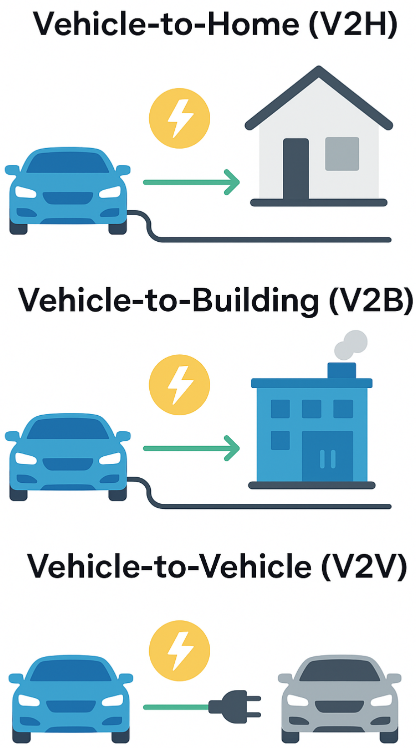


Figure 1.2: Other schemes of bidirectional power flow.

1. **Vehicle-to-Home (V2H):** An EV powers a household during outages or high-cost periods, boosting domestic energy security.

2. **Vehicle-to-Building (V2B):** This concept is extended to commercial or industrial facilities, where EVs support load management and optimize energy consumption.

3. **Vehicle-to-Vehicle (V2V):** Direct power transfer between EVs provides a solution for emergency charging or resource sharing.

Collectively, these modalities underscore the versatility of EV batteries as distributed energy resources, advancing the transition to a more sustainable energy ecosystem.

1.2 Challenges in EV Integration into the Electricity Grid and the Role of Artificial Intelligence

Modern electricity systems face increasing strain from the integration of intermittent **Renewable Energy Sources (RESs)** like wind and solar. The inherent variability of RESs leads to significant power generation swings, creating supply-demand mismatches that fuel price volatility and complicate grid management.

This continuous instability challenges the economic efficiency and reliability of the grid, proving difficult for conventional control frameworks to manage [32, 29]. The parallel rise of EV adoption and RES deployment has created an environment of unprecedented uncertainty and complexity. This situation makes traditional, rule-based controllers, designed for a more predictable and centralized grid, increasingly inadequate. **Is it possible, then, that a new control paradigm is needed?** The literature, as reviewed by NaXu et al. [49] and Feyijimi et al. [1], strongly suggests so, highlighting the limitations of legacy systems and framing the problem in a way that points toward data-driven methods. This shift signals a genuine paradigm change towards a *smart grid*, where adaptive, real-time, and autonomous operation is vital [49].
This has led to a surge in research focused on **Reinforcement Learning (RL)**, a paradigm that can, in theory, learn optimal control policies directly from environmental interaction without a perfect system model. While meta-heuristic algorithms have been explored, they often lack the real-time adaptability required for dynamic control [41]. The hypothesis to be tested is whether the theoretical promise of RL holds up against the engineering realities of the V2G problem. From this perspective, RL is not merely an optimization tool but an **enabling technology** for a more cognitive and robust energy infrastructure capable of navigating a decarbonized future. Within this domain, **Deep Reinforcement Learning (DRL)** has emerged as a particularly powerful approach, valued for its capacity to derive near-optimal strategies in dynamic and uncertain environments without relying on precise models or forecasts [32].

1.3 Objectives and Contributions of the Thesis

This thesis confronts the complex multi-objective optimization problem at the heart of V2G systems. The overarching objective is to move beyond a purely theoretical analysis by actively developing, testing, and enhancing a high-fidelity simulation architecture. This platform serves as a digital twin to rigorously evaluate and compare advanced control strategies, balancing economic benefits, user mobility needs, battery health, and grid stability under realistic stochastic conditions.
More than a simple review of existing literature, this work focuses on the practical implementation and validation of a V2G simulation framework in Python. This tool is leveraged to demonstrate and explore novel perspectives for training intelligent agents. The main contributions are:

- **Enhancement of a V2G Simulation Architecture:** A key contribution is the systematic testing, validation, and enhancement of the **EV2Gym** simulation framework. This work solidifies its role as a robust platform for benchmarking control algorithms and includes the development of an interactive data application using **Streamlit** for results visualization and scenario analysis.

- **Exploration of Novel RL Perspectives:** The validated simulation environment is used to investigate and implement advanced training methodologies for RL agents. A key focus is placed on techniques like **adaptive reward shaping**, where the reward function dynamically evolves during training to guide the agent towards a more holistic and robust control policy.

- **Practical Implementation and Comparison of MPC Formulations:** The thesis details the development and implementation of multiple model-based controllers. This includes a classic **implicit MPC**, and a novel **Adaptive Horizon Model Predictive Control (AHMPC)**, all formulated in PuLP.

1.3.1 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Overview of Optimal Management of EV Charging and Discharging** provides foundational knowledge on the V2G technology, the complex multi-objective nature of EV charging optimization, and presents a comprehensive review of state-of-the-art research approaches.
- **Chapter 3: The V2G Simulation Framework: A Digital Twin for V2G Research** details the architecture and core models of the simulation environment. This chapter describes the enhancements made to the framework, establishing it as the central experimental platform for implementing and evaluating the control agents analyzed in this work.
- **Chapter 4: Experimental Campaign and Results Analysis** presents the main results from the comparative analysis of the different control strategies (DRL, MPCs, heuristics). It analyzes the performance of novel training techniques, discusses the implications of the findings, and provides a detailed **sensitivity analysis** on the most critical system parameters to assess the robustness of the conclusions.
- **Bibliography** lists all cited references.

Chapter 2

Foundations and State of the Art in Intelligent Control for the V2G Systems

In order to provide an overview of the research domain, the literature review was complemented by a quantitative analysis of the papers collected for this thesis. Using a custom Python script, a citation graph was constructed, and the relevance of key terms was evaluated through PageRank scores. The outcome of this process is shown in Figure 2.1, which presents a weighted word cloud derived from the titles and abstracts of the reviewed works. In this visualization, the size of each word reflects its relative importance within the citation network.

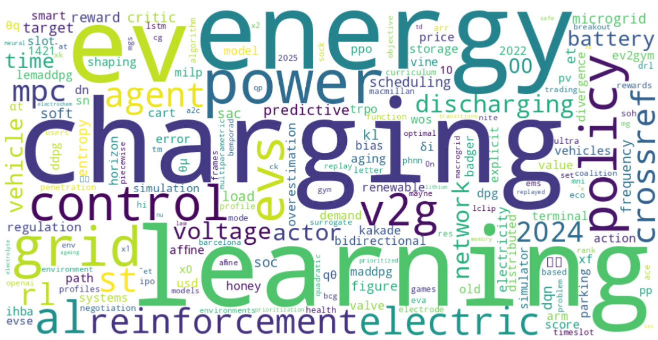


Figure 2.1: Weighted word cloud generated from the literature review, highlighting key research themes. The size of each word corresponds to its frequency and importance, derived from the PageRank analysis.

This visualization confirms the research focus, with primary terms like **Energy**, **Power**, **Grid**, and **EVs** setting the context. The similar prominence of **Learning** and **Control**, alongside strong secondary terms like **Reinforcement**, **Agent**, and **Policy**, firmly anchors the methodology in RL. The inclusion of **MPC** (Model Predictive Control) highlights the central dialogue between model-free and model-based approaches explored in this thesis.

2.1 The V2G Imperative: A Foundation of Europe's Green Transition

> *"The green transition is the most ambitious industrial transformation ever. The region of the world that develops clean technologies first will come out on top — and I want it to be Europe."*
>
> — Ursula von der Leyen

Europe finds itself at the confluence of two unprecedented and deeply interlinked transformations reshaping its technological, economic, and societal landscape: the large-scale electrification of transport and a comprehensive restructuring of its energy systems. These parallel transitions involve a radical shift from internal combustion engines to battery EVs and a simultaneous integration of renewable generation, grid modernization, and the deployment of advanced storage and demand-side management solutions.

These transformations are not merely aspirational targets but constitute binding legal obligations established under the **European Green Deal** and the detailed **"Fit for 55"** legislative package. These frameworks translate climate ambitions into enforceable measures aimed at reducing net greenhouse gas emissions by 55% by 2030 [12]. The Renewable Energy Directive RED II directive (2018/2001/EU) sets rules to promote the use of RESs and establish binding national targets for renewable energy in the European Union (as can be seen in Figure 2.2). This policy architecture necessitates the rapid phase-out of fossil-fuelled vehicles alongside a dramatic expansion of renewable energy capacity, a goal further reinforced by the revised RED III, as detailed in Figure 2.2. Figure 2.2 highlights the significantly increased ambition in RED III, including a higher overall renewable energy target, a more aggressive goal for the transport sector, and stricter requirements for traceability and legal enforceability. This escalation in policy ambition underscores the critical need for flexibility solutions like the V2G to manage the grid and integrate higher shares of renewables.

	RED II (2018)	RED III (2023)
RENEWABLE ENERGY TARGET	32% by 2030	42.5% by 2030 (45% aspirational target)
TRANSPORT SECTOR TARGET	14% renewable energy	29% renewable energy, 14.5% GHG intensity reduction in transport
GHG SAVINGS THRESHOLD FOR BIOFUELS	50–65% depending on installation date	70% (existing), 80% (new installations)
MASS BALANCE TRACEABILITY	Encouraged	Mandatory
ENFORCEABILITY	Partially voluntary or indicative	Legally binding and auditable
CHAIN OF CUSTODY SYSTEMS	Not required	Required across the entire value chain
ALIGNMENT WITH OTHER EU LEGISLATION	Limited	Integrated with ETS, CBAM, and EUDR frameworks

Figure 2.2: Comparison of key targets between the Renewable Energy Directive II (RED II, 2018) and the revised RED III (2023).

EVs occupy a central position in this transition, simultaneously driving decarbonisation efforts while presenting complex challenges for grid stability. The initial response to mass EV adoption was characterised by concern within the power sector, viewing millions of new EVs as vast, correlated loads threatening to overwhelm distribution networks. This perspective has undergone a fundamental reassessment. EVs are now recognised not as burdens to be managed, but as essential, flexible assets for achieving Europe's energy objectives.

This conceptual shift finds its most concrete expression in the V2G technology, which fundamentally reimagines the role of EVs within the energy system.

The V2G technology transforms previously passive, unidirectional energy consumers into active, distributed, and intelligent grid resources. The underlying opportunity is substantial: private vehicles spend approximately 96% of their operational lifetime parked [14], representing an enormous, geographically distributed, and currently underutilised repository of mobile energy storage.

The transformative potential of V2G becomes apparent when individual vehicles are coordinated through centrally managed aggregation. While a single EV's contribution is modest, an orchestrated fleet can function as a unified **Virtual Power Plant (VPP)**. These software-defined power plants aggregate the collective capacity of numerous distributed energy resources, delivering grid services at scales comparable to conventional generation facilities. The rapid response characteristics of contemporary battery inverters, operating at millisecond timescales, enable these aggregated fleets to provide a comprehensive range of critical grid services. This capability is a prerequisite for maintaining stability in grids increasingly dependent on variable wind and solar generation, thereby enabling the technical and economic viability of the EU's ambitious renewable energy targets [43]. Figure 2.3 illustrates the multifaceted roles an EV fleet can play, orchestrated by an aggregator using RL. Key operations include gridstabilizing services (frequency control, demand response), economic optimization (bidding and pricing strategies), and user-centric management (energy storage, battery health). The objectives range from reducing electricity costs and enhancing grid flexibility to ensuring data privacy and transaction security, showcasing the complexity of optimizing the V2G operations.

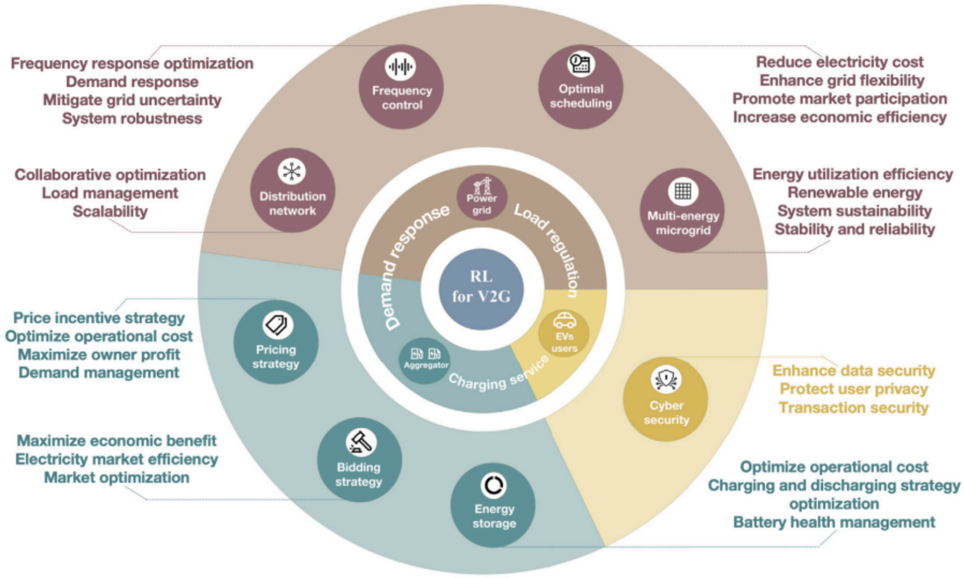


Figure 2.3: RL applications in the V2G context from the perspective of participating entities.

The grid services enabled by the V2G technology, many of which are illustrated in Figure 2.3, form the technical foundation for the smart, resilient, and decarbonised electricity system required for Europe's energy future:
Frequency Regulation: Grid stability depends on maintaining a precise equilibrium between electricity supply and demand, manifested as a stable grid frequency (50 Hz in Europe). Deviations signal imbalances that can trigger cascading failures. The V2G fleets, with their rapid-response inverters, can participate in ancillary service markets like Frequency Containment Reserve (FCR) and automatic Frequency Restoration Reserve (aFRR), injecting or absorbing power within seconds to counteract deviations and prevent blackouts [2, 47].
Demand Response and Peak Shaving: By intelligently shifting charging to off-peak periods and strategically discharging during peak demand, the V2G systems flatten daily load profiles. This directly mitigates the "duck curve" phenomenon illustrated in Figure 2.4 for the case of California, which is typically associated with high solar penetration. Such load management reduces reliance on expensive, carbon-intensive "peaker" plants and can defer or eliminate costly grid infrastructure upgrades [32, 36]. Net load is the total electricity demand minus variable renewable energy generation. The deepening "belly" of the duck during midday reflects overgeneration from solar power, while the steep "neck" in the evening represents a rapid increase in demand as solar generation fades. This steep ramp poses a significant challenge for grid operators. The V2G helps mitigate this by absorbing excess energy during the day (charging) and discharging it during the evening ramp to "flatten the curve" [7].

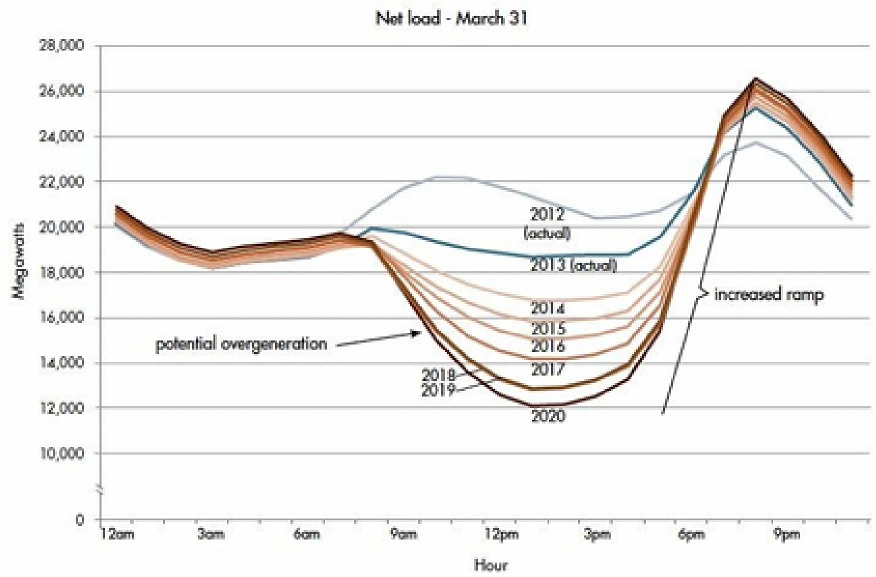


Figure 2.4: The "duck curve" illustrating the evolution of net load on the California grid.

Renewable Energy Integration: The most strategically significant contribution of the V2G is addressing renewable energy intermittency.The V2G fleets act as large-scale energy buffers, absorbing excess solar and wind generation that would otherwise be curtailed. This stored energy is then released during periods of low generation. This mechanism directly increases the utilisation of RESs.
Economic and Market Optimization: Beyond grid stability, the V2G enables sophisticated market participation. As shown in Figure 2.3, aggregators can deploy intelligent bidding and pricing strategies. RL algorithms can learn optimal policies for participating in day-ahead and intraday energy markets, maximizing revenue for both the aggregator and EV owners by capitalizing on price volatility.
User-Centric Management and Security: For V2G to succeed, it must align with user needs and address concerns. This includes optimizing charging and discharging cycles to minimize **battery degradation**, thus preserving vehicle lifespan. Furthermore, as the V2G involves financial transactions and data exchange, robust **cyber security** is paramount to protect user privacy and ensure transaction security.
This vision is being actively incorporated into European legal frameworks. The transformative **Alternative Fuels Infrastructure Regulation (AFIR, EU 2023/1804)** now mandates that new public charging infrastructure incorporate smart and bidirectional capabilities.

2.1.1 ISO 15118-20

The V2G is supported by technical standards like **ISO 15118-20**, which specifies protocols for a "Vehicle-to-Grid Communication Interface" (V2GCI). With mandatory implementation scheduled for 2027, the necessary infrastructure is being systematically established, supported by pilot initiatives like the **'SCALE'** and **'V2G Balearic Islands'** projects.

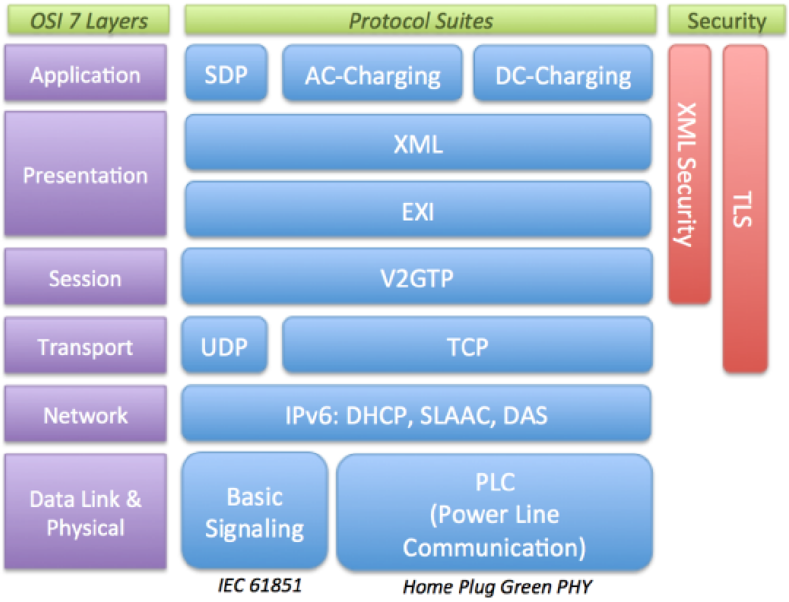


Figure 2.5: The protocol stack of ISO/IEC 15118 [40].

ISO 15118-20 is the latest part of the ISO 15118 family (In Figure 2.5 the protocol stack of ISO/IEC 15118), defining the communication between an EV and the Electric Vehicle Supply Equipment (EVSE). It extends previous versions (like ISO 15118-2) by introducing new functions, enhanced security, and full support for bidirectional power transfer (V2G). The goal of the protocol is to create a standardized, secure, and automatic data exchange that allows an EV and the charging station to negotiate how, when, and how much energy to transfer. Communication is established over Power Line Communication (PLC), using a TCP/IP stack over IPv6. The protocol uses TLS encryption to protect confidentiality and authenticity of the session. Once the physical link is established, the EV and the EVSE start a handshake to exchange certificates and initialize the communication channel. The EV then sends a *ServiceDiscovery* message to check which services the EVSE supports, such as alternating current charging, direct current charging, or bidirectional energy flow. After service negotiation, the two parties exchange *Authorization*, *ChargeParameterDiscovery*, and *PowerDelivery* messages to determine the charging profile, power limits, and time schedules. One of the main features of ISO 15118-20 is the *Plug & Charge* mechanism. This allows the EV to authenticate automatically using digital certificates issued by a trusted authority, removing the need for RFID cards or apps. Once authenticated, the EVSE can directly start charging and manage billing information securely. The protocol also supports *Bidirectional Power Transfer*, enabling the V2G communication, where the EV can return energy to the grid under controlled conditions. All messages are encoded in XML or EXI format and structured according to strict data models to ensure interoperability. In addition, ISO 15118-20 defines methods for smart charging, tariff and schedule negotiation, and the exchange of metering data. It provides flexibility for different energy markets, supporting both local and remote authorization modes. Compared to previous standards, ISO 15118-20 introduces stricter cybersecurity requirements, mandatory TLS usage, improved message structures, and extended functionality for wireless and high-power direct current charging.

2.1.2 The V2G obstacles

Despite the progress achieved in the V2G context, substantial obstacles to widespread V2G deployment persist:

- **Market and Economic Barriers:** A coherent, pan-European framework for compensating EV owners for grid services is still undeveloped. Accessing existing value streams is complex, and issues like the **"double taxation"** of electricity—taxing energy during both charging and discharging—create significant economic disincentives.
- **Regulatory and Grid Access Challenges:** The qualification of EV fleets as flexibility resources varies across national markets. Standardised procedures for grid interconnection, aggregator certification, and secure data exchange are needed to reduce fragmentation and simplify commercial deployment.
- **Technical and Consumer Adoption Barriers:** Consumer concerns regarding accelerated **battery degradation** and its impact on vehicle warranties are primary obstacles. Furthermore, much of the current EV and charging infrastructure lacks bidirectional capability, though this is changing with new vehicle platforms and standards.

The fundamental challenge addressed by this thesis extends beyond simply enabling V2G technology to encompass its *intelligent orchestration*. This requires developing control strategies sophisticated enough to operate within emerging regulatory frameworks, navigate economic uncertainties, accommodate diverse user preferences, and overcome technical constraints. The objective is to unlock the substantial potential of EVs as fundamental components of Europe's energy transition while ensuring system reliability, economic viability, and user acceptance.

2.1.3 The Optimizer's Trilemma: Navigating a Stochastic World

While the potential of V2G technology is substantial, the management of distributed vehicular assets presents a complex control challenge. Economic viability drives aggregator decisions, yet a narrow focus on profitability alone proves insufficient for sustainable operations.
The management of the V2G systems can be viewed through what may be called the "V2G Optimizer's Trilemma", which reflects the need to balance three often conflicting objectives: economic profitability, battery longevity, and user convenience. Rather than representing a straightforward, static trade-off, this constitutes a dynamic, multi-objective optimisation challenge characterised by **stochasticity** and **uncertainty** arising from multiple, interconnected sources [46]:

Market Volatility: Wholesale electricity prices exhibit significant variability driven by unpredictable supply variations (such as sudden reductions in wind generation capacity) and demand fluctuations (including heat-driven increases in cooling demand). Effective control systems must respond to these price signals dynamically and in real-time.
Renewable Intermittency: Co-located solar and wind generation exhibit inherently variable output patterns with limited predictability. Controllers must coordinate EV fleet operations to capture available generation during surplus periods without compromising other operational objectives.
Human Behaviour: Perhaps the most challenging uncertainty source involves EV owner patterns. Arrival times, departure schedules, and required SoC at departure lack deterministic characteristics. Emergency departures or unexpected schedule changes represent hard, non-negotiable constraints that intelligent systems must accommodate to preserve user trust and satisfaction.
This dynamic, uncertain, and multifaceted operational environment renders static, rule-based control approaches inadequate and brittle. More sophisticated and adaptive methodologies are required such as approaches capable of learning from operational experience and making optimal decisions under conditions of significant uncertainty.

2.1.4 Sources for Energy Price Data

Access to reliable, real-time, and historical market data remains crucial for both control agent training in simulation environments and real-world deployment. Key public sources for European market data include:
ENTSO-E Transparency Platform: The European Network of Transmission System Operators for Electricity maintains a mandatory, open-access platform serving as a comprehensive repository of pan-European electricity market data. This includes harmonised day-ahead prices, load forecasts, and generation data, serving as the primary source for academic research through both web portal access and free RESTful API services.
National Transmission System Operators (TSOs): Many national TSOs (including Terna) in Italy, National Grid in the UK, and RTE in France) publish detailed market data covering real-time frequency and imbalance prices for their respective jurisdictions.
Power Exchanges: Exchanges such as **EPEX SPOT** and **Nord Pool** constitute actual trading venues. While they represent direct price data sources, comprehensive real-time access typically requires commercial subscription services.

2.1.5 Buying vs. Selling: The Critical Retail-Wholesale Spread

A critical yet frequently overlooked aspect of the V2G economics involves the distinction between EV owner charging costs and aggregator grid sales revenue.

- **Selling Price (V2G Revenue):** When EVs provide energy to the grid, revenue calculation bases on **wholesale prices** (such as day-ahead spot prices). These prices reflect pure marginal energy costs at specific times.

- **Buying Price (Charging Cost):** End consumer EV charging costs reflect **retail prices**, significantly exceeding wholesale prices due to numerous non-energy components, termed "non-commodity costs":
  - Base wholesale energy costs
  - **Grid Tariffs:** Charges for high-voltage transmission and low-voltage distribution network usage
  - **Taxes and Levies:** National or regional taxation including VAT and environmental levies applied to electricity consumption
  - **Supplier Margin:** Retail energy provider profit margins

This substantial gap between retail purchasing prices and wholesale selling prices constitutes the "retail-wholesale spread", creating the primary opportunity for profitable energy arbitrage. Successful control strategies must account for these price differentials to enable economically rational decision-making.

2.1.6 Modelling the V2G Ecosystem

Before examining control algorithms, establishing clear, high-fidelity models of system core components becomes essential: the EVs as a controllable cyber-physical asset, and the operational environment or "scenario". The interaction between these elements defines the V2G optimisation task boundaries and objectives.

From a power grid perspective, EV represent sophisticated mobile energy storage devices. For the V2G applications, EVs can be characterised through several key state variables and parameters:

The Battery: The core grid asset component, defined by **nominal energy capacity** (in kWh), current **SoC**, and **State of Health (SoH)** representing degradation over time. Operation is constrained by **power limits** (in kW) dictating maximum charge or discharge rates, and charging/discharging **efficiencies** accounting for energy losses.

The On-Board Charger (OBC): For AC charging applications, the OBC converts grid alternating current to battery direct current. Power rating often constitutes the primary bottleneck for both charging and the V2G power output.

Communication Interface: The V2G participation requires vehicle-charging station (EVSE) communication capabilities. This is governed by standards including **ISO 15118** and protocols such as the **Open Charge Point Protocol (OCPP)**, enabling secure information exchange required for smart and bidirectional power flow operations.

Combined with vehicle availability patterns—arrival and departure times plus user energy requirements,these characteristics transform EVs from simple loads into fully dispatchable grid resources.

2.2 Reinforcement Learning

"RL is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal."

— Richard S. Sutton & Andrew G. Barto, *Reinforcement Learning: An Introduction* (2018) [42]

To address the complexities of uncertainty, multi-objective trade-offs, and dynamic systems, this work employs RL as a core methodology, in addition to Model Predictive Control (MPC) methods, which are presented in another part of this work. RL is a machine learning paradigm that learns optimal sequential decision-making policies through trial-and-error interaction with an environment. Unlike traditional optimal control methods, which rely on an explicit and accurate model of the system dynamics, this model-free approach offers significant robustness against uncertainty and unmodeled behaviors, enabling adaptive control in complex and dynamic environments.

2.2.1 The Agent-Environment Interface

As shown in the Figure 2.6 the problem of RL is formalized as the interaction between a learning **agent** and its **environment**. This interaction unfolds over a sequence of discrete time steps, $t = 0, 1, 2, \ldots$. At each time step t, the agent receives a representation of the environment's **state**, $S_t \in \mathcal{S}$, and on that basis selects an **action**, $A_t \in \mathcal{A}(S_t)$. One time step later, as a consequence of its action, the agent receives a numerical **reward**, $R_{t+1} \in \mathcal{R}$, and finds itself in a new state, S_{t+1}. This interaction loop forms the foundational framework of the RL problem.

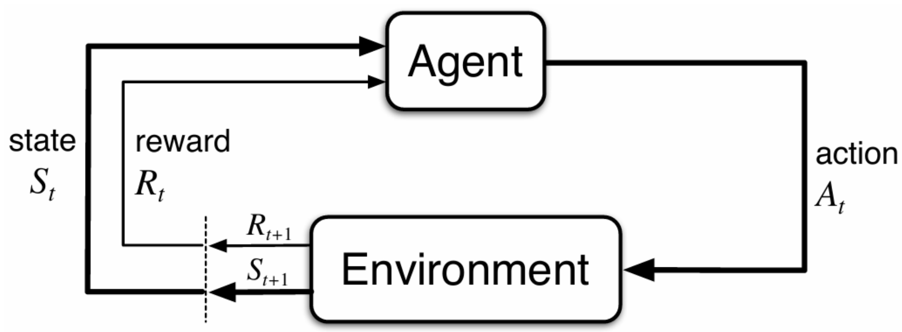


Figure 2.6: The agent-environment interaction loop in RL [42].

2.2.2 Goals, Rewards, and Returns

The agent's objective is formalized by the **reward hypothesis**: that all goals and purposes can be framed as the maximization of the expected cumulative reward. The agent's goal is not to maximize the immediate reward, R_{t+1}, but the cumulative reward in the long run. This cumulative reward is known as the **return**, denoted

G_t.
For *episodic tasks* that terminate, the return is the finite sum of future rewards. For *continuing tasks* that do not terminate, the return is defined as the infinite discounted sum of future rewards:

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.1}$$

where $\gamma \in [0, 1)$ is the **discount factor**. It determines the present value of future rewards, ensuring the infinite sum is finite and balancing immediate gratification against long-term gains. A value of $\gamma = 0$ results in a myopic agent concerned only with maximizing immediate rewards.

2.2.3 The Language of Learning: Markov Decision Processes

The mathematical foundation of RL is the **Markov Decision Process (MDP)**. An MDP provides a formal framework for modeling decision-making in stochastic environments where outcomes are partly random and partly under the control of a decision-maker.
An MDP is formally defined as a tuple $\langle S, A, P, R \rangle$, where:

- S is a finite set of states.
- A is a finite set of actions.
- P is the state transition probability function, $P(s'|s, a)$, which represents the probability of transitioning to state s' from state s after taking action a.
- R is the reward function, $R(s, a, s')$, which is the immediate reward received after transitioning from state s to state s' as a result of action a.

The future is independent of the past given the present. A state signal S_t is said to have the **Markov Property** if the environment's response at time $t + 1$ depends only on the state and action at time t. The probability of transitioning to state s' and receiving reward r is independent of all previous states and actions:

$$p(s', r|s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a\} \tag{2.2}$$

This property is fundamental as it allows decisions to be made based solely on the current state, without needing the complete history of interaction.
A key assumption in an MDP is that the environment is fully observable, meaning the agent knows its current state with certainty. However, in many real-world scenarios, the agent may not have complete information about its state. This is where the **Partially Observable Markov Decision Process (POMDP)** comes into play.
A POMDP extends the MDP framework to situations where the agent's observations of the environment are incomplete or noisy. A POMDP is represented by a tuple $\langle S, A, P, R, \Omega, O \rangle$, which includes all the elements of an MDP plus:

- Ω is a finite set of observations the agent can receive from the environment.
- O is the observation probability function, $O(o|s', a)$, which is the probability of observing o after transitioning to state s' having taken action a.

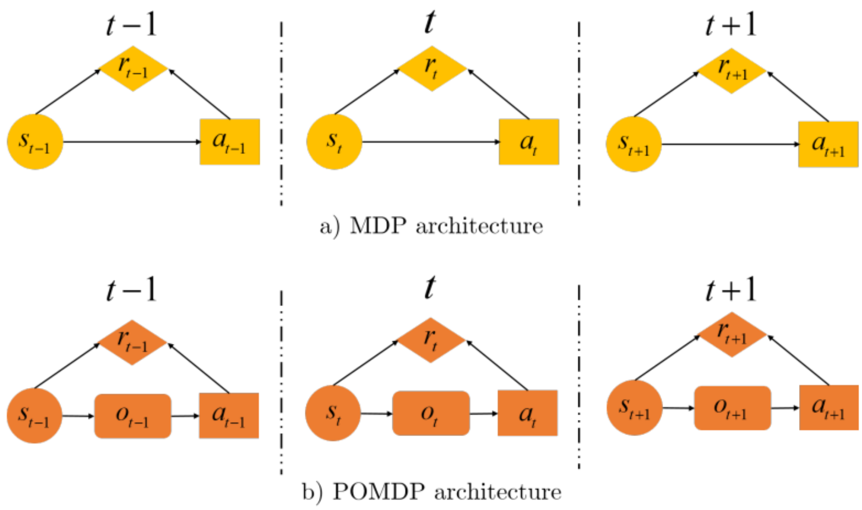


Figure 2.7: Differences between POMDP and MDP [36].

As shown in Figure 2.7 the fundamental difference between an MDP and a POMDP lies in the agent's perception of the environment's state. In an MDP, the agent directly observes the state s. In a POMDP, the agent receives an observation o and must infer a belief, which is a probability distribution over the possible states, to make a decision. As discussed in Sadeghi's work, this partial observability in POMDPs introduces a significant layer of complexity because the agent must act based on a belief state rather than a certain state [36].
In our current thesis, we will focus on the MDP framework, assuming that the state of the environment is fully observable to the agent.

2.2.4 Policies and Value Functions

The agent's learning objective is to find a good **policy**, $\pi(a|s)$, which is a mapping from states to probabilities of selecting each possible action. The goodness of a policy is assessed by its **value functions**.

- The **state-value function**, $v_\pi(s)$, is the expected return starting from state s and following policy π thereafter:

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t|S_t = s] \tag{2.3}$$

- The **action-value function**, $q_\pi(s, a)$, is the expected return starting from state s, taking action a, and thereafter following policy π:

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t|S_t = s, A_t = a] \tag{2.4}$$

2.2.5 The Bellman Equations

The Bellman equations provide a recursive decomposition that is foundational to solving MDPs. They express the value of a state in terms of the values of its successor states. For a given policy π, the state-value function must satisfy a self-consistency condition. The value of a state equals the expected immediate reward plus the discounted expected value of the next state:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v_\pi(s')] \tag{2.5}$$

This is the **Bellman expectation equation** for v_π. It forms the basis for policy evaluation algorithms. The ultimate goal is to find an **optimal policy**, π_*, which is a policy that achieves a higher or equal expected return than all other policies from all states. All optimal policies share the same optimal value functions, $v_*(s)$ and $q_*(s, a)$. The optimal value function for a state is the maximum expected return achievable from that state:

$$v_*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1})|S_t = s, A_t = a] = \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v_*(s')] \tag{2.6}$$

This is the **Bellman optimality equation**. Solving it means finding the optimal policy.

2.2.6 Generalized Policy Iteration (GPI)

Most RL algorithms can be understood within the framework of **Generalized Policy Iteration (GPI)**, as described in Chapter 4 of the book: [42]. GPI refers to the general idea of letting two interacting processes, policy evaluation and policy improvement, work towards a common optimal solution.

- **Policy Evaluation**: Given a policy π, compute its value function v_π. This step aims to make the value function consistent with the current policy.

- **Policy Improvement**: Given a value function v, improve the policy by making it greedy with respect to v. For a given state s, the new policy will select the action a that maximizes $q_\pi(s, a)$.

These two processes compete in the short term (improving the policy makes the value function inaccurate) but cooperate in the long term to converge to the optimal policy and optimal value function. **Dynamic Programming (DP)** methods like policy iteration and value iteration are classic examples of GPI, assuming a perfect model of the environment. In addition to DP methods based on a perfect model of the environment, there are also **value-based** techniques that fit within the GPI framework but do not require complete knowledge of the dynamics of the environment.

In these methods, the focus is primarily on learning the value function (such as v_π or q_π), and the policy is implicitly derived from it, usually by making it *greedy* with

respect to the learned value function. Typical examples include **Q-Learning** and **SARSA**. These algorithms iteratively update value estimates based on experiences gathered from interacting with the environment, without the need for a complete model.
The main distinction between Policy-Based and Value-Based is that in *policy-based* methods, the policy is directly parameterized and optimized via gradient or other optimization methods (e.g., *REINFORCE*, *Actor-Critic*), while in *value-based* methods, the policy is derived from the value function. In other words, value-based methods use value as an intermediary to improve the policy, while policy-based methods update the policy directly.

2.2.7 Learning from Experience: Monte Carlo (MC) and Temporal-Difference (TD) Methods

When a model of the environment is not available, we must learn from sampled experience. Chapters 5 and 6 of the book [42] introduce two primary model-free approaches.

MC Methods

MC methods learn value functions by averaging the returns from sample episodes.

- **Principle**: An update to $V(S_t)$ is made only at the end of an episode.
- **Update Target**: The target for the update is the actual, complete return G_t.
- **Update Rule** (Constant-α): $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$
- **Properties**: MC methods are unbiased but can have high variance and are only applicable to episodic tasks. They do not *bootstrap*: the updates comes from only complete sampled returns.

TD Learning

TD learning is a central and novel idea in RL, combining ideas from both MC and DP.

- **Principle**: TD methods update the value estimate for a state based on the observed reward and the estimated value of the successor state. They learn from incomplete episodes.
- **Update Target**: The target is an estimate of the return, called the TD Target: $R_{t+1} + \gamma V(S_{t+1})$.
- **Update Rule**: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$
- **Properties**: TD methods *bootstrap*: they update a guess from a guess. This introduces bias but often leads to lower variance and faster learning. They are naturally implemented in an online, fully incremental fashion.

In practice,**most RL algorithms prefer TD methods over MC methods**. This is because TD allows incremental step-by-step updates, without having to wait for the end of the episode, and is therefore suitable for both long episodic and continuous tasks. Furthermore, bootstrapping reduces variance and makes learning more sample-efficient, combining the advantages of DP and real-world learning.

2.2.8 Q-Learning: Off-Policy TD Control

Building directly upon the principles of TD learning, Q-Learning stands as one of the most foundational and influential algorithms in RL. It is an **off-policy** TD control algorithm that directly learns the optimal action-value function, $q_*(s, a)$, independent of the policy being followed during exploration. This separation of the policy being learned from the policy used to generate experience is a hallmark of off-policy methods and allows for more flexible exploration strategies.
For environments with discrete state and action spaces, Q-Learning maintains a table of Q-values, often referred to as the Q-table, with an entry for every state-action pair. The learning process involves iteratively updating these Q-values using the Bellman Optimality Equation 2.6 as a foundation, but in a model-free context. After the agent takes an action A_t in state S_t and observes the reward R_{t+1} and the next state S_{t+1}, the Q-value is updated according to the following rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{2.7}$$

Here, α is the learning rate, which determines how much the new information overrides the old estimate. The core of the update is the TD error, calculated from the TD target $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$. The term $\max_a Q(S_{t+1}, a)$ represents the agent's current best estimate of the optimal future value from the next state S_{t+1}. By always using the maximum Q-value of the next state, the algorithm learns a greedy, optimal policy, regardless of what exploratory action is actually taken next.
The primary limitation of tabular Q-Learning, however, is its inability to scale to problems with large or continuous state spaces, a problem often called the "curse of dimensionality". Storing and updating a massive Q-table becomes computationally and memory-prohibitive. This challenge directly motivates the use of function approximators, such as neural networks, to estimate the Q-function, leading to the development of DQN.

2.2.9 Actor-Critic Architectures

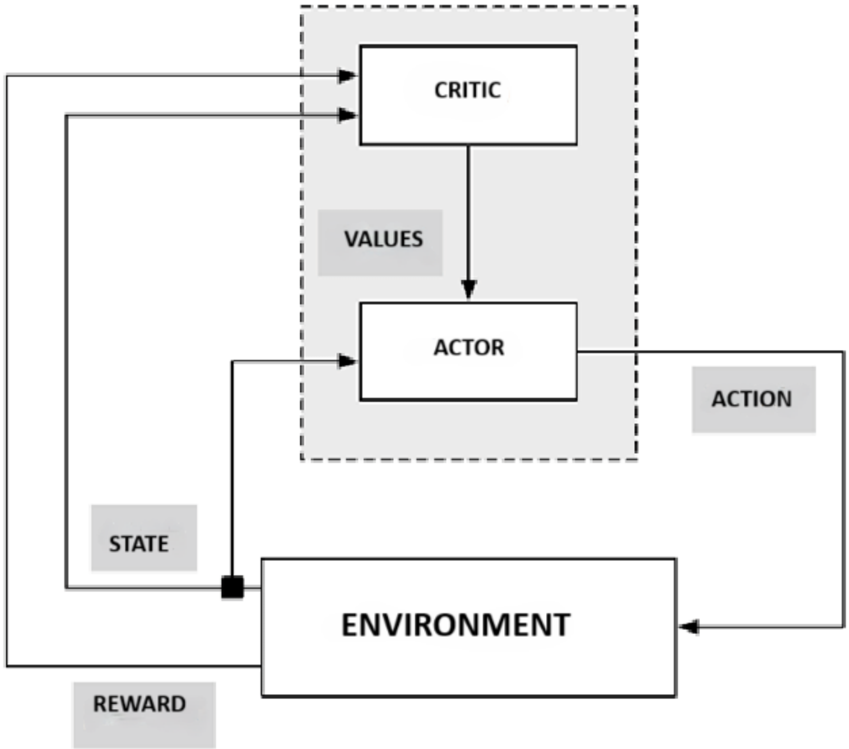


Figure 2.8: The Actor Critic scheme.

The **Actor-Critic** architecture shown in the Figure 2.8 provides a powerful and widely adopted method for solving RL problems, particularly in continuous action spaces. It explicitly represents both the policy and the value function using two distinct function approximators (e.g., neural networks).

- **The Critic**: Learns a value function (e.g., $v_\pi(s)$ or $q_\pi(s, a)$). Its role is to evaluate the actor's decisions by computing the TD error:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \tag{2.8}$$

- **The Actor**: Represents the policy, $\pi_\theta(a|s)$, parameterized by θ. It receives the current state as input and outputs an action. It uses the TD error from the critic as a learning signal to update its parameters, improving its strategy over time.

This separation of concerns allows for direct policy optimization in continuous or large action spaces while leveraging the stable learning dynamics of TD-based value estimation.

2.3 Reward Engineering: Shaping Agent Behavior

The design of an effective reward function is arguably the most critical aspect of any RL system. It serves as the primary communication channel through which designers convey desired behaviors and objectives to a learning agent. A poorly designed reward function can lead to agents learning unintended, suboptimal, or even harmful behaviors, despite successfully maximizing their assigned objective. Consequently, reward engineering has emerged as a fundamental and increasingly sophisticated discipline within modern RL, proving indispensable for the successful application of algorithms in complex, real-world scenarios [20]. The V2G challenge, with its inherent multi-objective nature, encompassing profit maximization, assurance of user satisfaction, preservation of battery health, and maintenance of grid stability—presents particularly stringent demands on reward function design.

2.3.1 Potential-Based Reward Shaping

Potential-Based Reward Shaping (PBRS) stands as one of the most theoretically grounded and widely adopted methods for augmenting an environment's intrinsic reward signal. The core idea behind PBRS is to supplement the original reward, $R(s, a, s')$, received by an agent for transitioning from state s to state s' via action a, with an additional shaping term, $F(s, a, s')$. The resulting shaped reward, R', is defined as:

$$R'(s, a, s') = R(s, a, s') + F(s, a, s') \tag{2.9}$$

The crucial characteristic of PBRS lies in the specific construction of the shaping term. To guarantee that the optimal policy remains unchanged (a property known as policy invariance), this shaping term must be defined as the difference in value of an arbitrary **potential function**, $\Phi : S \to \mathbb{R}$, evaluated at the successive states s and s':

$$F(s, a, s') = \gamma\Phi(s') - \Phi(s) \tag{2.10}$$

where $\gamma \in [0, 1)$ is the discount factor. The potential function $\Phi(s)$ assigns a scalar value to each state, intuitively representing how "good" or "desirable" that state is. Although F is formally a function of s, a, s', this specific potential-based form makes its value independent of the action a, which is the key insight for guaranteeing policy invariance.

2.3.2 Theoretical Foundation and Policy Invariance

The work by Ng et al. [31] established the theoretical robustness of PBRS by proving a critical property: **policy invariance**. This property guarantees that adding a potential-based shaping reward to an MDP does not alter its set of optimal policies. In other words, **any policy that is optimal for the shaped reward function R' will also be optimal for the original reward function R, and vice versa**. This is a profound result because it ensures that while PBRS

can significantly accelerate learning by providing denser and more informative feedback, it will not mislead the agent into converging on a suboptimal policy with respect to the original task.
The proof of policy invariance hinges on the observation that the shaping term $F(s, s')$ can be absorbed into the value function. Specifically, if $V^*(s)$ and $Q^*(s, a)$ are the optimal value and Q-functions for the original MDP with reward R, then the optimal value and Q-functions for the shaped MDP with reward R' are given by:

$$V^{*'}(s) = V^*(s) + \Phi(s)$$
$$Q^{*'}(s, a) = Q^*(s, a) + \Phi(s)$$

Since the $\Phi(s)$ term is added uniformly to all Q-values for a given state s, the action that maximizes $Q^{*'}(s, a)$ will be the same action that maximizes $Q^*(s, a)$. This preserves the optimal policy:

$$\pi^{*'}(s) = \arg\max_{a \in \mathcal{A}} Q^{*'}(s, a) = \arg\max_{a \in \mathcal{A}}(Q^*(s, a) + \Phi(s)) = \arg\max_{a \in \mathcal{A}} Q^*(s, a) = \pi^*(s)$$

2.3.3 Practical Implications and Design Considerations

PBRS offers several practical advantages due to its policy invariance. It accelerates learning by providing immediate shaping rewards, reducing reward sparsity and speeding up convergence, particularly in tasks with delayed rewards. Exploration is safer, as the agent is guided toward promising states, decreasing the likelihood of undesirable behaviors or local optima during early training. Expert knowledge can be integrated through the potential function $\Phi(s)$, which may encode task-specific insights such as distance to a goal in navigation tasks or battery/-grid considerations in V2G applications. Effective design of $\Phi(s)$ is crucial and can follow strategies such as distance-based potentials, subgoal-based rewards for intermediate objectives, or feature-based functions capturing relevant state characteristics.
Dynamic or adaptive reward functions evolve during training to address shifting objectives or environmental changes. They help agents manage multi-objective tasks by initially emphasizing simpler goals, gradually introducing more complex objectives as proficiency improves. Conflicting goals can be mitigated by prioritizing certain objectives early on, while adaptive mechanisms, including time-varying weights, curiosity-driven rewards, reward scaling, or meta-learning approaches, allow the reward function to adjust based on agent performance and environmental conditions. In V2G, for instance, an agent may initially be rewarded for connecting to the grid and maintaining minimum charge, while later rewards emphasize grid stability or profitable energy transactions, guiding learning toward comprehensive V2G management.

2.4 Curriculum Learning

Curriculum learning (CL) is described as a strategy where the same model is trained on scenarios of increasing difficulty, starting from the easiest and progressing to the most difficult. This general idea, however, is implemented in different ways. The research by Pocius et al. [33] focuses on a CL based on **task simplification**, whereas Freitag et al. [15] propose a more specific and advanced approach based on **reward function simplification**.

2.4.1 Specific Principles and Applications

The core principle of CL is to guide the agent's learning to prevent it from being overwhelmed by the full complexity of the final problem. **Pocius et al.** [33] empirically compared curriculum learning, reward shaping, and visual hints in a navigation task within a Minecraft environment [33]. Their CL involved training the agent first on a simpler task (navigating a single room) before moving to a more complex one (navigating through two rooms). Their key finding was that, for their specific task, **CL** had the most significant impact on performance, surpassing the effectiveness of reward shaping. This suggests that for certain problems, structuring the learning experience through progressively harder tasks is a more powerful strategy than finely tuning the immediate reward signal [33]. On the other hand, **Freitag et al.** [15]addressed the problem of complex reward functions with multiple and potentially conflicting terms, which often lead agents to get stuck in local optima (e.g., an agent learning to satisfy a constraint without completing the main objective). To solve this, they proposed a **two-stage reward curriculum**:

1. **Stage 1:** The agent is trained using only a subset of the reward function, termed the "base reward" (r_b), which encodes the primary task objective.

2. **Stage 2:** Once the agent has sufficiently learned the basic task, the curriculum switches to training on the full reward function, which also includes the constraint terms (r_c).

One of their key innovations is a mechanism to **automatically switch from one stage to the next** by monitoring how well the actor's policy fits the critic's Q-function. They demonstrated that this approach is particularly effective when constraints have a high weight, as it prevents the agent from being "distracted" by the constraints before understanding the main goal, leading to more stable and higher-performing final policies [15].

2.4.2 Curriculum Learning in V2G

For the V2G challenge, the two-stage reward curriculum approach is particularly suitable, given the multi-objective nature of the problem. A curriculum could be structured as follows:

1. **Stage 1: Basic Charge Management.** The agent is trained with a simplified reward function (the "base reward," r_b) that solely rewards maintaining the charge levels of EVs above a minimum threshold. All other objectives, such as grid interaction or profit, are ignored.

2. **Stage 2: Full Optimization.** Once the agent has effectively learned to manage charging, it transitions to the full reward function. This includes the base reward plus the "constraint reward" terms (r_c), which introduce incentives for grid stability, cost minimization, and adherence to user preferences.

This structured approach, as demonstrated by Freitag et al., prevents the agent from being overwhelmed by conflicting objectives from the start, promoting the development of more robust and generalizable V2G management policies [15].

2.5 The Rise of Deep Reinforcement Learning for the V2G Control

The convergence of RL with the substantial representational capabilities of deep neural networks (DRL) has given rise to DRL, which currently represents the leading edge of the V2G control research. The development of DRL algorithms has yielded a comprehensive toolkit, primarily divided into two main families: off-policy and on-policy methods, each exhibiting distinct operational characteristics.

2.5.1 Neural Networks as Function Approximators

Neural networks are advanced computational models whose architecture is inspired by the complex and interconnected structure of neurons in the human brain. Their primary function is to recognize and learn complex patterns within data, making them extremely powerful tools for tasks such as classification, regression, and, as we will see, control.
A neural network is composed of layers of interconnected nodes, commonly called *neurons*. Each neuron acts as a small processing unit: it receives a series of inputs, each of which is multiplied by a specific weight. These weights represent the importance or strength of the connection between neurons. Subsequently, all weighted products are summed, and a *bias* value is added to this sum. The bias allows the neuron to activate even in the absence of significant inputs or to adjust the activation threshold. The result of this weighted sum and bias is then passed through a *non-linear activation function* to produce the neuron's output.
This process can be described mathematically for a single neuron as follows:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{2.11}$$

Here, y represents the output of this individual neuron. The x_i are the inputs that the neuron receives, and each of them is multiplied by its corresponding weight w_i. The term b is the bias, which is added to the weighted sum. The entire expression $\left(\sum_{i=1}^{n} w_i x_i + b\right)$ is often called the "net input" or "weighted sum." This result is then transformed by the activation function, f.
In the context of RL, while this equation describes a single neuron's operation, a full neural network (especially a DNN) is composed of many such neurons

arranged in layers. The **inputs** x_i to the first layer of the network typically represent the **state of the environment,** s. This state can be a vector of numerical features, raw pixel values from an image, or any other data representation describing the agent's current situation. The **final output** y of the entire neural network, which is the result of many interconnected neurons performing the operation described above, will vary depending on the specific RL algorithm:

- In **Deep Q-Networks (DQN)**, the network's output layer produces the **estimated Q-values for each possible action** a in the given state s. So, if there are N discrete actions, the network will have N output neurons, and their collective outputs represent $Q(s, a_1), Q(s, a_2), \ldots, Q(s, a_N)$.
- In **Policy Gradient methods**, the network's output layer typically produces a **probability distribution over the possible actions** for the given state s. Each output neuron corresponds to an action, and its output (often after a softmax activation) represents the probability $\pi(a_i|s)$ of taking action a_i in state s.

The role of the activation function is of fundamental importance. It introduces non-linearity into the model, a crucial characteristic that allows the network to learn and model complex, non-linear relationships present in the data. Without non-linear activation functions, a neural network, regardless of the number of layers, would behave like a simple linear model, drastically limiting its learning capacity. Common examples of activation functions include the Sigmoid function (which compresses the output between 0 and 1), the hyperbolic tangent (tanh, which compresses the output between -1 and 1), and the Rectified Linear Unit (ReLU, which returns the input if positive, otherwise zero), the latter being particularly popular for its computational efficiency and ability to mitigate the vanishing gradient problem.

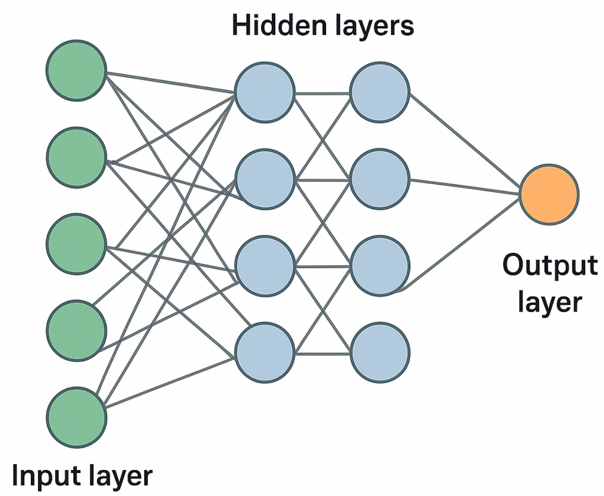


Figure 2.9: Structure of a neural network.

A neural network is structured into several layers. The *input layer*, as shown in Figure 2.9, is the first layer and receives raw data from the environment or dataset. This is followed by one or more *hidden layers*, where most of the data processing and transformations occur. Finally, the *output layer* produces the final prediction, decision, or action of the agent.
When a network contains multiple hidden layers, it is termed a *deep neural network* (DNN). This "depth" is a key characteristic that allows the network to learn a hierarchical representation of features from the data. This means that each layer learns to identify patterns progressively more complex: for example, in an image, the first layer might detect edges and corners, the second layer might combine these edges to form more complex shapes, and subsequent layers might assemble these shapes to recognize entire objects. This ability to extract features at different levels of abstraction is what makes DNNs so effective.

2.5.2 Deep Q-Network for Value Function Approximation

A paradigmatic example of DNN application in DRL is the DQN algorithm. In this approach, a DNN is employed to approximate the action-value function, $Q(s,a)$. The function $Q(s,a)$ estimates the "value" or expected cumulative reward an agent can obtain by performing an action a in a given state s and following an optimal policy thereafter.
Neural Network Input in DQN: The DQN's neural network receives the **environment's state, s, as input**. This state can be a direct representation of the environment's observation, such as the raw pixels of a game screen (as in the case of Atari games), or a vector of extracted features describing the current state.
Neural Network Output in DQN: In response to the input state s, the neural network outputs the **estimated Q-values for each possible action, a**. If the agent can choose among N discrete actions, the network's output will be a vector of N values, where each element $Q(s, a_i; \theta)$ represents the estimated Q-value for action a_i in state s, given the network's weights θ.
Learning Process and Loss Function: The network is trained by minimizing a loss function, which is typically the Mean Squared Error (MSE) between the network's predicted Q-value and a "target value" derived from the Bellman equation.
The loss function is given by:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta)\right)^2\right] \tag{2.12}$$

In this equation, θ represents the weights of the neural network being trained (the "main network"). The term θ^- denotes the weights of a separate *target network*. This target network is a copy of the main network that is updated periodically (e.g., every C training steps) rather than at every iteration. The use of a target network is crucial for stabilizing the training process, as it provides a more stable target compared to a target that would continuously change with the main network's updates.
The term $r + \gamma \max_{a'} Q(s',a';\theta^-)$ is the *target value*. Here:

- r is the immediate reward received after performing action a in state s.

- $\max_{a'} Q(s', a'; \theta^-)$ is the maximum estimated Q-value for the next state s' (the state reached after performing action a), calculated using the target network. This represents the best expected future reward from the next state.

The expectation $\mathbb{E}$ is taken over a batch of experiences (s, a, r, s') sampled from a *replay memory* D. The replay memory is a buffer that stores observed transitions (s, a, r, s'). Randomly sampling from it breaks temporal correlations between consecutive experiences, improving learning stability and efficiency.

This approach enabled agents to learn directly from high-dimensional sensory inputs, like raw pixels in Atari games, achieving human-level performance and sometimes surpassing it [45].

2.5.3 Policy Gradient Methods

Beyond value approximation, DNNS are also pivotal in *policy gradient methods*. In these methods, the DNN does not approximate a value but directly parameterizes the agent's *policy*, π. The policy defines the agent's behavior, specifying the probability of choosing a certain action in a given state.

Neural Network Input and Output in Policy Gradient Methods: In this setup, the network, often called a *policy network*, takes a state s as input and outputs a probability distribution over the possible actions. For example, if there are N actions, the output will be a vector of N probabilities, where the sum of these probabilities is 1. Each probability indicates the likelihood that the agent will choose the corresponding action in that state.

Network Weight Update: The network's weights, θ, are updated by performing *gradient ascent* on an objective function, $J(\theta)$, which represents the expected cumulative reward. The goal is to maximize this function, meaning to find the weights θ that lead to policies maximizing rewards. The *policy gradient theorem* provides a way to calculate the gradient of this objective function with respect to the policy parameters:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right] \tag{2.13}$$

Here, the gradient of the objective function is calculated as the expectation of the sum of the gradients of the log probabilities of the actions taken, each weighted by the cumulative future reward, G_t. G_t is the sum of discounted rewards from time t until the end of the episode. This update mechanism effectively increases the probability of actions that led to higher returns and decreases the probability of those that led to lower returns [42].

2.6 Off-policy and on-policy methods

The development of DRL algorithms has yielded a comprehensive toolkit, primarily divided into two main families: off-policy and on-policy methods, each

exhibiting distinct operational characteristics. Figure 2.10 illustrates the evolutionary relationships between the algorithms that will be discussed, highlighting how newer ideas were built to overcome the limitations of their predecessors.
This distinction can be considered as a **rule of thumb** in the case of actor-critic algorithms : many off-policy algorithms use the critic as a Q-function because it allows updating the actor using experiences from the replay buffer, without strictly following the current policy; while on-policy algorithms often use the critic as a V-function because directly evaluating the state with the current policy is more stable.

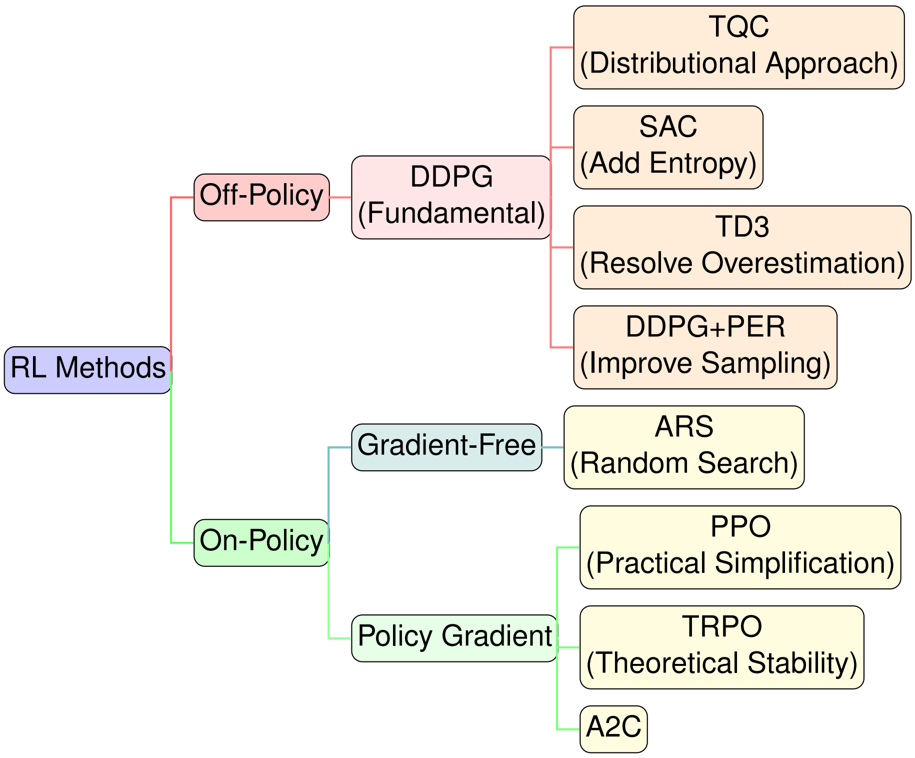


Figure 2.10: Hierarchical classification of RL methods.

2.6.1 Off-policy RL

Off-policy algorithms distinguish themselves through their ability to learn optimal policies from data generated by different, often more exploratory, behavioral policies. **This enables them to reuse historical experiences stored in large *replay buffers***, breaking temporal correlations between experiences and achieving high sample efficiency. A central challenge in RL is balancing the exploration of new actions with the exploitation of known good actions. Off-policy methods are particularly well-suited to this task, as they can leverage past experiences to guide learning without being constrained by the current policy.

Soft Actor-Critic

Soft Actor-Critic (SAC) is a state-of-the-art off-policy, model-free, actor-critic algorithm designed for continuous control tasks. It is recognized for its **superior**

sample efficiency and stability[19, 18]. SAC's key innovation lies in its **maximum entropy framework**, where the agent's objective is to maximize both expected reward and policy entropy. This entropy bonus encourages agents to explore broadly, improving noise robustness and reducing the risk of converging to poor local optima. By explicitly promoting randomness in the policy, SAC addresses the exploration-exploitation trade-off in a principled manner. Shuifu Gu Et. al[17] extend this idea to safe RL, considering constraints such as battery degradation to maintain reliable and safe operations in real-world V2G applications.

Deep Deterministic Policy Gradient

Deep Deterministic Policy Gradient (DDPG) extends DQN to high-dimensional, continuous action spaces, providing a foundational actor-critic approach for control problems, including V2G [25]. The actor deterministically maps states to actions, while the critic estimates Q-values. However, DDPG suffers from practical challenges such as training instability and **overestimation bias**, where critics systematically overestimate Q-values, leading to suboptimal policy learning [32, 2]. Despite these limitations, DDPG has been successfully applied in V2G contexts, including demand response management [24] and large-scale coordination through transfer learning [50].

Prioritized Experience Replay

Prioritized Experience Replay (PER) represents not a standalone algorithm but a crucial orthogonal modification for off-policy methods. Standard replay buffers sample past transitions uniformly. PER samples transitions based on their "importance," typically proportional to TD error magnitude [38]. This focuses learning processes on surprising or informative experiences, significantly accelerating convergence and improving final performance.

Truncated Quantile Critics

Truncated Quantile Critics (TQC) addresses overestimation bias through a distributional RL approach [23]. Instead of learning only expected returns, TQC estimates full return distributions using quantile regression. A quantile is a statistical value that divides a probability distribution into equally sized intervals (e.g., the median is the 0.5 quantile, splitting the data into two halves). In quantile regression, the agent learns to predict several quantiles of the return distribution, rather than a single mean value. This allows TQC to model the entire distribution of possible returns and reduce overestimation by truncating the highest quantiles during target computation.
Quantile regression extends classical linear regression by estimating conditional quantiles rather than only the mean. Formally, given data $\{(x_i, y_i)\}_{i=1}^n$ with predictors $x_i \in \mathbb{R}^p$ and response $y_i \in \mathbb{R}$, the τ-th conditional quantile ($0 < \tau < 1$) of Y given $X = x$ is:

$$Q_Y(\tau \mid X = x) = x^\top \beta_\tau$$

where β_τ are the regression coefficients associated with quantile τ. The estimator is obtained by solving:

$$\hat{\beta}_\tau = \underset{\beta \in \mathbb{R}^p}{\arg\min} \sum_{i=1}^{n} \rho_\tau \big(y_i - x_i^\top \beta \big),$$

with the asymmetric loss function $\rho_\tau(u) = u \cdot (\tau - \mathbf{1}_{\{u<0\}})$. For example, $\tau = 0.5$ corresponds to the conditional median.
By learning multiple quantile estimates and truncating the most optimistic ones before averaging, TQC **reduces overestimation bias** effectively, leading to improved performance in continuous control tasks while maintaining robustness.

Twin Delayed DDPG

Twin Delayed DDPG (TD3) was developed to mitigate DDPG's instabilities and overestimation bias [16]. It introduces three main innovations: (1) **clipped double Q-learning**, using two critic networks and taking the minimum estimate; (2) **delayed policy updates**, updating the actor less frequently than the critics; (3) **target policy smoothing**, adding noise to target actions for regularization.
These enhancements make TD3 a robust and reliable baseline for complex V2G scheduling tasks. Studies have applied TD3 to optimize EV charging and discharging schedules while considering grid stability [3, 26], demonstrating its ability to balance economic incentives with physical constraints.

2.6.2 On-Policy RL

On-policy methods learn exclusively from data generated by current policies being optimized. Once data is used for updates, it is discarded. While this makes them inherently less sample-efficient than off-policy counterparts, their updates often demonstrate greater stability and reduced divergence risk.

Advantage Actor-Critic

The **Advantage Actor-Critic (A2C)** algorithm represents a foundational *on-policy Actor-Critic* method that combines the strengths of both policy-based and value-based RL. In A2C, the *actor* is responsible for learning a policy that selects actions, while the *critic* estimates the value function to evaluate how good the current policy is. The term *advantage* refers to the use of an advantage function $A(s, a) = Q(s, a) - V(s)$, which stabilizes training by reducing the variance of policy gradient estimates. A2C operates *synchronously*, collecting experience from multiple environments in parallel and updating the shared model after aggregating the gradients from all workers. Building upon this idea, the **Asynchronous Advantage Actor-Critic (A3C)** algorithm introduced a major advancement by enabling *asynchronous parallelism* [30]. In A3C, multiple workers each maintain their own model and environment copy, interacting independently and updating a global network asynchronously. This design decorrelates data streams across workers, leading to improved stability and faster learning due to the diversity of experiences gathered in parallel. While on-policy methods like A2C and its variants are often considered less sample-efficient than off-policy approaches, they remain actively studied in the **Vehicle-to-Grid (V2G)** domain. For instance, Chifu *et al.* [11] propose a Deep Q-Learning–based method for smart scheduling of EVs in demand response, which shares some conceptual similarities with on-policy learning schemes.

Trust Region Policy Optimization

Trust Region Policy Optimization (TRPO) was the first algorithm to formalize policy update size constraints for guaranteed monotonic policy improvement [**schulman2020trust**]. It maximizes a "surrogate" objective function subject to policy change constraints, measured by Kullback-Leibler (KL) divergence. This creates a "trust region" within which new policies are guaranteed to improve upon previous ones, preventing catastrophic updates that can permanently derail learning. However, implementation complexity arises from second-order optimization requirements.

Proximal Policy Optimization

Proximal Policy Optimization (PPO) achieves TRPO's stability benefits and reliable performance using only first-order optimization, making it far simpler to implement and more broadly applicable [39]. It employs a novel **clipping** mechanism in its objective function to discourage large policy updates that would move new policies too far from previous ones, effectively creating "soft" trust regions. Due to its excellent balance of performance, stability, and implementation simplicity, PPO has become a default choice for many on-policy applications.

2.6.3 Gradient-Free Methods

Unlike gradient-based RL algorithms, which rely on computing or estimating the gradient of an objective function with respect to policy parameters, **gradient-free methods** (also called *derivative-free optimization methods*) search for optimal policies without explicitly using gradient information. Instead of following the gradient direction, these methods explore the policy parameter space through stochastic perturbations, random sampling, or evolutionary strategies, updating parameters based on performance evaluations.
This makes gradient-free approaches particularly attractive in scenarios where gradients are difficult to compute, noisy, or discontinuous. Although they often require more samples to converge compared to gradient-based methods, their simplicity, robustness, and parallelizability make them a valuable alternative for certain RL tasks.

Augmented Random Search

Augmented Random Search (ARS) represents a gradient-free approach that optimizes policies directly in parameter space [27]. It operates by exploring random policy parameter perturbations and updating central policy parameter vectors based on observed performance of these perturbations. While often less sample-efficient for complex, high-dimensional problems, its extreme simplicity, scalability, and robustness to noisy rewards can make it competitive in certain domains.

2.7 The Model-Based Benchmark: Model Predictive Control

While DRL offers powerful model-free approaches, model-based techniques require benchmarking against their most established and robust counterpart: **Model Predictive Control (MPC)**. MPC originated in the 1970s within the process control industry, with pioneering contributions by Richalet *et al.* [35] and Cutler and Ramaker [13]. The theoretical foundations for stability and optimality were subsequently established by Mayne and Rawlings [28]. At its core, MPC represents a proactive, forward-looking strategy that employs explicit mathematical system models to predict future evolution. At each control step, it solves a finite-horizon optimal control problem to determine the optimal control action sequence. Its primary strength, explaining widespread industrial adoption, lies in its inherent capability to handle complex system dynamics and operational constraints in a predictive manner [29]. Before formalizing the MPC optimization problem, it is essential to introduce the main temporal concepts that define its structure.
-**Sampling time (T):** It is the time difference between two consecutive state measurements or control updates. It defines the discrete-time evolution of the system.
- **Time horizon (N_T):** It is the total number of time instants over which the control input is applied to the system.
- **Prediction horizon (N):** It is the number of future steps considered in the optimization process. Over this window, the system states are predicted and optimized. Typically, $2 \leq N \leq N_T$.
- **Control horizon (N_C):** It defines the number of future control moves actually optimized. Beyond this horizon, the control input is often held constant, with $N_C \leq N$.

These concepts together define the temporal structure of MPC. At each sampling instant t, the controller predicts the system evolution over the *prediction horizon* N, optimizes a control sequence of length N_C, and applies only the first control action. The process is then repeated after T seconds, forming a closed-loop scheme known as **Receding Horizon Control (RHC)**.

2.7.1 Model Predictive Control Formulation

At each time step t, given the current state measurement $x(t)$, the MPC law is defined by solving a constrained finite-time optimal control problem.
Consider the discrete-time linear time-invariant (LTI) system:

$$x(k+1) = Ax(k) + Bu(k), \tag{2.14}$$

where $x(k) \in \mathbb{R}^n$ is the state vector and $u(k) \in \mathbb{R}^m$ is the control input.

At the current time t, using $x(t)$ as the initial state x_0, the optimization problem is formulated as:

$$J_0^*(x(t)) = \min_{U_0} J_0(x(t), U_0) \tag{2.15}$$

subject to system and constraint dynamics, where the quadratic cost function is:

$$J_0(x(0), U_0) = x_N^\top P x_N + \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) \tag{2.16}$$

The minimization is constrained as follows for $k = 0, \dots, N-1$:

$$x_k \in \mathcal{X}, \tag{2.17a}$$
$$u_k \in \mathcal{U}, \tag{2.17b}$$
$$x_N \in \mathcal{X}_f, \tag{2.17c}$$
$$x_0 = x(t). \tag{2.17d}$$

Here:

- $x_k \in \mathbb{R}^n$ denotes the predicted state at step k, starting from $x_0 = x(t)$;
- $U_0 = [u_0^\top, \dots, u_{N-1}^\top]^\top \in \mathbb{R}^{mN}$ is the decision vector of future control inputs;
- $Q, P \succeq 0$ are state penalty matrices, and $R \succ 0$ is the input penalty matrix;
- $\mathcal{X} \subseteq \mathbb{R}^n$ and $\mathcal{U} \subseteq \mathbb{R}^m$ are admissible state and input sets (often polyhedral);
- $\mathcal{X}_f \subseteq \mathbb{R}^n$ is the terminal constraint set ensuring stability.

The optimization yields an optimal sequence of control inputs:

$$U_0^*(x(t)) = \{u_0^*, u_1^*, \dots, u_{N-1}^*\}. \tag{2.18}$$

In the RHC strategy, only the first control action of the optimal sequence is applied:

$$u(t) = u_0^*(x(t)). \tag{2.19}$$

At the next sampling instant $t+T$, a new measurement $x(t+T)$ is acquired, and the optimization is repeated over a shifted prediction window $[t+T, t+T+N]$. This process continues iteratively, forming a closed-loop predictive control system.

2.7.2 MPC Classification

MPC can be classified in the following ways:

- **Based on the type of system model used in optimization:**
    1. **Linear MPC:** The system model and the constraints are linear. The cost function can be linear or quadratic, which results in linear or quadratic programming problems that are convex optimization problems.

2. **Nonlinear MPC:** The system model is nonlinear and constraints are either linear or nonlinear. The cost function is usually chosen as a linear or quadratic function, which results in a nonlinear programming problem that is non-convex.

- **Based on the implementation:**

  1. **Implicit MPC:** Also known as traditional MPC, in which the control input at each time instant is computed by solving an optimization problem online.
  2. **Explicit MPC:** In this approach, the online computation is reduced by transferring the optimization problem offline.

2.7.3 Implicit MPC: Online Optimization

The most common formulation involves **Implicit MPC**, where constrained optimization problems are solved online at each control step. For linear systems with quadratic costs, this typically constitutes a Quadratic Program (QP). The controller's objective involves finding future control input sequences

$$U = [u_{t|t}, \dots, u_{t+N-1|t}]$$

that minimize a cost function J over a prediction horizon N.

A key insight from [8] is that in this formulation, the control law is defined **implicitly** as the result of an optimization problem. There is no pre-computed algebraic function; the control action is determined at each step through a numerical computation. This approach is powerful due to its directness but is fundamentally limited by the need to perform this computation online. The authors of [8] highlight this, stating:

> "One limitation of MPC is that running the optimization algorithm on-line at each time step requires substantial time and computational resources."

The finite-horizon optimal control problem is formulated as:

$$\min_{U_t} J(x_t, U_t) = \sum_{k=0}^{N-1} \left(x_{t+k|t}^\top \mathbf{Q} x_{t+k|t} + u_{t+k|t}^\top \mathbf{R} u_{t+k|t} \right) + x_{t+N|t}^\top \mathbf{P} x_{t+N|t} \tag{2.20}$$

subject to system dynamics and operational constraints:

$$x_{t+k+1|t} = \mathbf{A} x_{t+k|t} + \mathbf{B} u_{t+k|t} \tag{2.21a}$$

$$x_{\min} \leq x_{t+k|t} \leq x_{\max} \tag{2.21b}$$

$$u_{\min} \leq u_{t+k|t} \leq u_{\max} \tag{2.21c}$$

At each time step t, this complete problem is solved, but only the first action of the optimal sequence, $u^*_{t|t}$, is applied to the system. The process then repeats at the next time step, $t+1$, using new system state measurements [10].

2.7.4 Improvement of the Fixed-Horizon MPC Formulation

Consider a controlled nonlinear discrete system described by

$$x(k+1) = f\big(x(k),\, u(k)\big), \tag{2.22}$$

where $x(k) \in \mathbb{R}^n$ is the state at discrete step k and $u(k) \in \mathbb{R}^m$ is the control input applied at the same step. (Note that the notation x^+ often used in the literature indicates $x(k+1)$.)

The classical MPC approach solves, at each instant, a finite optimization problem with a fixed prediction horizon N. We seek the control sequence $u_N = \big(u(0), \ldots, u(N-1)\big)$ that minimizes the cost

$$V_N(x) \;=\; \sum_{k=0}^{N-1} l\big(x(k),\, u(k)\big) \;+\; V_f\big(x(N)\big), \tag{2.23}$$

subject to the dynamics (2.22), the state and input constraints, and the terminal condition $x(N) \in \mathcal{X}_f$. Here V_f is the terminal cost defined on the terminal set $\mathcal{X}_f$ and serves, when appropriately chosen, as a Lyapunov function to ensure local stability.

The Lagrangian $l(x,u)$. The function $l : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}_{\geq 0}$, called the *Lagrangian* or *stage cost*, assigns an instantaneous cost to each state-action pair. Some important observations:
- **Interpretation**: $l(x,u)$ quantifies, instant by instant, how undesirable the pair (x,u) is with respect to the objective; for example, it penalizes deviation from the reference state and the control effort.
- **Required properties**: For stability results, it is typically required that $l(x,u) \geq 0$ for all (x,u) and that l be *positive definite* with respect to u; in many analyses, comparison-type conditions are also imposed using functions of the $\mathcal{K}_\infty$ class.
- **Role in MPC**: The Lagrangian defines the objective being minimized, allows a tradeoff between performance and control overhead, and allows the use of Lyapunov-type arguments (especially when combined with an appropriate terminal cost V_f).
The main disadvantage of the fixed-horizon formulation is the static choice of N: A N large enough to guarantee success even under the worst-case scenario unnecessarily increases the computational load in most cases. For this reason, adaptive horizon strategies (e.g., AHMPC) are studied that vary N online as a function of the current state. The core innovation of Adaptive Horizon Model Predictive Control (AHMPC) is to make the horizon length state-dependent, adapting it online to be as short as possible while maintaining stability.
In the article [22] was first deployed "ideal" version of AHMPC, it relies on a function $N(x)$, defined as the minimum integer N for which a feasible control sequence exists that drives the state x into the terminal set $\mathcal{X}_f$. The resulting optimal cost and control law are then state-dependent through this horizon:

$$V(x) = V_{N(x)}(x) \tag{2.24a}$$

$$\kappa(x) = \kappa_{N(x)}(x) \tag{2.24b}$$

This formulation leads to a valid Lyapunov function, confirming its stabilizing properties. However, this scheme is impractical because "in general, it is impossible to compute the function $N(x)$" [22].
In order to adress the issue the key idea is to use the terminal controller $\kappa_f(x)$ and terminal cost $V_f(x)$ to *verify online* if the chosen horizon is sufficient.
At a given state x and with a current horizon guess N, the algorithm is as follows:

1. **Solve and Extend:** Solve the N-horizon optimal control problem to get the trajectory $x^0(\cdot)$. Then, extend this trajectory for L additional steps using the terminal feedback law:

$$x^0(k+1) = f(x^0(k), \kappa_f(x^0(k))), \quad \text{for } k = N, \ldots, N+L-1 \tag{2.25}$$

2. **Verify Stability Conditions:** Check if the Lyapunov conditions, hold along this extended part of the trajectory:

$$V_f(x^0(k)) \geq \alpha(|x^0(k)|) \tag{2.26a}$$
$$V_f(x^0(k)) - V_f(x^0(k+1)) \geq \alpha(|x^0(k)|) \tag{2.26b}$$

for $k = N, \ldots, N+L-1$. Failure to satisfy these conditions implies that the terminal state $x^0(N)$ is likely not in a region of stability.

3. **Adapt the Horizon:** The adaptation logic is described as follows:
   - **If conditions hold:** The horizon N is considered sufficient. The control $u^0(0)$ is applied. At the next state x^+, the controller attempts a shorter horizon, typically $N-1$.
   - **If conditions fail:** The horizon N is too short. The controller "increase[s] N by 1 and... solve[s] the optimal control problem over the new horizon" from the same state x [22]. This is repeated until a sufficient horizon is found.

This practical scheme offers significant:

- **Computational Efficiency:** The primary advantage is that "the AHMPC horizon length decreases as the process is stabilized thereby lessening the on-line computational burden".
- **No Explicit Terminal Set:** The algorithm cleverly "proceeds without knowing the minimum horizon length function $N(x)$ and without knowing the domain of Lyapunov stability of the terminal cost $V_f(x)$".

2.7.5 Explicit MPC: Offline Pre-computation

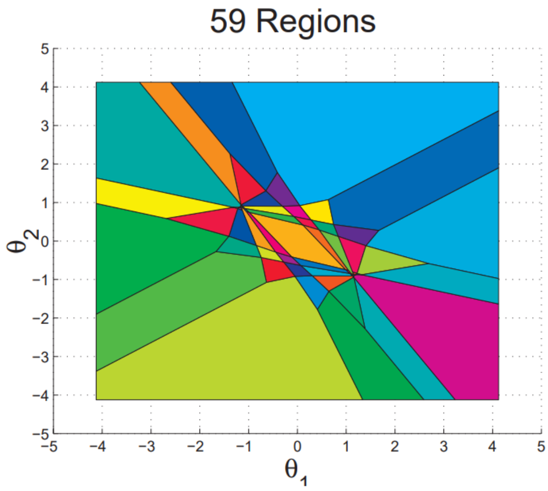


Figure 2.11: Example of a structure of a polyhedral region.

For systems with fast dynamics or limited online computational capacity, **Explicit MPC** offers an alternative approach[4]. The key idea is to move the computational effort entirely offline. As detailed extensively in [8], this is achieved by treating the current state x_t not as a fixed value, but as a vector of parameters. The constrained optimization problem is then solved offline for all possible initial states within a given range using **multi-parametric programming**. The authors of [8] frame this as the main contribution of their work:

> "we want to determine the [...] feedback control law $f(x)$ that generates the optimal $u_k = f(x(k))$ **explicitly** and not just **implicitly** as the result of an optimization problem."

The result is not an online algorithm, but a pre-computed, explicit control law, $K(x_t)$, which for linear systems is a **piecewise affine (PWA) function** of the state vector x_t:

$$u^*(x_t) = \mathbf{F}_i x_t + \mathbf{g}_i \quad \text{if } x_t \in \mathcal{X}_i \tag{2.27}$$

The state space is partitioned into a set of convex polyhedral regions $\mathcal{X}_i$, with a unique affine control law defined for each region. Online operation is thereby reduced from solving a QP to a simple function evaluation: first, a fast lookup operation identifies which region $\mathcal{X}_i$ the current state x_t occupies, and second, the corresponding simple affine control law is applied. This trades a very high offline computational burden and significant memory requirements for extremely fast and deterministic online execution [5, 8].

2.7.6 Deep Learning for an Efficient Explicit MPC Representation

Explicit MPC approach suffers from a significant drawback: the number of polyhedral regions, and thus the memory required to store the corresponding affine laws, can grow exponentially with the prediction horizon and the number of constraints . This "curse of dimensionality" severely limits its practical application, especially in memory-constrained embedded systems[21].
To address this challenge, the work in [21] proposes a novel approach that leverages deep learning to create a highly efficient representation of the explicit MPC law. The methodology bridges the gap between the implicit and explicit approaches by using the former to train the latter. The core idea is to train a DNN to act as the explicit controller. This requires a comprehensive dataset of optimal state-action pairs, which is generated in a crucial offline phase. As detailed in [21], this is achieved by repeatedly using the **implicit MPC solver**:

1. A large number of state points ($x_{tr,i}$) are sampled from the relevant operating region of the system.
2. For each state point, the full MPC optimization problem is solved to find the corresponding optimal control input ($u^*_{tr,i}$). This is precisely the task an implicit MPC controller performs online.
3. The resulting pairs ($x_{tr,i}, u^*_{tr,i}$) form a large dataset that effectively maps states to their optimal control actions.

This dataset is then used to train a DNN via supervised learning, minimizing the error between the network's output and the optimal control actions. In essence, the computationally intensive implicit solver is used offline to "teach" a fast and compact neural network how to behave optimally.
The power of this method stems from the theoretical insight that a DNN with Rectified Linear Unit (ReLU) activation functions is not merely an approximator but can, in fact, **exactly represent** the PWA function that defines the explicit MPC feedback law [21].
The foundation of this exact representation lies in two main results. First, any scalar PWA function $K_i(x_t)$ (representing the i-th component of the control vector) can be expressed as the difference of two convex PWA functions:

$$K_i(x_t) = \gamma_i(x_t) - \eta_i(x_t) \tag{2.28}$$

Second, any convex PWA function, which can be described as the pointwise maximum of a set of affine functions, can be exactly represented by a deep ReLU network. Specifically, a convex function composed of N affine regions can be perfectly modeled by a network with width n_x+1 (where n_x is the state dimension) and depth N [21]. Combining these findings, the complete multi-output MPC control law $K(x_t)$ can be exactly represented by a vector of n_u pairs of DNNs, where each pair models the γ_i and η_i components for a single control output:

$$K(x_t) = \begin{bmatrix} \mathcal{N}(x_t; \theta_{\gamma,1}) - \mathcal{N}(x_t; \theta_{\eta,1}) \\ \vdots \\ \mathcal{N}(x_t; \theta_{\gamma,n_u}) - \mathcal{N}(x_t; \theta_{\eta,n_u}) \end{bmatrix} \tag{2.29}$$

where $\mathcal{N}$ denotes a deep ReLU network with its corresponding set of parameters θ. The particular advantage of using *deep* neural networks lies in their representational efficiency. While the number of parameters (and thus, the memory footprint) of a deep network grows linearly with its depth, the number of linear regions it can represent can grow exponentially [21]. This provides an inverse relationship to the problem of traditional Explicit MPC, allowing a deep network to capture an exponentially large number of control regions with only a modest, linearly growing memory cost. Consequently, this deep learning-based representation transforms the explicit MPC law into a highly compact and fast-to-evaluate form, overcoming the primary obstacle of prohibitive memory requirements and enabling the deployment of complex predictive controllers on embedded systems.

2.8 A Primer on Lithium-Ion Battery Chemistries and Degradation

The effectiveness, safety, and economic viability of any V2G strategy remain fundamentally constrained by the physical and chemical characteristics of vehicle batteries. Battery chemistry selection dictates EV operational envelopes, influencing energy density, power capabilities, lifespan, and safety profiles. Clear understanding of these trade-offs proves essential for developing robust, realistic, and responsible control algorithms.

2.8.1 Fundamental Concepts and Degradation Mechanisms

Battery degradation is a complex and irreversible process that gradually reduces a cell's ability to store and deliver energy. It manifests primarily as capacity loss (energy fade) and as an increase in internal resistance (power fade). These phenomena are usually attributed to two categories of mechanisms: *calendar aging* and *cyclic aging* [6].

Calendar aging occurs while the battery is at rest, regardless of whether it is fully charged or nearly empty. The main mechanism behind this process is the slow growth of the **Solid Electrolyte Interphase (SEI)** layer on the graphite anode. While a thin and stable SEI layer is necessary for battery operation, its continuous thickening consumes both active lithium and electrolyte, resulting in irreversible capacity loss. The rate of SEI growth is particularly sensitive to operating conditions. High temperatures accelerate chemical reaction rates, including parasitic ones, making storage in hot environments detrimental. Similarly, high states of charge (SoC) correspond to low anode potentials, which increase the reactivity of graphite with the electrolyte and foster faster SEI growth [44]. For this reason,

leaving EVs stored for long periods at 100% SoC is generally discouraged.
Cyclic aging, in contrast, results from the processes that take place during charging and discharging. This form of degradation is of particular importance for V2G applications, which inherently involve frequent cycling. Several mechanisms contribute to cyclic aging. The repeated intercalation and de-intercalation of lithium ions induce mechanical stress due to the expansion and contraction of electrode materials. Over time, this stress may generate micro-cracks in electrode particles, leading to a loss of electrical contact and a reduction of active material, effects that become more severe with larger **Depths of Discharge (DoD)**. Mechanical volume changes can also compromise the integrity of the SEI layer, causing cracks that expose fresh anode surface to the electrolyte and trigger renewed SEI growth. In addition, under demanding conditions,such as very high charging rates (high C-rates) or low temperatures, lithium ions may deposit as metallic lithium on the anode surface rather than intercalating properly. This phenomenon, known as lithium plating, is particularly harmful: it leads to rapid capacity loss and may produce dendritic structures capable of piercing the separator, thereby creating internal short circuits and serious safety hazards [6].
Both calendar and cyclic aging are unavoidable, the latter is especially critical for V2G systems. Since these applications require frequent and sometimes deep cycling, understanding and mitigating cyclic degradation is paramount to guarantee long-term system viability.

2.8.2 Key Automotive Chemistries

The EV market is dominated by several key lithium-ion battery families, primarily distinguished by cathode materials. Each chemistry presents different balances of performance characteristics.

- **Lithium Nickel Manganese Cobalt Oxide (NMC):** For years, this has been the most popular choice due to its excellent balance of energy density, power, and cycle life. The ratio of Nickel, Manganese, and Cobalt can be adjusted (such as NMC111, NMC532, NMC811) to prioritize either energy density (high Nickel) or safety and longevity (lower Nickel).

- **Lithium Nickel Cobalt Aluminum Oxide (NCA):** Offers very high specific energy (energy density by weight), enabling longer vehicle range. However, it typically comes at the cost of slightly lower cycle life and narrower safety margins compared to NMC, requiring more sophisticated thermal management.

- **Lithium Iron Phosphate (LFP):** Is rapidly gaining market share, particularly in standard-range vehicles, due to lower cost (no cobalt, an expensive and ethically contentious material) and superior safety. LFP batteries offer exceptional cycle life (often 3-4 times that of NMC/NCA) and are considered the safest common Li-ion chemistry due to high thermal stability. Their main drawback involves lower energy density.

- **Lithium Titanate Oxide (LTO):** This represents a more niche chemistry using titanate-based anodes instead of graphite. This provides exceptional safety (virtually no thermal runaway risk), extremely long cycle life (>10,000 cycles), and excellent low-temperature performance. However, very low energy density and high cost currently limit its use to specialized applications.

2.8.3 Voltage Profiles and the Challenge of SoC Estimation

The relationship between battery open-circuit voltage and SoC represents a critical, non-linear function unique to each chemistry. The derivative of cell voltage with respect to DoD, $\frac{dV_{cell}}{d(DoD)}$, constitutes a key parameter for Battery Management Systems (BMSs). Steep, monotonic slopes allow BMS to accurately infer SoC from simple voltage measurements. Conversely, flat slopes make this estimation extremely difficult.

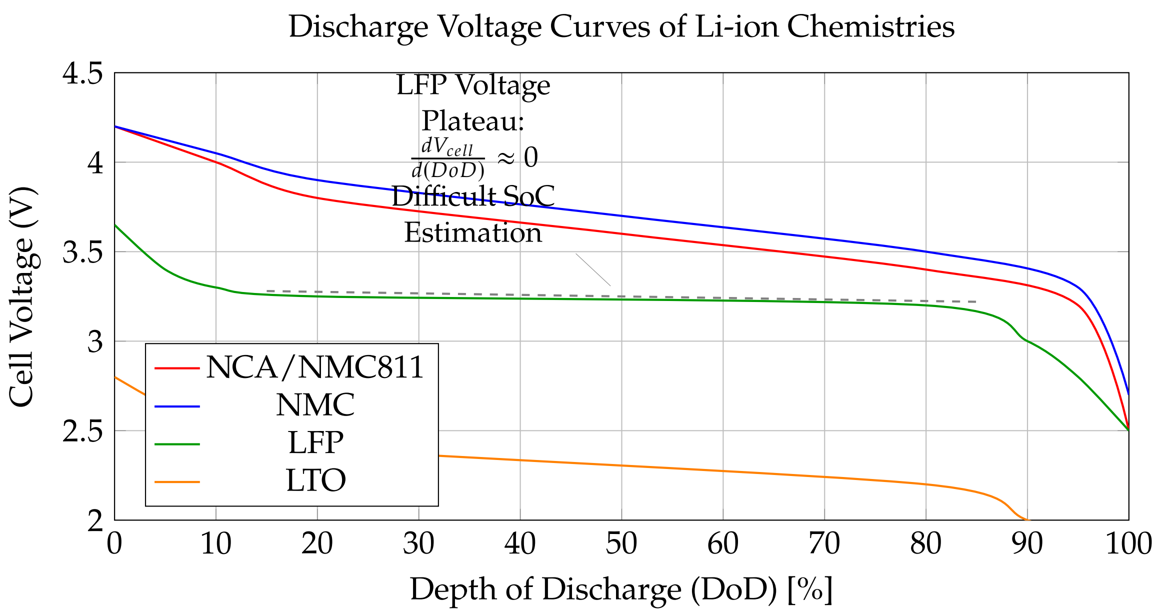


Figure 2.12: Typical discharge voltage curves for various lithium-ion chemistries.

As shown in Figure 2.12, LFP's remarkably flat voltage plateau makes it nearly impossible for BMS to determine precise SoC in the central operating range (from approximately 20% to 80%) using voltage alone. This necessitates more complex estimation techniques, such as Coulomb counting (integrating current over time), which can suffer from drift. To correct this drift, LFP-equipped vehicles require periodic full charges to 100% for BMS recalibration. This represents an important operational constraint that V2G control strategies must consider.

2.8.4 Comparative Analysis and Safety Considerations

The trade-offs between chemistries are summarized in Table 2.1. Safety remains paramount, with the primary risk being thermal runaway,a dangerous, self-sustaining exothermic reaction. This risk relates directly to cathode material chemical and thermal stability. Higher energy density generally means more energy packed into smaller mass, which can be released violently if cells are compromised. Consequently, critical temperatures for initiating thermal runaway are generally lower for higher energy density chemistries. LFP's stable phosphate-based structure makes it far more resistant to thermal runaway than nickel-based counterparts, a key reason for its growing popularity.

Table 2.1: Comparative analysis of key automotive battery chemistries, highlighting the trade-offs between performance and safety.

Metric	NCA	NMC	LFP	LTO	LCO
Energy Density (Wh/kg)	200 - 260 (Highest)	150 - 220 (High)	90 - 160 (Moderate)	60 - 110 (Low)	150-200 (High)
Cycle Life	1000 - 2000	1000 - 2500	2000 - 5000+	>10,000	500 - 1000
Safety	Good	Very Good	Excellent	Excellent	Poor
Thermal Runaway Temp (°C)	~150 - 180	~180 - 210	~220 - 270	> 250	~150

Chapter 3

An Enhanced V2G Simulation Framework for Robust Control

Developing, validating, and benchmarking advanced control algorithms for V2G systems presents significant challenges. Real-world experimentation is often impractical due to high costs, logistical constraints, and potential risks to grid stability and vehicle hardware. A realistic, flexible, and standardized simulation environment is therefore essential. This thesis builds upon **EV2Gym**, an open-source simulator designed for the V2G smart charging research [32]. The work presented here extends the original framework substantially, transforming it into a high-fidelity **digital twin** suitable not only for single-scenario optimization, but also for developing and rigorously evaluating **robust, generalist control agents**. The enhanced framework supports two complementary modes of experimentation: detailed analysis of agents specialized for a single environment, and a novel methodology for training and testing agents capable of generalizing across multiple diverse and unpredictable scenarios. This chapter presents the extended architecture, its data-driven models, and its evaluation capabilities, establishing the methodological foundation for subsequent chapters.

3.1 Core Simulator Architecture

The framework maintains the modular architecture of EV2Gym, which mirrors the key entities of a real-world V2G system. It is built on the OpenAI Gym (now Gymnasium) API, which provides a standardized agent-environment interface based on states, actions, and rewards [9].

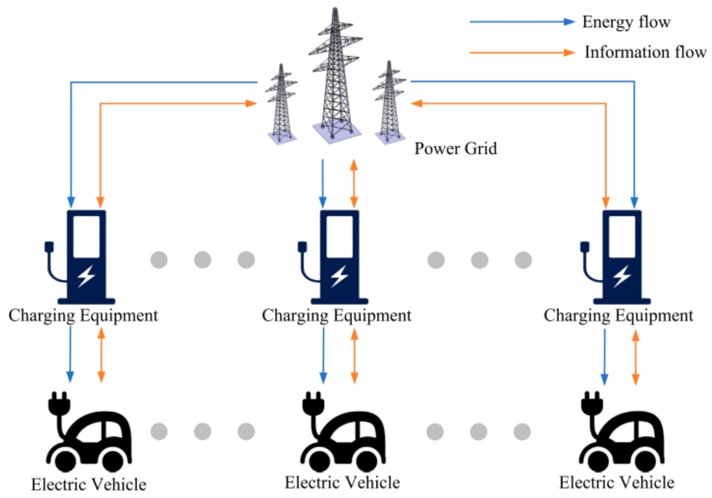


Figure 3.1: Diagram of charging and discharging scheduling for EVs. [3].

The architecture consists of several interacting components:

- **Charge Point Operator (CPO):** The central management component of the simulation, responsible for controlling the charging infrastructure and serving as the primary interface for the control algorithm (the DRL agent). The CPO aggregates system state information and dispatches control actions to individual chargers.

- **Chargers:** Digital representations of physical charging stations, configurable by type (AC/DC), maximum power, and efficiency. This enables simulation of heterogeneous charging infrastructures.

- **Power Transformers:** These components model the physical connection points to the grid, aggregating the electrical load from multiple chargers. They enforce the physical power limits of the local distribution network and can model inflexible base loads (e.g., buildings) and local renewable generation (e.g., solar panels).

- **Electric Vehicles (EVs):** Dynamic and autonomous agents, each defined by its battery capacity, power limits, current and desired energy levels, and specific arrival and departure times.

The simulation process follows a reproducible three-phase structure: (1) **Initialization** from a comprehensive YAML configuration file, (2) a discrete-time **Simulation Loop** where the agent interacts with the environment, and (3) a final **Evaluation and Visualization** phase that generates standardized performance metrics.

3.2 A Unified Experimentation and Evaluation Workflow

A key contribution of this thesis is the development of a unified and powerful experimentation workflow, orchestrated primarily by the **run_experiments.py** script (with an interactive interface provided by **run_interactive.py**). This approach replaces the previous fragmentation, offering a single platform to manage the entire lifecycle of training, benchmarking, and evaluation for V2G control agents. The workflow is designed to be both flexible for research and rigorous for evaluation, supporting the goal of developing both specialized and generalized agents.

Orchestration and Execution Interfaces

The **run_experiments.py** script serves as the central hub for all experimentation. To ensure flexibility and reproducibility, the project offers two execution modes:

1. **Interactive Mode (run_interactive.py):** Provides a guided, step-by-step command-line interface, ideal for initial configuration, testing, and running single experiments. The user responds to successive prompts to select scenarios, algorithms, and training parameters.
2. **Scripted/Batch Mode (run_experiments.py):** Utilizes the **argparse** module to accept all configuration parameters (e.g., **–scenarios**, **–reward**$_{f}unc$,

The key steps in the workflow include:

- **Algorithm Selection:** The script dynamically defines the algorithms to be executed, which can include DRL agents (e.g., SAC,DDPG, DDPG+PER, TQC), classical optimization methods (Model Predictive Control), and heuristics (e.g., AFAP, ALAP).
- **Scenario and Reward Selection:** The user selects one or more **.yaml** configuration files and the desired reward function from the **reward.py** module.
- **Battery Calibration:** The option to run **Fit_battery.py** is provided to calibrate the parameters of the battery degradation model, ensuring that simulations use the most up-to-date data.

Scenario structure Yaml file

The YAML (YAML Ain't Markup Language) configuration file serves as the foundational blueprint for defining a simulation experiment within the EV2Gym framework. It is a powerful and indispensable tool for the systematic study of EV smart charging strategies. It provides a structured and transparent method for defining the physical, economic, and stochastic properties of the simulation environment. A thorough understanding and deliberate manipulation of these parameters are essential for formulating meaningful research questions, generating valid results, and drawing robust conclusions about the performance of various control algorithms.

3.3 Core Simulation Parameters

This primary section establishes the temporal boundaries and fundamental operational logic of the simulation.

timescale Defines the temporal granularity of a single simulation step, measured in minutes. This parameter is critical as it dictates the resolution of the control problem; a smaller timescale allows for finer control but increases computational complexity.

```
# Timescale in minutes per simulation step
timescale: 15

```

simulation_length Specifies the total duration of a single simulation episode, measured in the number of steps defined by **timescale**.

```
# Duration in steps per simulation
simulation_length: 112

```

3.3.1 Temporal and Contextual Settings

These parameters anchor the simulation to a specific point in time, which is crucial for sourcing external data such as electricity prices and solar irradiance profiles.

date The **year**, **month**, and **day** parameters set the initial date. The **random_day** boolean determines if a new, random date is selected upon each environment reset, facilitating stochastic analysis over different time periods.

```
year: 2022
month: 1
day: 17
random_day: True

```

time The initial **hour** and **minute** of the simulation. These typically remain fixed across resets to ensure consistent starting conditions.

simulation_days Constrains the simulation to specific day types (**weekdays**, **weekends**, or **both**), allowing for the study of distinct demand patterns.

3.3.2 Stochastic Demand Modeling

This section governs the arrival and behavior of Electric Vehicles (EVs), which constitute the primary source of flexible load in the simulation.

scenario Defines the archetypal environment (e.g., **workplace**, **public**, **private**), which loads a corresponding statistical model for EV arrival times, departure times, and required energy.

spawn_multiplier A scalar value that adjusts the baseline EV arrival rate. A value greater than 1 increases the density of EVs, thereby intensifying the stress on the charging infrastructure and creating more challenging control problems.

```yaml
scenario: workplace
spawn_multiplier: 5

```

3.3.3 Economic Framework

These parameters establish the financial incentives and constraints that drive the optimization objective.

discharge_price_factor A foundational economic lever for V2G operations. It defines the remuneration for discharging energy back to the grid as a fraction of the purchasing price. A value less than 1.0 models a realistic bid-ask spread, ensuring economic viability for the grid operator.

```yaml
# A factor of 0.95 means selling energy yields 95% of the
    purchasing price.
discharge_price_factor: 0.95

```

3.3.4 Physical Infrastructure

This section specifies the hardware and topological constraints of the charging network. These are the "hard" constraints that no algorithm can violate without incurring significant penalties.

v2g_enabled A global boolean flag to enable or disable V2G capabilities for all charging stations.

number_of_charging_stations The total count of charging points available.

transformer Defines the properties of the local transformer, most critically its **max_power** capacity in kW. This parameter represents the primary bottleneck of the system; exceeding this limit results in an overload condition.

```yaml
transformer:
  max_power: 100 # in kW

```

charging_station Specifies the default electrical properties of a charging station, including maximum current, voltage, and number of phases. These are used to calculate the maximum power per station.

```yaml
charging_station:
  max_charge_current: 32 # Amperes
  voltage: 400 # Volts
  phases: 3

```

3.3.5 Exogenous Energy Events

These parameters introduce external, often uncontrollable, energy flows and grid requests that add realism and complexity to the simulation.

inflexible_loads Models background energy consumption from sources other than EVs (e.g., building lighting, HVAC). When enabled, this load consumes a portion of the transformer's capacity, reducing the available power for EV charging.

solar_power Simulates on-site photovoltaic generation. This introduces a source of low-cost, intermittent energy that intelligent algorithms can leverage to reduce charging costs.

demand_response Models a request from the grid operator to curtail power consumption for a specified duration. The **notification_of_event_minutes** parameter is crucial, as it provides the forecast horizon that advanced control algorithms (like MPC) require to plan and adapt their charging strategies effectively.

3.3.6 Electric Vehicle Fleet Specification

This section defines the characteristics of the vehicles being charged.

heterogeneous_ev_specs A boolean that, if true, populates the simulation with a diverse fleet of EVs with varying battery capacities and charging rates, as defined in the **ev_specs_file**. This enhances simulation realism.

ev This subsection defines the default properties of a single EV. Key parameters include:

- **charge_efficiency** and **discharge_efficiency**: Model the energy losses inherent in the charging and discharging processes. Values less than 1.0 are physically realistic and are the source of the **Q_lost** metric.
- **desired_capacity**: Represents the target state-of-charge requested by the user, modeling the fact that not all users require a full 100

```
ev:
  charge_efficiency: 0.95
  discharge_efficiency: 0.95
  desired_capacity: 0.9

```

3.3.7 Dual-Mode Training: Specialists and Generalists

The new workflow seamlessly manages the training of both "specialist" agents (optimized for a single scenario) and "generalist" agents (robust across multiple scenarios). The behavior is determined by the number of selected scenarios:

- **Single-Domain Specialization:** If only one scenario is selected, the RL agent is trained exclusively on that **EV2Gym** environment.
- **Multi-Scenario Generalization:** If multiple scenarios are selected, the script automatically utilizes the custom **MultiScenarioEnv** wrapper. This Gymnasium environment:
  - **Random Selection:** At the start of each training episode, it randomly selects a configuration file and initializes a new **EV2Gym** environment.
  - **Handling Heterogeneous Spaces:** To allow a single neural network to control environments with a varying number of charging stations (and thus different observation and action space sizes), the maximum size (**MAX_CS**) across all scenarios is calculated. The returned observation is **padded** with zeros up to the maximum size (**max_obs_shape**), and the network's action is **sliced** to the actual size required by the current environment (**curren_env.action_space.shape**). This mechanism, supported by the **CompatibilityWrapper** during evaluation, is crucial for training generalist agents.

Advanced Multi-Scenario Training Strategies

In addition to random selection, the workflow supports advanced strategies to enhance generalization:

- **Curriculum Learning (CurriculumEnv):** The agent is trained in a predefined sequence of scenarios. The environment progresses to the next curriculum level only after the agent has completed a fixed number of training steps (**steps_per_level**) in the current level. This allows for starting with simpler scenarios and progressing toward complexity.
- **Shuffled Multi-Scenario (ShuffledMultiScenarioEnv):** At the beginning of each "epoch," the list of scenarios is randomly shuffled. The agent runs one full episode on each scenario in this random order before the list is reshuffled for the next epoch. This ensures the agent sees all scenarios in a non-repetitive order within a complete cycle.

3.3.8 MPC Integration and Approximation

The workflow integrates a classical Linear MPC.**Linear MPC:** The **run_experiments.py** script utilizes an MPC based on a linear optimization model (**OnlineMPC_Solver**).

3.3.9 Rigorous and Reproducible Benchmarking

The **run_benchmark** function in **run_experiments.py** is designed to ensure fair and rigorous evaluation:

- **Replay Mechanism:** To compare algorithms under identical conditions, the benchmark first runs a "replay" simulation with a null action, saving the entire sequence of stochastic events (EV arrivals, prices, etc.) to a **.pkl** file. Subsequently, each algorithm is evaluated by loading and reproducing *exactly* the same event sequence. This eliminates stochastic variability between runs, making the comparison between algorithms highly reliable.
- **Performance Metrics:** Results are aggregated over multiple simulations (**num_simulations**) and include key metrics such as:
  - Total Profit (€)
  - Average User Satisfaction (%)
  - Peak Transformer Loading (%)
  - Total Battery Degradation (%), with detail on cyclic loss and calendar loss.
- **Automatic Visualization:** Plotting functions (**plot_performance_metrics**, **plot_degradation_details**) automatically generate bar charts for the key metrics, grouping algorithms by category (heuristics, MPC, RL Off-Policy) for clear visual analysis.

3.3.10 Software Implementation and Project Structure

The practical implementation is organized within a modular Python package named **ev2gym**. This structure promotes code reusability and separation of concerns. High-level experimentation scripts, such as **run_experiments.py** , reside in the project's root directory and orchestrate experiments by utilizing the core functionalities provided by the **ev2gym** package.

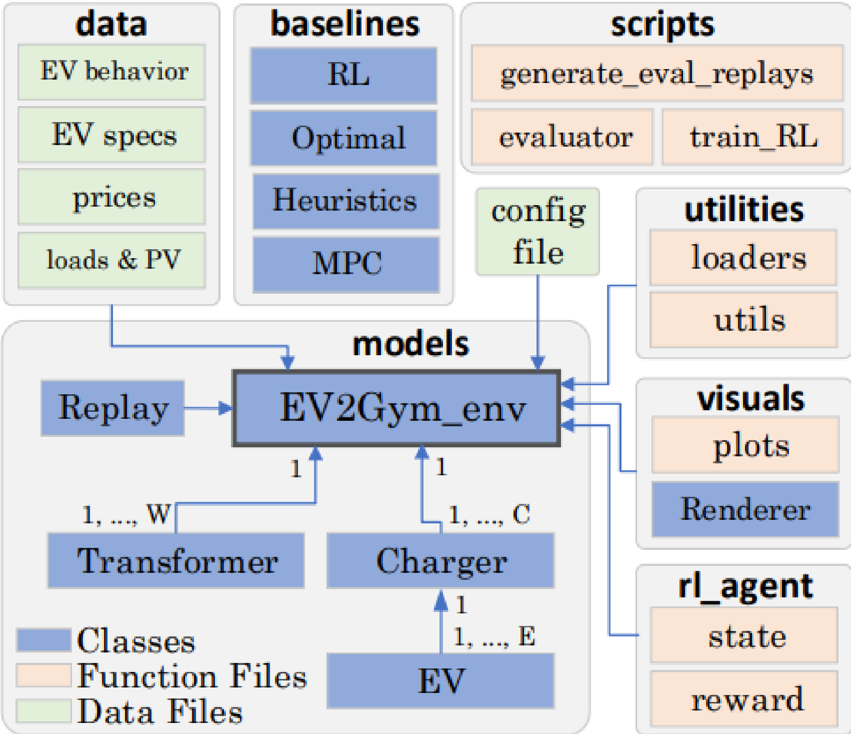


Figure 3.2: Directory and file structure of the EV2Gym package. [32].

The key subdirectories within the **ev2gym** package are:

- **baselines/**: Contains implementations for all non-RL controllers, including rule-based heuristics (in **heuristics.py**) and all variants of the Model Predictive Controllers (in **pulp_mpc.py**).
- **rl_agent/**: The central location for all RL logic, containing modules for state vector construction (**state.py**), the library of available reward functions (**reward.py**), and the implementation of custom RL algorithms (**custom_algorithms.py**).
- **utilities/**: A collection of helper functions and utility classes used across the entire framework.
- **models/**: Designated for storing serialized, pre-trained machine learning models, such as the Random Forest model used by the Approximate-Explicit MPC.

This modular software design allows for independent development and testing of different components, such as control algorithms and reward functions, while maintaining a consistent and unified simulation environment.

3.4 Core Physical Models

The simulation's fidelity depends on its detailed, empirically validated models, which are necessary for developing control strategies robust enough for real-world application.

3.4.1 EV Model and Charging/Discharging Dynamics

The framework implements a realistic two-stage charging/discharging model that captures the non-linear behavior of lithium-ion batteries, simulating both the **constant current (CC)** and **constant voltage (CV)** phases. Each EV is defined by a comprehensive parameter set: maximum capacity (E_{max}), a minimum safety capacity (E_{min}), separate power limits for charging and discharging ($P_{ch}^{max}, P_{dis}^{max}$), and distinct efficiencies for each process (η_{ch}, η_{dis}).

3.4.2 Battery Degradation Model

To address the critical issue of battery health in V2G operations, the simulator incorporates a semi-empirical battery degradation model. It quantifies capacity loss (Q_{lost}) as the sum of two primary aging mechanisms [32]: calendar aging and cyclic aging.

- **Calendar Aging (d_{cal}):** Time-dependent capacity loss, influenced by the battery's average SoC and temperature (Θ). The formula is:

$$d_{cal} = 0.75 \cdot (\epsilon_0 \cdot \overline{SoC} - \epsilon_1) \cdot e^{-\epsilon_2/\Theta} \cdot \frac{t_{days}}{(t_{days} + 1)^{0.25}} \tag{3.1}$$

- **Cyclic Aging (d_{cyc}):** Wear resulting from charge/discharge cycles, dependent on energy throughput ($E_{exchanged}$), depth-of-cycle (implicitly via $\overline{SoC}$), and the total accumulated charge (Q_{acc}). The formula is:

$$d_{cyc} = (\zeta_0 + \zeta_1 \cdot |\overline{SoC} - 0.5|) \cdot \frac{E_{exchanged}}{\sqrt{Q_{acc}}} \tag{3.2}$$

The total capacity loss is the sum $Q_{lost} = d_{cal} + d_{cyc}$. This integrated model enables direct quantification of how different control strategies impact the battery's long-term State of Health (SoH), supporting the training of agents that balance profitability with battery preservation.
A key feature of thissim framework is that the physical parameters for this model ($\epsilon_0, \epsilon_1, \epsilon_2, \zeta_0, \zeta_1, Q_{acc}$) are not fixed. They can be empirically calibrated from real-world experimental data using the provided **Fit_battery.py** script, as detailed in Section 3.6.

3.4.3 Reproducible Benchmarking and Evaluation

To ensure a fair and scientifically valid comparison, the **run_benchmark** function implements a rigorous evaluation protocol. For each scenario, it first generates a "replay" file containing the exact sequence of stochastic events (e.g., EV arrivals, energy demands). This exact same sequence is then used to evaluate every algorithm, eliminating randomness as a factor in performance differences. The script runs multiple simulations for statistical robustness, aggregates the mean results, and automatically generates a suite of comparative plots, including overall performance metrics and detailed battery degradation analyses.

3.4.4 Interactive Web-Based Dashboard

To complement the command-line-driven workflow, the project includes an interactive web-based dashboard built with the Streamlit library, executed via the **streamlit_app.py** script. This graphical user interface (GUI) serves two primary functions, significantly enhancing usability and accessibility for experimentation and results analysis.

Simulation Orchestrator

The first part of the dashboard acts as a GUI wrapper for the **run_experiments.py** script. It provides a user-friendly web form where users can:

- Select which algorithms to benchmark from a multi-select list.
- Choose one or more scenarios to test.
- Pick a specific reward function for the RL agents from a dropdown menu.
- Set simulation parameters, such as the number of evaluation runs.
- Toggle optional steps, like running the **Fit_battery.py** calibration or enabling RL model training.

Upon clicking the "Run Simulation" button, the application constructs the equivalent command-line arguments and executes **run_experiments.py** as a subprocess. It captures and displays the console output in real-time on the web page, providing a seamless user experience without requiring direct terminal interaction.

Results Visualizer

The second part of the dashboard is a dedicated results browser. It automatically scans the **results/** directory and presents a list of all completed benchmark runs (organized by timestamp). The user can select a specific benchmark, and the application will find and display all the generated plots (e.g., performance comparisons, battery degradation graphs) directly on the page. This feature allows for quick and convenient inspection and comparison of outcomes from different experiments.

3.5 Evaluation Metrics

To ensure a fair and comprehensive comparison, all algorithms are evaluated against an identical set of pre-generated scenarios through a "replay" mechanism. The **mean** and **standard deviation** of performance are calculated across multiple simulation runs. The key metrics include:

- **Total Profit ($):** The net economic outcome, calculated as revenue from energy sales minus the cost of energy purchases.

$$\Pi_{\text{total}} = \sum_{t=0}^{T_{\text{sim}}} \sum_{i=1}^{N} \left(C_{\text{sell}}(t) P_{\text{dis},i}(t) - C_{\text{buy}}(t) P_{\text{ch},i}(t) \right) \Delta t$$

- **User Satisfaction (Average):** The fraction of energy delivered compared to what was requested by the user, averaged across all EV sessions. A score of 1 indicates perfect service.

$$US_{\text{avg}} = \frac{1}{N_{\text{EVs}}} \sum_{k=1}^{N_{\text{EVs}}} \min \left(1, \frac{E_k(t_k^{\text{dep}})}{E_k^{\text{des}}} \right)$$

- **Transformer Overload (kWh):** The total energy that exceeded the transformer's rated power limit. An ideal controller should achieve a value of 0.

$$O_{\text{tr}} = \sum_{t=0}^{T_{\text{sim}}} \sum_{j=1}^{N_T} \max(0, P_j^{\text{tr}}(t) - P_j^{\text{tr,max}}) \cdot \Delta t$$

- **Battery Degradation** The cost of battery aging due to both cyclic and calendar effects.

$$D_{\text{batt}} = \sum_{k=1}^{N_{\text{EVs}}} (\text{CyclicCost}_k + \text{CalendarCost}_k)$$

3.6 Simulator Implementation Details

During the analysis and implementation of new metrics, fundamental details about the **EV2Gym** simulator's architecture emerged, which warrant documentation. The configuration of Electric Vehicles (EVs) and the calculation of their degradation follow a specific logic dependent on a key parameter in the **.yaml** configuration files.

Vehicle Definition Modes

The simulator operates in two distinct modes, controlled by the boolean flag **heterogeneous_ev_specs**:

- **Heterogeneous Mode (True):** In this mode, the simulator ignores the default vehicle specifications in the **.yaml** file. Instead, it loads a list of vehicle profiles from an external JSON file, specified by the **ev_specs_file** parameter (e.g., **ev_specs_v2g_enabled2024.json**). This allows for the creation of a realistic fleet with diverse battery capacities, charging powers, and efficiencies. For instance, the fleet may include a **Peugeot 208** with a 46.3 kWh battery and a 7.4 kW charge rate, alongside a **Volkswagen ID.4** with a 77 kWh battery and an 11 kW charge rate. A vehicle is randomly selected from this list for each new arrival event.

- **Homogeneous Mode (False):** In this mode, the external JSON file is ignored. All vehicles created in the simulation are identical, and their characteristics are defined exclusively by the **ev:** block within the **.yaml** configuration file. The **battery_capacity** parameter in this block becomes the single source of truth for the entire fleet.

Empirical Calibration of the Degradation Model

A significant enhancement in this work is the move towards a more physically representative and flexible battery degradation model. While the underlying semi-empirical model for calendar and cyclic aging remains, the methodology for parameterizing it has been fundamentally improved, addressing previous inconsistencies. This is achieved through the **Fit_battery.py** script, a new utility for empirical model calibration. The script implements the following workflow:

1. **Data Loading:** It loads time-series data from real-world battery aging experiments. The expected data includes measurements of capacity loss over time, along with contextual variables like state of charge (SoC), temperature, and energy throughput.

2. **Model Fitting:** Using the **curve_fit** function from the SciPy library, the script fits the parameters of the **Qlost_model** (which combines calendar and cyclic aging) to the empirical data. This optimization process finds the physical constants (e.g., ϵ_0, ζ_0) that best explain the observed degradation.

3. **Parameter Export:** The script outputs the calibrated parameters. These values can then be used directly in the simulator's configuration, ensuring that the degradation model for a specific EV fleet is grounded in experimental evidence for that battery type.

This calibration workflow, integrated optionally into the main **run_experiments.py** script, elevates the simulation's fidelity. It allows the framework to move beyond a single, fixed degradation model (previously calibrated for a 78 kWh battery) and enables the creation of high-fidelity digital twins for a wide variety of EV batteries, provided that the necessary experimental data is available.

3.7 Simulation Framework Components

The simulation environment is constructed from a set of modular classes, each representing a key physical or logical component of the EV charging ecosystem. This section details the implementation and mathematical underpinnings of each component.

3.7.1 Electric Vehicle Model (ev.py)

The **EV** class is the most fundamental component, encapsulating the state, physical constraints, and charging behavior of a single electric vehicle. It serves as the primary dynamic element within the simulation.

State and Attribute Representation

Each EV instance is defined by a set of static and dynamic attributes that govern its behavior throughout its connection period.

- **Static Attributes:** These are defined upon the EV's arrival and remain constant. They include the maximum battery capacity ($E_i^{\max}$), a minimum operational capacity ($E_i^{\min}$), maximum AC charging power ($P_i^{\text{ch,max}}$), maximum discharging power ($P_i^{\text{dis,max}}$), charging efficiency (η_{ch}), discharging efficiency (η_{dis}), arrival time (t_i^{arr}), and departure time (t_i^{dep}).
- **Dynamic State Variables:** The core state variable is the current energy stored in the battery, denoted as $E_{i,t}$ at time step t. This state is updated at every simulation step.
- **Goal-Oriented Attributes:** The EV's objective is defined by its desired energy at departure, E_i^{des}. This is used to calculate user satisfaction.

Charging and Discharging Dynamics

The model implements a sophisticated two-stage charging process to accurately reflect the behavior of lithium-ion batteries, particularly the transition from the constant current (CC) to the constant voltage (CV) phase.

Two-Stage Charging Model (_charge method). The charging rate is not constant up to full capacity. The model captures the characteristic tapering of current as the battery approaches its maximum charge.

1. **Constant Current (CC) Phase:** When the State of Charge ($SoC_{i,t}$) is below a transition threshold, SoC^{trans}, the battery charges at a constant power. The rate of change of energy is limited by both the pilot signal from the charger, $P_{i,t}^{\text{pilot}}$, and the EV's own maximum charging power, $P_i^{\text{ch,max}}$. The effective charging power is $P_{i,t}^{\text{ch}} = \min(P_{i,t}^{\text{pilot}}, P_i^{\text{ch,max}})$. The energy evolution is:

$$E_{i,t} = E_{i,t-1} + \eta_{\text{ch}} \cdot P_{i,t}^{\text{ch}} \cdot \frac{\Delta t}{60} \tag{3.3}$$

   where Δt is the timescale in minutes.

2. **Constant Voltage (CV) Phase:** Once $SoC_{i,t}$ exceeds SoC^{trans}, the charging power begins to decrease, even if the pilot signal remains high. The implementation in the **_charge** method approximates this tapering effect using an exponential decay function. The model calculates a new transition SoC based on the pilot signal and then computes the new SoC using a formula that ensures a smooth, decaying charge rate as the SoC approaches 100

Discharging Model (_discharge method). Discharging is modeled as a linear process, constrained by the EV's maximum discharge power and the battery's minimum energy level. The energy delivered to the grid, $P_{i,t}^{\text{dis}}$, results in a larger energy withdrawal from the battery due to inefficiency:

$$E_{i,t} = E_{i,t-1} + \frac{P_{i,t}^{\text{dis}}}{\eta_{\text{dis}}} \cdot \frac{\Delta t}{60} \tag{3.4}$$

where $P_{i,t}^{\text{dis}}$ is negative by convention. The operation is constrained such that $E_{i,t} \geq E_i^{\text{min}}$.

User Satisfaction Metric

Upon departure at time t_i^{dep}, the performance of the charging service is quantified by a user satisfaction metric, S_i. This is defined as the ratio of the final energy level to the desired energy level, capped at 100%.

$$S_i = \min\left(1, \frac{E_{i,t_i^{\text{dep}}}}{E_i^{\text{des}}}\right) \tag{3.5}$$

This metric provides a crucial feedback signal for control algorithms, penalizing strategies that fail to meet user requirements.

3.7.2 Charging Station Model (ev_charger.py)

The **EV_Charger** class acts as an intermediary between the grid infrastructure and the individual EVs. It manages a collection of charging ports and enforces its own set of physical and operational constraints.

Role and Attributes

A charging station (CS) is defined by its number of ports (N_{ports}), its maximum aggregate charging and discharging currents ($I_{\text{CS}}^{\text{max,ch}}$, $I_{\text{CS}}^{\text{max,dis}}$), its operating voltage (V), and the number of phases. It serves as a local aggregation point for power and energy.

Operational Logic (step method)

At each simulation step, the **step** method executes the core logic of the charging station:

1. **Action Translation:** It receives a normalized action vector $a_t \in [-1, 1]^{N_{\text{ports}}}$. For each port j, it translates the normalized action $a_{j,t}$ into a physical pilot current signal, $I_{j,t}^{\text{pilot}}$.

$$I_{j,t}^{\text{pilot}} = \begin{cases} a_{j,t} \cdot I_{\text{CS}}^{\text{max,ch}} & \text{if } a_{j,t} > 0 \\ a_{j,t} \cdot |I_{\text{CS}}^{\text{max,dis}}| & \text{if } a_{j,t} < 0 \\ 0 & \text{if } a_{j,t} = 0 \end{cases} \tag{3.6}$$

2. **EV Interaction:** It passes this pilot current to the **step** method of the corresponding connected **EV** instance, which then calculates its actual power exchange based on its internal two-stage model.
3. **Power Aggregation:** It aggregates the actual power, $P_{j,t}$, from all its ports to determine its total power exchange with the grid at that time step.

$$P_{\text{CS},t} = \sum_{j=1}^{N_{\text{ports}}} P_{j,t} \tag{3.7}$$

4. **Economic Calculation:** It calculates the net profit for the time step based on the aggregated energy and real-time electricity prices (c_t^{buy}, c_t^{sell}).
5. **Departure Management:** It checks for departing EVs, collects their final user satisfaction scores, and frees up the corresponding ports.

3.7.3 Transformer Model (transformer.py)

The **Transformer** class represents a critical piece of grid infrastructure, modeling the point of common coupling for a group of charging stations and other local loads. Its primary function is to enforce an aggregate power limit.

Model Components

Each transformer is characterized by its maximum power capacity, $P_{\text{TR}}^{\max}$. It also manages two external time-series data streams:

- **Inflexible Load (P_t^{inflex}):** Represents the background power consumption from other sources (e.g., buildings) connected to the same transformer.
- **Solar Power (P_t^{PV}):** Represents local photovoltaic generation, which acts as a negative load.

Core Functionality

The transformer's logic is centered around monitoring its total load and checking for capacity violations.

Load Aggregation. At each time step t, the total power flowing through the transformer, $P_{\text{TR},t}$, is the algebraic sum of the inflexible load, the solar generation, and the sum of the power from all charging stations connected to it.

$$P_{\text{TR},t} = P_t^{\text{inflex}} + P_t^{\text{PV}} + \sum_{i \in \text{CSs connected to TR}} P_{\text{CS},i,t} \tag{3.8}$$

Overload Detection. The critical function is **is_overloaded**, which returns true if the absolute total power exceeds the transformer's rating. This condition serves as a primary constraint for the control problem.

$$\text{Overload} \iff |P_{\text{TR},t}| > P_{\text{TR}}^{\max} \tag{3.9}$$

Forecasting with Uncertainty. The model includes methods (**generate_..._forecast**) to simulate imperfect knowledge of future loads and generation. It creates forecasts by taking the true profiles and adding zero-mean Gaussian noise, where the standard deviation is a configurable percentage of the true value. This allows for the development of robust control strategies that can handle prediction errors.

3.7.4 Main Gym Environment (ev2gym_env.py)

The **EV2Gym** class is the central orchestrator of the entire simulation. It integrates all the aforementioned physical models into a cohesive whole and exposes it through the standardized **gymnasium.Env** interface, making it compatible with a wide range of RL algorithms.

Environment Structure and Main Loop

The class manages the high-level simulation flow.

Initialization (init). Upon creation, the environment reads a configuration file to instantiate all the necessary components: it creates the specified number of **Transformer** and **EV_Charger** objects, loads the EV arrival scenarios, and sets up electricity price profiles.

Simulation Step (step(actions)). This method drives the simulation forward by one time step (Δt). The sequence of operations is as follows:

1. It receives a single, flat action vector containing the control signals for every charging port in the entire system.
2. It slices this vector and passes the appropriate actions to each **EV_Charger**'s **step** method.
3. This triggers the **step** methods of all connected **EV**s, updating their internal states ($E_{i,t}$).
4. It aggregates the resulting power at each **Transformer** and checks for overload conditions.
5. It processes the EV queue, spawning new EVs that are scheduled to arrive at the next time step.
6. It computes a scalar reward signal using a user-defined reward function.

7. It constructs the next state observation using a user-defined state representation function.

8. It checks for termination conditions (e.g., simulation end, constraint violation).

9. It returns the standard '(observation, reward, terminated, truncated, info)' tuple.

Reset (reset). This method re-initializes the entire environment to a starting state, allowing for the start of a new simulation episode, which could represent a new day or a different random scenario.

3.8 Rule-Based Heuristic Baselines

In addition to sophisticated optimization-based (MPC) and learning-based (DRL) controllers, it is crucial to benchmark performance against simple, rule-based heuristic algorithms. These heuristics serve as fundamental baselines to quantify the value added by more intelligent strategies. The `heuristics.py` module implements several such baseline controllers.

As-Fast-As-Possible

This is the most straightforward and intuitive charging strategy. The As-Fast-As-Possible (AFAP) controller, implemented as `ChargeAsFastAsPossible`, commands all connected EVs to charge at their maximum permissible rate at all times, irrespective of electricity prices, grid constraints, or user needs beyond being ready by departure. Its objective is singular: minimize the time to reach full capacity. While simple, it often leads to undesirable outcomes such as high charging costs during peak price hours and contributing to grid congestion. A variation, `ChargeAsFastAsPossibleToDesiredCapacity`, stops charging once the user's specified desired SoC is met, rather than continuing to 100%.

As-Late-As-Possible

The As-Late-As-Possible (ALAP) strategy, implemented in the `ChargeAsLateAsPossible` class, represents the opposite temporal extreme to AFAP. For each EV, it calculates the latest possible moment to initiate charging at maximum power to ensure the battery is full precisely at the scheduled departure time. This is achieved by determining the minimum number of time steps required to charge and deferring the start of the charging session until that point. This approach can shift load away from peak periods if those peaks occur early in the parking session, but it is inflexible and cannot respond to unexpected price drops or grid signals. It also carries the risk of failing to fully charge the vehicle if the departure time is earlier than expected. The `ChargeAsLateAsPossibleToDesiredCapacity` variant applies the same logic but targets the desired SoC.

Round Robin Schedulers

The `RoundRobin` family of heuristics attempts a more nuanced approach to meet a given power setpoint. The basic `RoundRobin` algorithm maintains a queue of connected EVs. To meet the power target, it calculates how many EVs need to be charged simultaneously (based on an average charging power) and activates that number of EVs from the front of the queue. In each time step, the EVs that were just charged are moved to the back of the queue, ensuring a fair distribution of charging opportunities over time.
More advanced versions like `RoundRobin_GF` (Grid-Friendly) are designed to track a power setpoint more precisely. Instead of activating a fixed number of EVs, it sequentially activates EVs and sums their potential power contribution until the setpoint is met or exceeded, providing a finer level of control.

3.9 Reinforcement Learning Formulation

Since the control problem is formalized as a Markov Decision Process (MDP); defined by the tuple (S, A, P, R), now will be explained each tuple (exception makes P).

3.9.1 State Space (S)

The state $s_t \in S$ is a feature vector providing a comprehensive snapshot of the environment at time t. The framework uses the **V2G_profit_max_loads** state representation, which is specifically designed to provide the agent with all necessary information for complex, forward-looking profit maximization strategies. It combines temporal information, market signals, grid constraints, and detailed data for each individual electric vehicle (There is also the version for the no load case but without loss of generality we can consider only this one).
The state vector s_t is a flattened concatenation of several distinct components [32](There are no key changes from the paper) :

$$s_t = [k_t \oplus P_{t-1}^{\text{total}} \oplus \mathbf{c}_{t \to t+H-1} \oplus (\bigoplus_{j=1}^{N_T} s_{t,j}^{\text{tr}}) \oplus (\bigoplus_{i=1}^{N_{CS}} s_{t,i}^{\text{ev}})] \tag{3.10}$$

where $\oplus$ denotes vector concatenation, H is the forecast horizon (e.g., 20 steps), N_T is the number of transformers, and N_{CS} is the total number of charging stations. The components are defined as follows:

- **Temporal and Power Information**: The initial part of the state vector grounds the agent in the current context.
  - k_t: The current timestep of the simulation. This provides a sense of progression through the episode.
  - P_{t-1}^{total}: The total power consumed by all charging stations in the previous timestep, $t-1$. This gives the agent immediate feedback on the outcome of its last action.

- **Market Forecast**: A vector of future electricity prices, providing the primary signal for economic optimization.
  - $\mathbf{c}_{t \to t+H-1} \in \mathbb{R}^H$: A vector containing the forecast of electricity charging prices for the next H timesteps. This is crucial for planning charge/discharge cycles to maximize profit.
- **Grid Constraint Forecasts (per Transformer)**: For each transformer j, the state includes forecasts for local grid conditions. This is essential for preventing overloads and ensuring system stability.
  - $\mathbf{P}^{\text{netload}}_{j,t \to t+H-1} \in \mathbb{R}^H$: A vector forecasting the net inflexible load for the next H timesteps. This is calculated as (Inflexible Loads - PV Generation).
  - $\mathbf{P}^{\text{limit}}_{j,t \to t+H-1} \in \mathbb{R}^H$: A vector representing the transformer's maximum power limit over the next H timesteps.
- **EV Status (per Charging Station)**: For each charging station i, the state provides the critical information of the connected vehicle. If a station is empty, these values are replaced with zeros.
  - $SoC_{i,t} \in [0, 1]$: The current SoC of the connected EV.
  - $\Delta t^{\text{dep}}_{i,t} \in \mathbb{Z}^+$: The remaining time until the EV's scheduled departure, measured in timesteps. This represents the window of opportunity for charging.

This rich state representation provides the agent with a multi-faceted view of the system, combining immediate feedback (P^{total}_{t-1}), predictive market and grid information ($\mathbf{c}$, $\mathbf{P}^{\text{netload}}$, $\mathbf{P}^{\text{limit}}$), and the specific needs and constraints of each vehicle (SoC, Δt^{dep}). This enables it to learn sophisticated, forward-looking policies that can anticipate future conditions to balance profitability with physical and user-defined constraints.

3.9.2 Action Space (A)

The action $a_t \in A$ is a continuous vector in $\mathbb{R}^N$, where N is the total number of charging ports. For each port i, the command $a_i(t) \in [-1, 1]$ is a normalized value that is translated into a physical pilot **current** signal by the charging station model. A positive value corresponds to charging, while a negative value corresponds to discharging (V2G).

3.9.3 Reward Function (R)

The reward function $R(t)$ encodes the objectives of the control agent. The framework allows for the selection of different reward functions from the **reward.py** module to suit various goals, such as maximizing profit, minimizing degradation, or a combination thereof. The original [32] reward function was a base to deploy a new, more reliable, and adaptive one.

3.10 A History-Based Adaptive Reward for Profit Maximization

To guide the learning agent towards policies that are both highly profitable and operationally reliable, we designed a novel, history-based adaptive reward function, named `FastProfitAdaptiveReward` (that is a reward function defined in reward.py). This function dynamically balances its objectives, shifting focus between profit maximization and constraint satisfaction based on the agent's recent performance. The core principle is to allow aggressive profit-seeking when the system is stable and user satisfaction is high, but to automatically curtail this ambition to focus on stability and service quality when performance degrades.
The total reward at each timestep t, R_t, is formulated as a dynamically scaled net economic profit Π_t minus any active penalties for user dissatisfaction P_t^{sat} or transformer overload P_t^{tr}.

$$R_t = (\mu_p \cdot \Pi_t) - P_t^{\text{sat}} - P_t^{\text{tr}} \tag{3.11}$$

The complete logic for this calculation is detailed in Algorithm 1. The key components are:

- Π_t is the net economic profit at timestep t.
- μ_p is the **dynamic profit multiplier**, which scales based on historical performance.
- P_t^{sat} and P_t^{tr} are the adaptive penalties for user dissatisfaction and transformer overload.
- $\bar{s}_{hist}$ and $\bar{f}_{ov}$ are the average user satisfaction and overload frequency over a recent history window.

Algorithm 1 Dynamic & History-Based Adaptive Reward (`FastProfitAdaptiveReward`)

Require: Current profit/cost C_t, user satisfaction list S_t, overload amount O_t; history queues H_{sat} (satisfaction) and H_{ov} (overload frequency)
Ensure: Adaptive reward value R_t

Initialize base parameters: $\mu_{base} = 0.1$, $\mu_{max\_bonus} = 0.9$
function CALCULATEREWARD(C_t, S_t, O_t)
  1. Historical performance metrics
  Compute average satisfaction $\bar{s} = \text{Average}(H_{sat})$ (default 1.0 if empty)
  Compute average overload frequency $\bar{f}_{ov} = \text{Average}(H_{ov})$ (default 0.0 if empty)
  2. Dynamic profit multiplier
  $b_{sat} = \max(0, (\bar{s} - 0.9)/0.1)$ ▷ Bonus for high satisfaction
  $b_{ov} = \max(0, 1 - (\bar{f}_{ov}/0.2))$ ▷ Bonus for low overload frequency
  $\mu_p = \mu_{base} + (\mu_{max\_bonus} \cdot b_{sat} \cdot b_{ov})$
  Compute profit component: $R_{profit} = C_t \cdot \mu_p$
  3. Adaptive satisfaction penalty
  $\sigma_{sat} = 200.0 \cdot (1 - \bar{s})^2$ ▷ Quadratic penalty growth
  $P_{sat} \leftarrow 0$
  if S_t not empty and $\min(S_t) < 0.95$ **then**
    $P_{sat} = \sigma_{sat} \cdot (1 - \min(S_t))$
  end if
  4. Adaptive overload penalty
  $P_{ov} \leftarrow 0$
  if $O_t > 0$ **then**
    $\sigma_{ov} = 50.0 \cdot \bar{f}_{ov}$ ▷ Penalty grows linearly with overload frequency
    $P_{ov} = 5.0 + (\sigma_{ov} \cdot O_t)$
  end if
  5. Final reward computation and history update
  $R_t = R_{profit} - P_{sat} - P_{ov}$
  Append Average(S_t) to H_{sat}
  Append (1 if $O_t > 0$ else 0) to H_{ov}
  return R_t
end function

3.10.1 Dynamic Economic Profit

The foundation of the reward is the economic profit Π_t, but its contribution is scaled by a dynamic multiplier, μ_p. This multiplier is designed to increase the agent's focus on profit only when it is performing well on other key metrics. It is calculated based on a base value ("$\mu_{\text{base}} = 0.1$") plus a bonus that depends on historical satisfaction ("$\bar{s}_{hist}$") and overload frequency ("$\bar{f}_{ov}$"). The agent is maximally rewarded for profit ("$\mu_p \to 1.0$") only when satisfaction is near-perfect and overloads are rare. If performance on either metric degrades, the multiplier automatically decreases, reducing the incentive for profit-seeking and forcing the agent to prioritize correcting its behavior.

3.10.2 Adaptive User Satisfaction Penalty

The penalty for failing to meet user charging demands, P_t^{sat}, adapts based on the system's recent performance. The environment maintains a short-term memory of user satisfaction scores, from which an average historical score, $\bar{s}_{hist}$, is calculated. A *satisfaction severity multiplier*, σ_{sat}, is then computed, which grows quadratically as the historical average satisfaction drops. This ensures that as performance degrades, the consequences of new failures become substantially more severe.

$$\sigma_{sat} = 200.0 \cdot (1 - \bar{s}_{hist})^2 \tag{3.12}$$

A penalty is only applied if the minimum satisfaction score among currently connected users, $S_{\min}$, falls below a critical threshold (95%). The magnitude of the penalty is the product of the adaptive multiplier and the current satisfaction deficit.

3.10.3 Adaptive Transformer Overload Penalty

The transformer overload penalty, P_t^{tr}, operates on a similar adaptive principle, responding to the recent frequency of overloads, F_{hist}^{tr}. If the total power drawn, O_t, exceeds the transformer's limit, a penalty is applied. This penalty consists of a fixed base amount plus an adaptive component, which scales with both the historical frequency of overloads and the magnitude of the current overload.

$$P_t^{\text{tr}} = 5.0 + (50.0 \cdot F_{hist}^{\text{tr}}) \cdot O_t, \quad \text{if } O_t > 0 \tag{3.13}$$

This structure enforces a critical lesson: while an occasional, minor overload might be tolerable, habitual overloading becomes increasingly costly as penalties escalate, forcing the agent to respect physical constraints.

3.11 Reinforcement Learning Algorithms

This work benchmarks several state-of-the-art DRL algorithms. The following sections provide a detailed mathematical description of the selected off-policy, actor-critic algorithms that form the core of the experimental comparison.

Soft Actor-Critic (SAC)

SAC is an off-policy actor-critic algorithm designed for **continuous action spaces** that optimizes a stochastic policy. Its distinguishing feature is the explicit incorporation of entropy maximization into the objective function, which promotes exploration and yields policies that are more robust to model inaccuracies and environmental perturbations. Unlike traditional RL algorithms that focus solely on reward maximization, SAC aims to maximize both the expected cumulative reward and the entropy of the policy distribution.

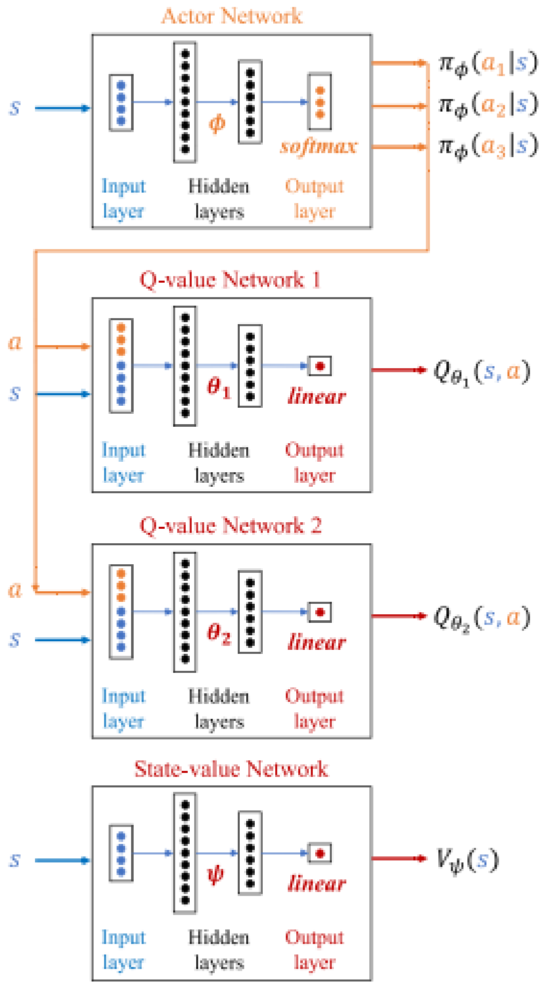


Figure 3.3: SAC actor-critic architecture. The actor network outputs a stochastic policy parameterized by a mean and standard deviation, while two separate Q-networks (critics) estimate state-action values. The minimum of the two Q-values is used for policy updates to mitigate overestimation bias. Image from [34].

The objective function is:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[r(s_t, a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))\right]$$

where $\mathcal{H}$ is the entropy of the policy π, defined as $\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a_t\sim\pi}[\log \pi(a_t|s_t)]$, and α is the temperature parameter, which controls the trade-off between reward exploitation and entropy-driven exploration. A higher α encourages more stochastic behavior, while a lower α biases the policy toward deterministic, reward-maximizing actions. The complete training procedure is outlined in Algorithm 2.

Algorithm 2 Soft Actor-Critic (SAC)

```
Initialize critic networks Q_{θ1}, Q_{θ2} and actor network π_φ with random parameters.
Initialize target networks θ̄1 ← θ1, θ̄2 ← θ2.
Initialize replay buffer D.
Initialize temperature parameter α.
for each timestep t = 1, . . . , T do
    Select action with exploration: a_t ∼ π_φ(·|s_t).
    Execute action a_t and observe reward r_t and next state s_{t+1}.
    Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D.
    if t is a multiple of update frequency then
        Sample a minibatch of N transitions {(s_j, a_j, r_j, s_{j+1})}_{j=1}^N from D.
                                                ▷ Compute the target for the Q-functions
        With next actions ã_{j+1} ∼ π_φ(·|s_{j+1}):
        y_j ← r_j + γ (min_{i=1,2} Q_{θ̄_i}(s_{j+1}, ã_{j+1}) − α log π_φ(ã_{j+1}|s_{j+1}))
                                                ▷ Update the critic networks
        Update critic parameters θ_i by one step of gradient descent on:
        L(θ_i) = (1/N) Σ_{j=1}^N (Q_{θ_i}(s_j, a_j) − y_j)^2 for i = 1, 2.
                                                ▷ Update the actor network
        With new actions ã_j ∼ π_φ(·|s_j):
        Update policy parameters φ by one step of gradient ascent on:
        J(φ) = (1/N) Σ_{j=1}^N (min_{i=1,2} Q_{θ_i}(s_j, ã_j) − α log π_φ(ã_j|s_j)).
                                                ▷ Update the temperature parameter (optional)
        Update α by one step of gradient descent on:
        J(α) = (1/N) Σ_{j=1}^N (−α(log π_φ(ã_j|s_j) + H̄)), where H̄ is a target entropy.
                                                ▷ Update the target networks
        θ̄_i ← τθ_i + (1 − τ)θ̄_i for i = 1, 2.
    end if
end for
```

Implementation Details The implementation of SAC is built upon the robust, industry-standard **Stable-Baselines3** library, which provides a highly optimized and well-tested version of the algorithm. The standard `SAC` class from this library is used directly, leveraging its PyTorch-based backend for efficient training and inference.

SAC employs a soft Q-function, which incorporates the policy entropy into the value estimation. The soft Q-function is trained to minimize the soft Bellman residual:

$$L(\theta_Q) = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim D}\left[\left(Q(s_t, a_t) - \left(r_t + \gamma V_{\bar{\psi}}(s_{t+1})\right)\right)^2\right]$$

where D is the replay buffer, γ is the discount factor, and the soft state value function V is defined as:

$$V_{\text{soft}}(s_t) = \mathbb{E}_{a_t\sim\pi}[Q_{\text{soft}}(s_t, a_t) - \alpha \log \pi(a_t|s_t)]$$

To mitigate the positive bias inherent in Q-learning, SAC employs two independent Q-networks (implementing the Clipped Double-Q technique) and uses the minimum of the two target Q-values during the Bellman update. This conservative approach prevents overestimation from propagating through the learning process. Additionally, SAC can automatically tune the temperature parameter α during training by treating it as a constrained optimization problem, ensuring that the policy entropy remains above a target threshold.

Deep Deterministic Policy Gradient + PER (DDPG+PER)

DDPG is an off-policy algorithm that concurrently learns a deterministic policy $\mu(s|\theta^{\mu})$ and a Q-function $Q(s, a|\theta^{Q})$. It extends the Deterministic Policy Gradient (DPG) theorem to deep neural networks, enabling continuous control in high-dimensional state spaces. Unlike stochastic policy methods, DDPG directly outputs a single action for each state, which can be advantageous in environments where deterministic behavior is preferred or when the action space is high-dimensional. The full procedure, enhanced with PER, is detailed in Algorithm 3.

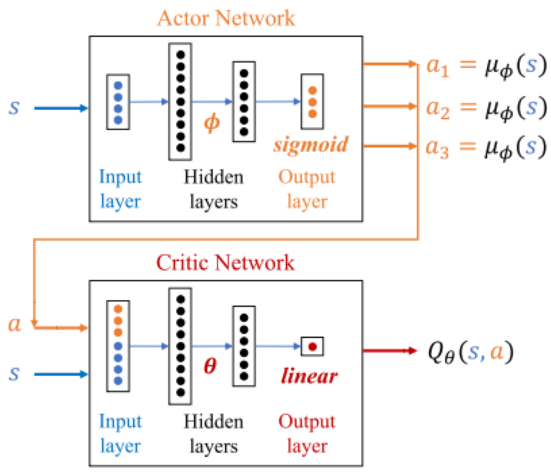


Figure 3.4: DDPG actor-critic architecture. The actor network produces a deterministic action, which is evaluated by the Q-network (critic). Both networks maintain slowly-updating target networks (μ' and Q') for stable Bellman updates. During training, exploration noise is added to the actor's output. Image from [34].

Algorithm 3 DDPG with Prioritized Experience Replay (PER)

Initialize critic network $Q(s, a|\theta^Q)$ and actor network $\mu(s|\theta^\mu)$ with random parameters.
Initialize target networks $Q' \leftarrow Q$, $\mu' \leftarrow \mu$.
Initialize Prioritized Replay Buffer D with parameters α, β.
for each episode **do**
  Initialize a random process $\mathcal{N}$ for action exploration.
  Receive initial state s_1.
  for each timestep $t = 1, \ldots, T$ **do**
    Select action: $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$.
    Execute action a_t and observe reward r_t and next state s_{t+1}.
    Store transition (s_t, a_t, r_t, s_{t+1}) in D with maximal priority.
    ▷ Sample from buffer and update
    Sample minibatch of N transitions $\{(s_j, a_j, r_j, s_{j+1})\}_{j=1}^N$ from D with probabilities $P(j) = \frac{p_j^\alpha}{\sum_k p_k^\alpha}$.
    Compute importance-sampling (IS) weights $w_j = (N \cdot P(j))^{-\beta}$.
    ▷ Update the critic network
    Compute targets: $y_j = r_j + \gamma Q'(s_{j+1}, \mu'(s_{j+1}|\theta^{\mu'})|\theta^{Q'})$.
    Compute TD-errors: $\delta_j = y_j - Q(s_j, a_j|\theta^Q)$.
    Update critic by minimizing the weighted loss: $L(\theta^Q) = \frac{1}{N}\sum_{j=1}^N w_j \cdot \delta_j^2$.
    Update transition priorities in D: $p_j \leftarrow |\delta_j|$.
    ▷ Update the actor network
    Update actor using the sampled policy gradient:
    $\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_{j=1}^N \nabla_a Q(s, a|\theta^Q)|_{s=s_j, a=\mu(s_j)} \nabla_{\theta^\mu}\mu(s_j|\theta^\mu)$.
    ▷ Update the target networks
    $\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$.
    $\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$.
  end for
end for

- **Critic Update:** The critic is updated by minimizing the mean-squared Bellman error, analogous to Q-learning. Target networks (Q' and μ') are used to stabilize training by providing consistent targets during the temporal difference update.

$$L(\theta^Q) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim D}\left[(y_t - Q(s_t, a_t|\theta^Q))^2\right]$$

  where the target y_t is given by:

$$y_t = r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1}|\theta^{\mu'})|\theta^{Q'})$$

  The target networks are updated via soft updates (polyak averaging): $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$, where $\tau \ll 1$ is a small constant.

- **Actor Update:** The actor is updated using the deterministic policy gradient theorem, which states that the gradient of the expected return with respect to the policy parameters can be computed as:

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t \sim D}[\nabla_a Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s_t|\theta^\mu)]$$

This gradient ascent step improves the policy by moving it in the direction that increases the Q-value according to the critic's estimate.

- **Prioritized Experience Replay (PER):** This work enhances DDPG with PER, which addresses a fundamental limitation of uniform experience replay. Instead of sampling transitions uniformly from the replay buffer D, PER samples transitions according to their temporal-difference (TD) error, prioritizing experiences where the model's predictions are most inaccurate and therefore most informative. The probability of sampling transition i is:

$$P(i) = \frac{p_i^\beta}{\sum_k p_k^\beta}$$

where $p_i = |\delta_i| + \epsilon$ is the priority based on the TD-error $\delta_i = y_t - Q(s_t, a_t)$, ϵ is a small constant to ensure non-zero probabilities, and β controls the degree of prioritization. To correct for the bias introduced by non-uniform sampling, PER applies importance-sampling (IS) weights during the gradient update:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\alpha$$

where α controls the amount of bias correction. These weights ensure that the gradient estimator remains unbiased despite the non-uniform sampling distribution.

Implementation Details To integrate Prioritized Experience Replay (PER) with DDPG, a custom class, `CustomDDPG`, was developed in the `ev2gym/rl_agent/custom_algorithms.py` module. This class inherits from the standard `DDPG` agent provided by **Stable-Baselines3**. The core `train` method is overridden to replace the default uniform-sampling replay buffer with one that supports prioritized sampling. This involves calculating TD-errors for each transition, updating their priorities in the buffer, and using the resulting importance-sampling weights during the critic update, thereby focusing the learning process on the most informative experiences.

Truncated Quantile Critics (TQC)

TQC represents a significant advancement over standard SAC by modeling the entire distribution of returns rather than just its expected value. This distributional perspective, combined with a novel truncation mechanism, provides superior protection against Q-value overestimation—a persistent pathology in off-policy RL that can lead to catastrophic policy degradation. The algorithm's logic is summarized in Algorithm 4.

Algorithm 4 Truncated Quantile Critics (TQC)

Initialize critic ensemble $\{Q_{\theta_i}\}_{i=1}^{N}$ and actor π_ϕ with random parameters.
Initialize target networks $\{\bar{\theta}_i \leftarrow \theta_i\}_{i=1}^{N}$ and $\bar{\phi} \leftarrow \phi$.
Initialize replay buffer D.
Set number of quantiles N and number of quantiles to drop k.
Define quantile fractions $\tau_i = \frac{i-0.5}{N}$ for $i = 1, \ldots, N$.
for each timestep $t = 1, \ldots, T$ **do**
    Select action with exploration: $a_t \sim \pi_\phi(\cdot|s_t)$.
    Execute action a_t and observe reward r_t and next state s_{t+1}.
    Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D.
    if t is a multiple of update frequency **then**
        Sample a minibatch of M transitions $\{(s_j, a_j, r_j, s_{j+1})\}_{j=1}^{M}$ from D.
        ▷ Compute the distributional target
        Sample next actions from target policy: $\tilde{a}_{j+1} \sim \pi_{\bar{\phi}}(\cdot|s_{j+1})$.
        Get next-state quantile estimates from target critics: $\{Q_{\bar{\theta}_l}(s_{j+1}, \tilde{a}_{j+1})\}_{l=1}^{N}$.
        Sort the estimates and drop the top k: $\{Q_{\text{sorted}}\}_{l=1}^{N-k}$.
        Form the target quantiles: $y_{j,l} = r_j + \gamma Q_{\text{sorted},l}$ for $l = 1, \ldots, N-k$.
        ▷ Update the critic networks
        Update each critic θ_i by minimizing the sum of quantile Huber losses:
        $L(\theta_i) = \frac{1}{M(N-k)} \sum_{j=1}^{M} \sum_{l=1}^{N-k} L_{QH}^{\tau_i}\left(y_{j,l} - Q_{\theta_i}(s_j, a_j)\right)$.
        ▷ Update the actor network
        Update policy ϕ by gradient ascent on:
        $J(\phi) = \frac{1}{M} \sum_{j=1}^{M} \left(\frac{1}{N} \sum_{i=1}^{N} Q_{\theta_i}(s_j, \tilde{a}_j) - \alpha \log \pi_\phi(\tilde{a}_j|s_j)\right)$, where $\tilde{a}_j \sim \pi_\phi(\cdot|s_j)$.
        ▷ Update the target networks
        $\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i$ for $i = 1, \ldots, N$.
        $\bar{\phi} \leftarrow \tau\phi + (1-\tau)\bar{\phi}$.
    end if
end for

- **Distributional Learning:** TQC employs an ensemble of N critic networks, $\{Q_{\phi_i}(s,a)\}_{i=1}^{N}$, where each network is trained to estimate a specific quantile τ_i of the return distribution. The target quantiles are implicitly defined as $\tau_i = \frac{i-0.5}{N}$, uniformly covering the cumulative distribution function. The critics are trained by minimizing the quantile Huber loss, L_{QH}, which is a robust asymmetric loss function that penalizes overestimation and underestimation differently:
$$L_{QH}(\delta) = |\tau_i - \mathbb{I}_{\{\delta<0\}}| \cdot L_\kappa(\delta)$$
where L_κ is the Huber loss with threshold κ, and δ is the temporal difference error.

- **Distributional Target Calculation:** A distributional target is constructed for the Bellman update. First, an action is sampled from the target policy for the next state: $\tilde{a}_{t+1} \sim \pi_{\theta'}(\cdot|s_{t+1})$. Then, a set of N Q-value estimates for the next state is obtained from the N target critic networks: $\{Q_{\phi'_j}(s_{t+1}, \tilde{a}_{t+1})\}_{j=1}^{N}$. This ensemble provides a rich representation of the value distribution's uncertainty.

- **Truncation:** This is the central innovation of TQC. To combat overestimation, the algorithm discards the k largest Q-value estimates from the set of N target values. This truncation removes the most optimistic (and typically most biased) estimates, which are the primary drivers of overestimation in ensemble methods. By dropping these outliers, TQC obtains more conservative and stable target values. The remaining $(N - k)$ quantile estimates are then used to compute the Bellman target, effectively implementing a form of pessimism that counteracts the inherent optimism bias in temporal-difference learning.

- **Critic Update:** The target value for updating the i-th critic is formed using the Bellman equation with the truncated set of next-state Q-values. The overall critic loss is the sum of the quantile losses across all critics:

$$L(\phi) = \sum_{i=1}^{N} \mathbb{E}_{(s,a,r,s')\sim D} \left[L_{QH} \left(r + \gamma Q_{\text{trunc}}(s', \tilde{a}') - Q_{\phi_i}(s, a) \right) \right]$$

where Q_{trunc} represents the value derived from the truncated set of target quantiles. The actor is then updated using the mean of all (non-truncated) critic estimates, similar to standard SAC, but benefiting from the improved stability of the distributional value estimates.

Implementation Details The TQC algorithm is leveraged from the **SB3-Contrib** library, a collection of community-contributed extensions to Stable-Baselines3. Using the library's standard `TQC` implementation allows the framework to benefit from this state-of-the-art distributional RL algorithm without requiring a custom implementation from scratch. The implementation maintains the entropy-regularized objective of SAC while incorporating the distributional critics and truncation mechanism that define TQC's superior performance characteristics.

3.12 Model Predictive Control (MPC) Implementation

The framework includes a Model Predictive Control implementation in **pulp_mpc.py**, which serves as a powerful classical optimization baseline.

3.13 Online MPC Formulation (PuLP Implementation)

The MPC implemented with PuLP solves a profit maximization problem at each control interval over a finite prediction horizon H. This formulation is designed for online, real-time control, where decisions are made based on the current system state and future predictions. This approach is often termed an "implicit" MPC because the control law is not pre-computed; instead, it is found by solving a full optimization problem at every control interval.

Implementation Details This online controller is implemented as the **OnlineMPC_Solver** class within the **pulp_mpc.py** module. At each invocation of its **get_action** method, it dynamically constructs and solves a Mixed-Integer Linear Programming (MILP) using the **PuLP** modeling library. PuLP acts as a high-level modeling interface for the underlying **CBC (COIN-OR Branch and Cut)** solver, an open-source solver capable of handling MILPs.

3.13.1 The CBC Solver and Branch and Cut Algorithm

The core of the online MPC controller is the CBC (COIN-OR Branch and Cut) solver, which is tasked with solving the MILP formulated at each time step. CBC is a powerful, open-source solver that implements a sophisticated algorithm known as **Branch and Cut**; which is an exact method for solving MILPs,is an improved version of the Branch-and-Bound algorithm. The algorithm is a hybrid of two powerful techniques:**the Branch and Bound algorithm and the Cutting Plane method**.

The Branch and Bound Method

The Branch and Bound algorithm is a systematic method for solving optimization problems with integer variables. It works by exploring a tree of subproblems, where each node in the tree represents a more constrained version of the original problem. The process can be summarized as follows:

1. **Relaxation:** The algorithm starts by solving a "relaxed" version of the problem, where the integer constraints are ignored (i.e., integer variables are treated as continuous). This provides a fractional solution and an initial upper bound (for a maximization problem) on the optimal integer solution.

2. **Branching:** If the solution to the relaxed problem contains fractional values for variables that should be integers, the algorithm "branches" on one of these fractional variables(As in the B&B). For a variable x with a fractional value (e.g., 2.5), it creates two new subproblems (branches): one with the added constraint $x \leq 2$ and another with the constraint $x \geq 3$. This process effectively divides the problem space.

3. **Bounding:** For each new subproblem, the relaxed version is solved. The solution provides a local upper bound for that branch. If this bound is worse than the best integer solution found so far (the "incumbent"), that entire branch can be "pruned" (discarded), as it cannot lead to a better solution.

4. **Termination:** The algorithm continues to branch and prune until an integer solution is found that is proven to be optimal (i.e., all other active branches have been explored and yield no better solutions).

The Cutting Plane Method

The Cutting Plane method enhances the Branch and Bound process by adding new constraints, known as "cutting planes" or "cuts," to the problem. These cuts are carefully constructed to "cut off" parts of the feasible region of the relaxed problem that do not contain any integer solutions, without removing any valid integer solutions. The key idea is that by adding these cuts, the feasible region of the relaxed problem becomes a tighter approximation of the true integer feasible region (the convex hull of the integer solutions). This leads to several benefits:

- The solutions to the relaxed problems are closer to being integer-feasible.
- The upper bounds generated are tighter, allowing the algorithm to prune branches more effectively and converge faster.

Branch and Cut in CBC for MPC

The Branch and Cut algorithm, as implemented in CBC, combines these two methods. At each node of the Branch and Bound tree, the solver may generate and add cutting planes to the subproblem before deciding whether to branch further. This synergy makes the solver highly efficient. In the context of the MPC for V2G scheduling, this is particularly important. The binary variables $\omega_{i,t}^{\text{ch}}$ and $\omega_{i,t}^{\text{dis}}$ (which decide whether to charge or discharge) make the problem a MILP. A naive exploration of all combinations would be computationally intractable, especially with a long prediction horizon and many EVs. The Branch and Cut algorithm allows CBC to intelligently navigate the vast search space, finding the optimal charging and discharging schedule that maximizes profit while respecting all physical and operational constraints, and doing so within a timeframe suitable for online control.

3.13.2 Mathematical Formulation and Linearization

The implementation of the MPC controller, contained in the **pulp_mpc.py** script, translates the optimization problem into a MILP that can be solved efficiently using the open-source CBC (COIN-OR Branch and Cut) solver. However, the original mathematical formulation presented in some research papers, such as the one by Orfanoudakis et al. [32], is inherently non-linear. To make the problem tractable in a real-time control context, several important simplifications and corrections were necessary.

The Original Problem: A Complex MINLP

The original theoretical formulation of the problem is a **Mixed-Integer Non-Linear Program (MINLP)**, which is an optimization problem with both continuous and integer variables, and with non-linear constraints or objective function. The non-linearity arises from the very definition of power and current, where a continuous variable (the current **I**) is multiplied by a binary variable (the state **omega**).

Specifically, the sources of non-linearity in the reference formulation are:

- **Equations (10) and (11) - Power Definition:** The charging (P^{ch}) and discharging (P^{dis}) power is defined as the product of current, voltage, and efficiency, but also includes the binary state variable. For example: $P^{\text{ch}} = I^{\text{ch}} \cdot V \cdot \eta^{\text{ch}} \cdot \omega^{\text{ch}}$. This is a multiplication of two variables (I^{ch} and ω^{ch}), making the expression quadratic and therefore non-linear.

- **Equation (17) - Charging Station Current:** The total current for a charging station (I^{cs}) is also defined as a sum of non-linear terms: $I^{\text{cs}} = \sum(I^{\text{ch}} \cdot \omega^{\text{ch}} + I^{\text{dis}} \cdot \omega^{\text{dis}})$.

Solving a MINLP is a computationally very arduous task. It requires specialized, often commercial, solvers and a computation time that can grow exponentially, making it unsuitable for a control application that requires responses in a matter of seconds.

The Linearization Strategy: Transforming the MINLP into a MILP

Algorithm 5 Online MPC Solver for EV Charging and Discharging Control

---

Require: Environment with transformer, charging stations, EVs; prediction horizon H_p; control horizon H_c; timescale Δt
Ensure: Optimal charging/discharging action vector $\mathbf{a}$ at each time step

Initialize parameters:
Compute $N_c = \max(1, \lfloor H/2 \rfloor)$ if control horizon is "half"
for each simulation step k **do**
  Determine effective horizon: $H_{eff} = \min(H, T_{sim} - k)$
  if $H_{eff} \leq 0$ **then**
    return a $=$ **0**
  end if
  Extract transformer limits $P^{\text{tr}}_{max}(t)$, load $P_L(t)$, PV $P_{PV}(t)$, prices $c_{ch}(t)$, $c_{dis}(t)$ for $t = 0 \ldots H_{eff} - 1$
  Identify active EVs:
  for all active EV i **do**
    Get $E_i^{\max}$, $E_i(0)$, $\eta_{ch,i}$, $\eta_{dis,i}$, $I^{ch}_{i,\max}$, $I^{dis}_{i,\max}$
  end for
  Define variables: $I^{ch}_{i,t} \geq 0$, $I^{dis}_{i,t} \geq 0$, $\omega^{ch}_{i,t}, \omega^{dis}_{i,t} \in \{0, 1\}$, $E_{i,t} \geq 0$
  Define power: $P^{ch}_{i,t} = I^{ch}_{i,t} V_i \sqrt{\phi_i}$, $P^{dis}_{i,t} = I^{dis}_{i,t} V_i \sqrt{\phi_i}$
  Objective: Maximize profit

$$\max \sum_{i,t} \left[-P^{ch}_{i,t} c_{ch}(t) + P^{dis}_{i,t} c_{dis}(t) \right] \Delta t$$

  Subject to:

$$
\begin{aligned}
&\omega^{ch}_{i,t} + \omega^{dis}_{i,t} \leq 1 && \text{(No simultaneous charge/discharge)} \\
&I^{ch}_{i,t} \leq I^{ch}_{i,\max} \omega^{ch}_{i,t} && \text{(Charging current limit)} \\
&I^{dis}_{i,t} \leq I^{dis}_{i,\max} \omega^{dis}_{i,t} && \text{(Discharging current limit)} \\
&E_{i,t} = E_{i,t-1} + (P^{ch}_{i,t} \eta_{ch,i} - \tfrac{P^{dis}_{i,t}}{\eta_{dis,i}}) \Delta t && \text{(Battery dynamics)} \\
&E_i^{\min} \leq E_{i,t} \leq E_i^{\max} && \text{(Energy limits)} \\
&E_{i,t_{dep}} \geq E_i^{des} && \text{(Desired departure energy)} \\
&\sum_i (P^{ch}_{i,t} - P^{dis}_{i,t}) + P_L(t) + P_{PV}(t) \leq P^{\text{tr}}_{max}(t) && \text{(Transformer limit)}
\end{aligned}
$$

  Solve MILP using CBC solver
  if optimal solution found **then**
    Compute first-step power: $P_i(0) = P^{ch}_{i,0} - P^{dis}_{i,0}$
    Normalize with station power limit and output **a**
  else
    return a $=$ **0** with infeasibility warning
  end if
end for

Simplification of Equations (10), (11), and (17) The multiplication by ω is removed from the power and current definitions. The powers P^{ch} and P^{dis} are defined in a purely linear way, as a function of only the current I.

The Key Role of Constraints (15) and (16) The "on/off" logic is entirely delegated to the current limit constraints, which are modified to include the ω variable. Our code implements:

$$I_{i,t}^{\mathrm{ch}} \leq \bar{I}_i^{\mathrm{ch}} \cdot \omega_{i,t}^{\mathrm{ch}} \tag{3.14}$$

$$I_{i,t}^{\mathrm{dis}} \leq \bar{I}_i^{\mathrm{dis}} \cdot \omega_{i,t}^{\mathrm{dis}} \tag{3.15}$$

This approach ensures that if the optimizer sets $\omega^{\mathrm{ch}} = 0$, the constraint forces the current I^{ch} to zero, effectively turning off the power. If $\omega^{\mathrm{ch}} = 1$, the current is free to flow up to its maximum. The same logic applies to the discharge constraint.

Omissions and Adherence to the Theoretical Formulation

In addition to linearization, the implementation in **pulp_mpc.py** makes specific choices regarding the physical accuracy of the reference formulation.

- **Omission of Non-Linearity:** As described, the main "omission" is the non-linear formulation itself, which is replaced by its efficient, linear counterpart.
- **Correction of Battery Dynamics (Eq. 13):** The original formulation sometimes states $E_t = E_{t-1} + (P^{\mathrm{ch}} + P^{\mathrm{dis}})\Delta t$, which is physically incorrect. Our implementation corrects this dynamic to $E_t = E_{t-1} + (P^{\mathrm{ch}}\eta^{\mathrm{ch}} - P^{\mathrm{dis}}/\eta^{\mathrm{dis}})\Delta t$, correctly modeling the accumulation and consumption of energy.
- **Correction of Transformer Load (Eq. 19):** Similarly, the total EV load on the transformer is calculated as $P^{\mathrm{ch}} - P^{\mathrm{dis}}$, correctly subtracting the power injected into the grid during discharge.

In conclusion, the **pulp_mpc.py** code is not a simple translation of the theoretical formula, but an engineered implementation that (a) transforms an intractable MINLP into an efficient MILP, and (b) corrects certain physical inaccuracies to achieve a more realistic and reliable model behavior.

3.14 Lyapunov-based Adaptive Horizon MPC

A key enhancement is the **Lyapunov-based Adaptive Horizon MPC**, which aims to reduce the computational burden of the online MPC while retaining its optimality and stability guarantees. The controller dynamically adjusts its prediction horizon H_t based on the stability of the system, which is formally assessed using a Lyapunov function. A Lyapunov function $V(x)$ is a scalar function that measures the system's deviation from a desired equilibrium state. For the V2G system, we define it as the sum of squared errors from the desired final energy state:

$$V(E_t) = \sum_{i \in \text{EVs}} (E_{i,t} - E_i^{\text{des}})^2 \tag{3.16}$$

The adaptive control logic, which wraps the MILP solver, is detailed in Algorithm 6. This intelligent adjustment makes the online MPC more efficient, reducing computation time during stable periods while retaining the ability to perform deep planning when necessary to guarantee system stability and constraint satisfaction.

Algorithm 6 Lyapunov-based Adaptive Horizon MPC

Initialization:
Initialize current horizon $H_{\text{current}} \leftarrow H_{\max}$.
Define horizon bounds $H_{\min}, H_{\max}$ and convergence rate α.
Initialize action plan $\mathcal{A} \leftarrow \emptyset$.
function GetAdaptiveAction(current state s_t)
    if action plan $\mathcal{A}$ is not empty **then**
        $a_t \leftarrow$ Pop first action from $\mathcal{A}$.
        return a_t.
    end if
    ▷ Plan is empty, re-optimization is needed
    $E_t \leftarrow$ Get current energy states from s_t.
    $V(E_t) \leftarrow \sum_i (E_{i,t} - E_i^{\text{des}})^2$.
    ▷ Solve MPC with the current horizon
    solution $\leftarrow$ SolveMPC(s_t, H_{current}) ▷ Using MILP solver
    if solution is Optimal **then**
        Extract optimal first action $a_t^* = (P_t^{\text{ch},*}, P_t^{\text{dis},*})$.
        Predict next energy state E_{t+1} using a_t^*.
        $V(E_{t+1}) \leftarrow \sum_i (E_{i,t+1} - E_i^{\text{des}})^2$.
        ▷ Verify Lyapunov stability condition
        if $V(E_{t+1}) \leq V(E_t) - \alpha V(E_t)$ **then**
            ▷ Stable: reduce computational load for next cycle
            $H_{\text{next}} \leftarrow \max(H_{\min}, H_{\text{current}} - 1)$.
        else
            ▷ Not stable enough: increase planning depth
            $H_{\text{next}} \leftarrow \min(H_{\max}, H_{\text{current}} + 1)$.
        end if
        Extract new action plan $\mathcal{A}$ from solution.
    else
        ▷ Solver failed: increase horizon as a safeguard
        $H_{\text{next}} \leftarrow \min(H_{\max}, H_{\text{current}} + 1)$.
        $\mathcal{A} \leftarrow$ default safe action.
    end if
    $H_{\text{current}} \leftarrow H_{\text{next}}$.
    $a_t \leftarrow$ Pop first action from $\mathcal{A}$.
    return a_t.
end function

3.15 Benchmark Results and Algorithm Comparison

This section presents a detailed analysis of the benchmark results obtained from a comprehensive simulation run. The raw data and plots are sourced from the benchmark directory. The experiments compare the performance of heuristic, Model Predictive Control (MPC), and DRL algorithms across a variety of simulated scenarios designed to test their economic efficiency, grid stability, and user satisfaction.

3.15.1 Scenario 1: Critical Case (High Demand High Price Volatility)

This scenario represents a critical, high-stress condition designed to test the limits and robustness of the control algorithms. It is characterized by high electricity price volatility, creating significant opportunities for V2G arbitrage but also increasing the risk of economic losses. Furthermore, it simulates a high demand for charging, with a large number of EVS present simultaneously, placing a heavy load on the grid infrastructure and increasing the likelihood of transformer overloads. The following plots illustrate the scenario conditions and the comparative performance of the tested algorithms.

Scenario Conditions

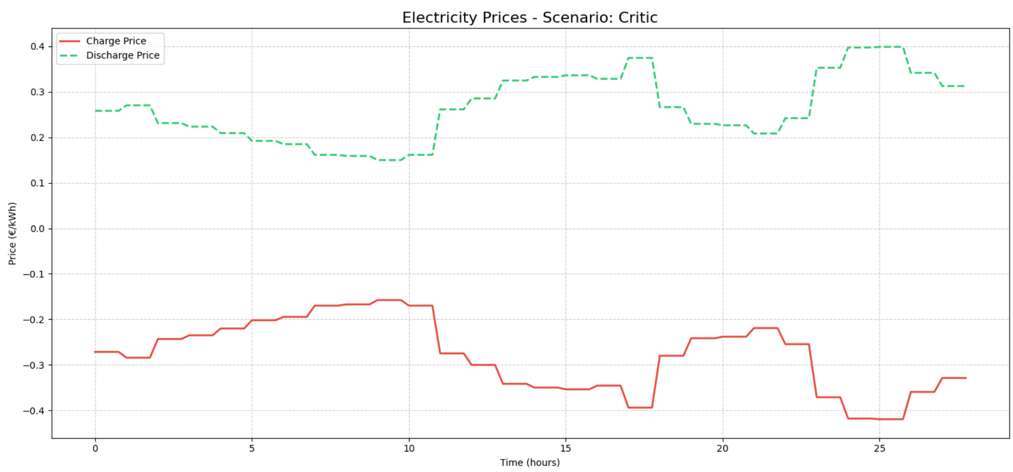


Figure 3.5: Electricity price profile for the Critical scenario. The high volatility, with significant peaks and troughs, creates a challenging environment that rewards effective charge/discharge scheduling.

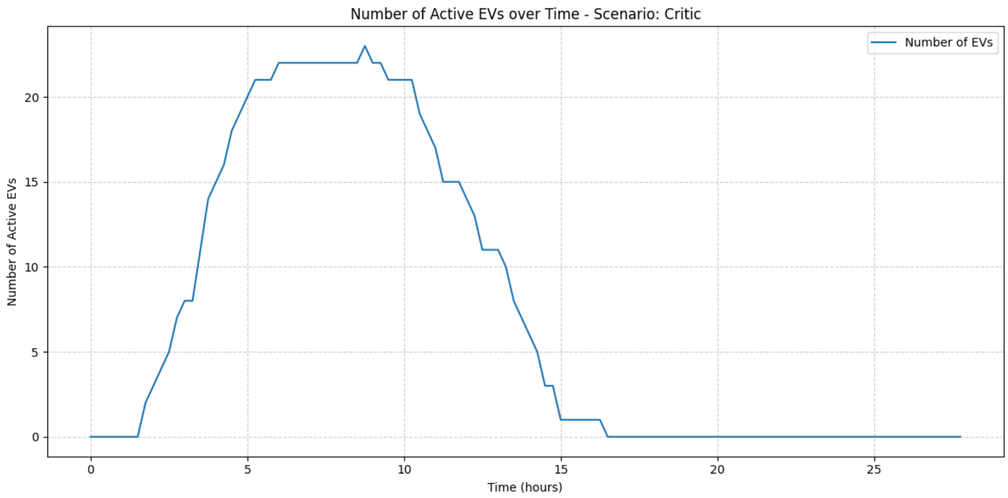


Figure 3.6: Number of EVs present at the charging station over time. The scenario simulates a high-density environment, leading to significant aggregate demand for power.

Overall Performance Summary

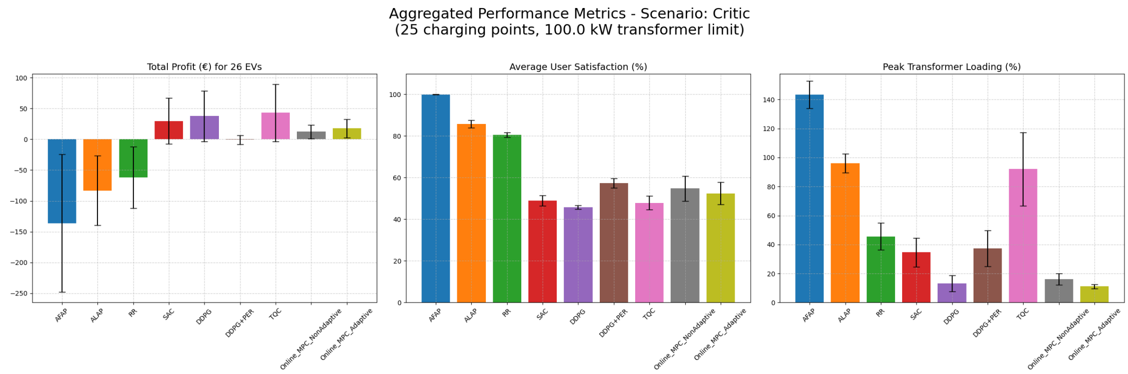


Figure 3.7: Overall performance summary comparing all algorithms across key metrics. In this high-stress scenario, the advanced RL agents (SAC, TQC) and the adaptive MPC demonstrate superior profitability, effectively leveraging V2G, but DDPG goes better in the overall metrics better managing the trasformer overload. Heuristic methods like AFAP and ALAP result in significant economic losses and grid overloads.

Aggregated Results - Scenario: Critic (N=3 simulations)

	Profits/Costs (€)	User Sat. (%)	Energy Ch. (kWh)	Energy Disch. (kWh)	Tr. Ov. (kWh)	Calendar Deg. (x10⁻³)	Cyclic Deg. (x10⁻³)	Exec. Time (s)	Reward (x10³)
AFAP	-136.35 ± 111.78	99.90 ± 0.14	637.07 ± 66.14	0.00 ± 0.00	312.67 ± 46.48	0.06 ± 0.01	0.26 ± 0.03	0.19 ± 0.19	-11.33 ± 4.64
ALAP	-83.07 ± 56.55	85.79 ± 1.74	357.83 ± 39.98	0.00 ± 0.00	210.25 ± 50.72	0.03 ± 0.00	0.10 ± 0.01	0.16 ± 0.17	-2.96 ± 0.96
RR	-61.82 ± 49.78	80.57 ± 1.21	299.12 ± 35.51	0.00 ± 0.00	57.89 ± 40.95	0.04 ± 0.00	0.09 ± 0.01	0.15 ± 0.16	-0.10 ± 0.09
SAC	29.74 ± 37.12	49.03 ± 2.51	237.78 ± 21.12	339.92 ± 44.12	20.80 ± 9.42	0.02 ± 0.00	0.19 ± 0.01	0.23 ± 0.16	0.02 ± 0.04
DDPG	37.62 ± 40.93	45.76 ± 0.89	203.13 ± 22.39	326.98 ± 74.24	2.40 ± 3.10	0.02 ± 0.00	0.19 ± 0.00	0.24 ± 0.21	0.02 ± 0.04
DDPG+PER	-0.73 ± 7.45	57.34 ± 2.25	292.19 ± 37.89	248.82 ± 61.02	18.11 ± 18.24	0.03 ± 0.00	0.20 ± 0.01	0.21 ± 0.17	-0.01 ± 0.01
TQC	43.06 ± 46.42	47.89 ± 3.33	417.90 ± 96.37	538.83 ± 142.04	58.28 ± 14.81	0.03 ± 0.00	0.31 ± 0.04	0.26 ± 0.23	-0.17 ± 0.22
Online_MPC_NonAdaptive	12.21 ± 11.21	54.83 ± 6.02	54.78 ± 17.70	106.68 ± 7.22	3.95 ± 2.76	0.03 ± 0.01	0.05 ± 0.00	10.03 ± 0.39	0.00 ± 0.01
Online_MPC_Adaptive	17.68 ± 14.77	52.42 ± 5.37	35.97 ± 8.07	119.29 ± 15.37	2.83 ± 2.97	0.03 ± 0.01	0.05 ± 0.00	11.42 ± 0.37	0.00 ± 0.02

Figure 3.8: Tabular summary of performance metrics. This table provides the precise numerical data for Total Profit, User Satisfaction, Grid Overload, and Battery Degradation, confirming the visual insights from the bar chart.

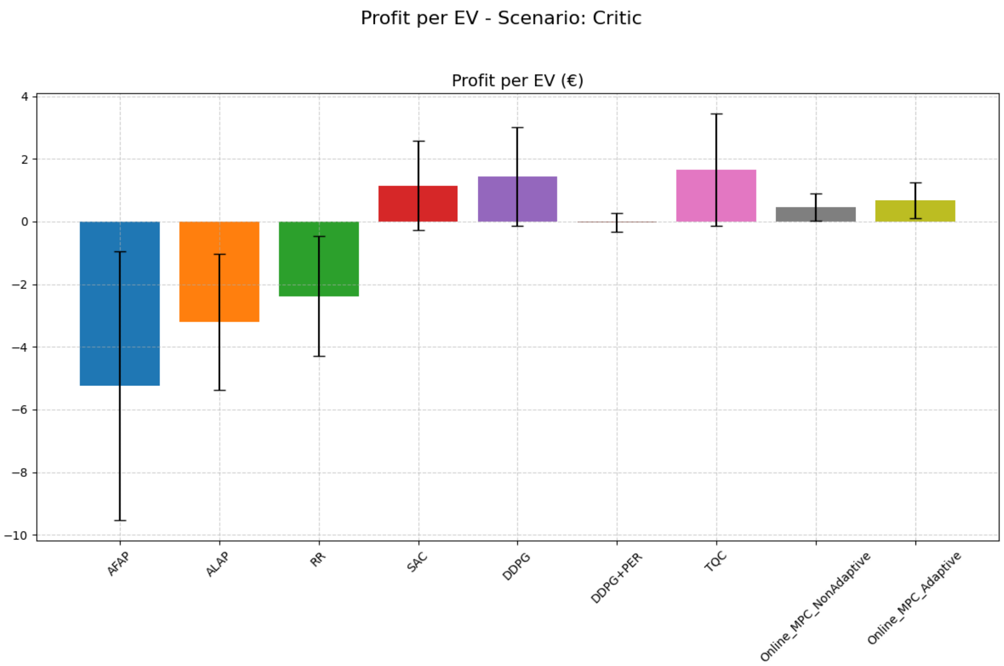


Figure 3.9: Average profit generated per EV. This metric normalizes the total profit by the number of vehicles served, highlighting the economic efficiency of each control strategy on a per-user basis.

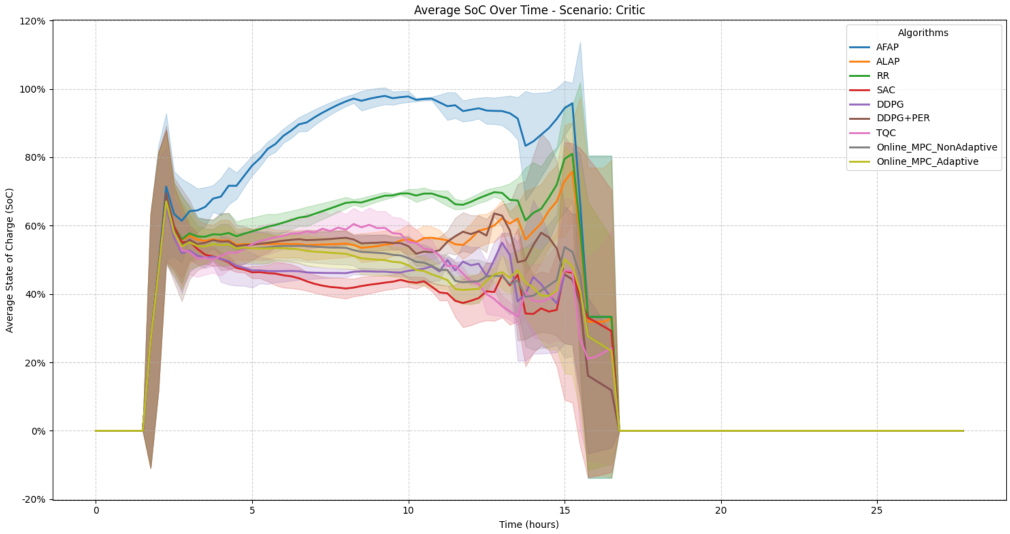
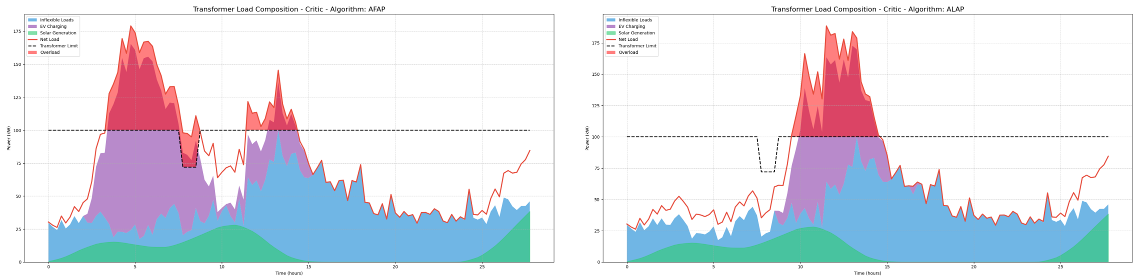


Figure 3.10: Average State of Charge (SoC) of all connected EVs over time. RL and MPC agents maintain a lower average SoC during low-price periods to prepare for charging, while heuristics like AFAP charge aggressively upon arrival.

Grid Impact Analysis

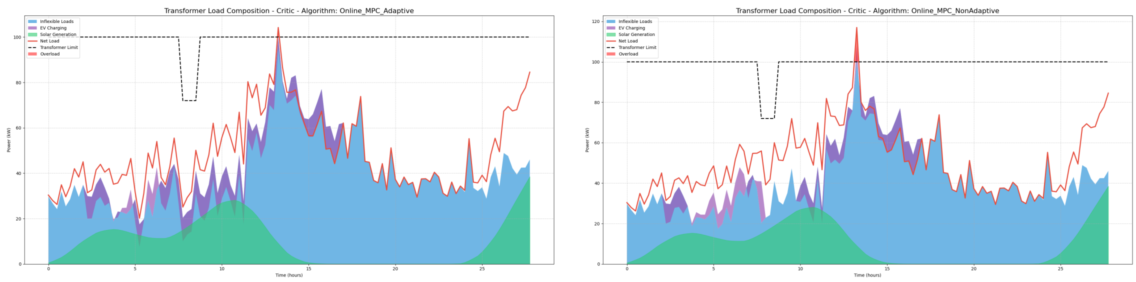
This subsection analyzes the impact of each algorithm on the grid, focusing on transformer load and overload events.



(a) Overload composition for AFAP algorithm.

(b) Overload composition for ALAP algorithm.

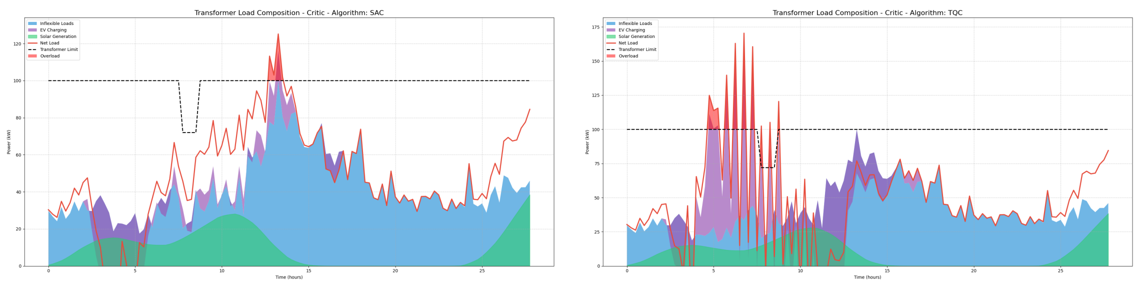
Figure 3.11: Grid load for heuristic algorithms. Both AFAP and ALAP show significant and prolonged periods of transformer overload, as they do not coordinate charging to respect grid limits.



(a) Overload composition for MPC Non-Adaptive algorithm.

(b) Overload composition for MPC Adaptive algorithm.

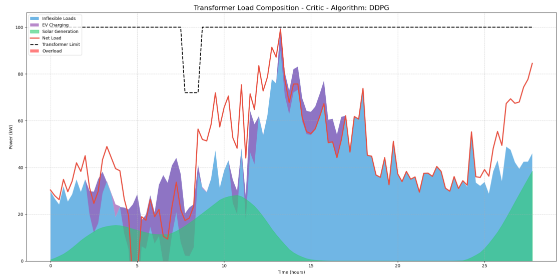
Figure 3.12: Grid load for MPC algorithms. The MPC controllers are enough effective at preventing overloads, keeping the total load just below the transformer's maximum capacity.



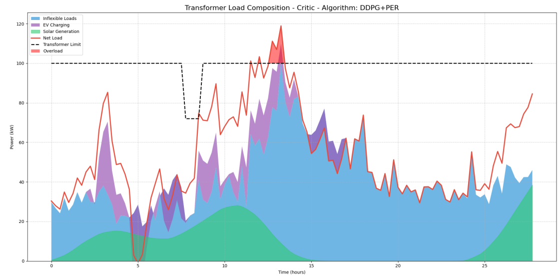
(a) Overload composition for SAC algorithm.

(b) Overload composition for TQC algorithm.

Figure 3.13: Grid load for SAC and TQC RL algorithms. The SAC agents learn to respect the grid constraints better than TQC.



(a) Overload composition for DDPG algorithm.

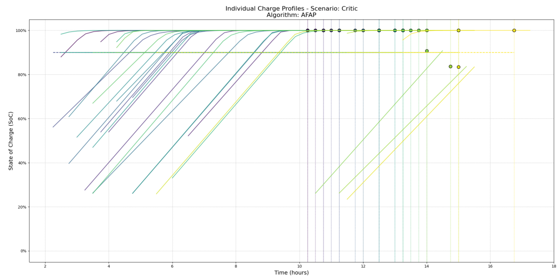


(b) Overload composition for DDPG+PER algorithm.

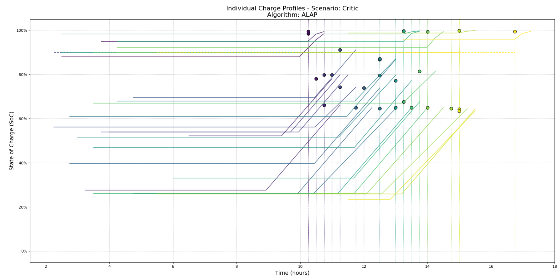
Figure 3.14: Grid load for DDPG-based RL algorithms. These agents also learn to avoid overloads, though some minor spikes can occasionally be seen during the learning process.

Individual EV Charging Profiles

The following plots show the detailed charging and discharging behavior for a sample of individual EVs, providing insight into the different strategies employed by each algorithm.

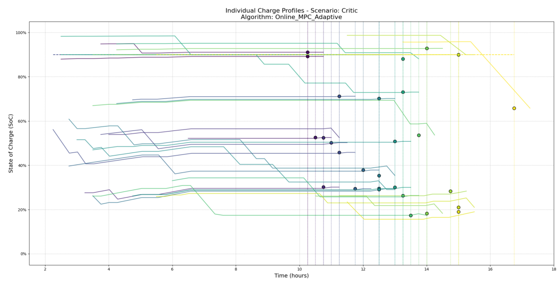


(a) Individual ev profiles for AFAP algorithm.

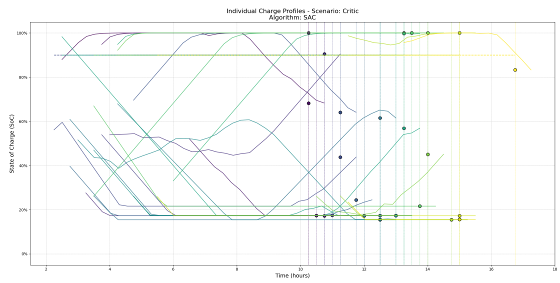


(b) Individual ev profiles for ALAP algorithm.

Figure 3.15: Individual EV profiles for heuristic algorithms. AFAP charges immediately upon arrival, while ALAP waits until the last possible moment. Neither strategy responds to price signals.



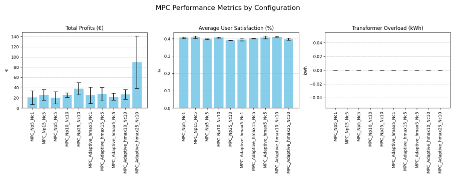
(a) Individual ev profiles for Adaptive MPC: it slightly charges at times, but occasionally exhibits a negative ramp due to discharging.



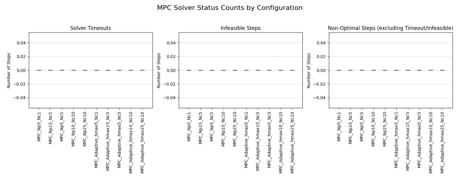
(b) SAC

Figure 3.16: The SAC tends to fully charge some EVs while leaving others partially charged, focusing instead on using certain vehicles to maximize profit.

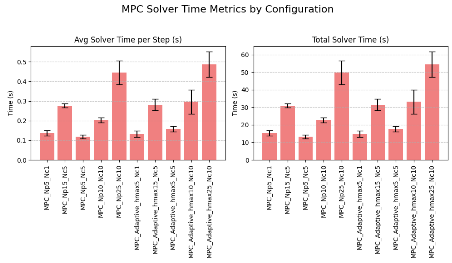
3.15.2 MPC sensitivity for the Np/Nc ratio



(a) Performance of the algorithms.



(b) Solver status.



(c) Solver time.

Figure 3.17: MPC sensivity

With this sensitivity analysis performed in the Figures 3.17, we can conclude that increasing the N_p/N_c ratio may improve profit but does not enhance average satisfaction. The overload remains zero, which is naturally a result of the high utilization of V2G and, consequently, the higher profit.
For the adpative MPC we can understand how the algorithm change adaptively through time from Figure 3.18.

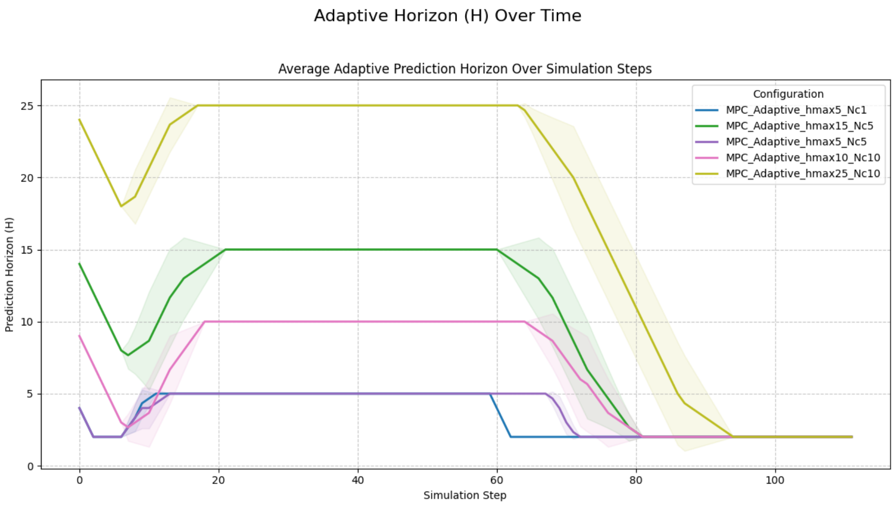


Figure 3.18: Adaptive horizon.

Chapter 4

Sensitivity Analysis of the V2G System

The performance of a Vehicle-to-Grid (V2G) system is governed by a complex interplay of physical, economic, and behavioral parameters. Understanding which of these parameters have the most significant impact on system outcomes is crucial for designing robust and effective control strategies. A controller optimized for one specific set of conditions may perform poorly under slightly different circumstances. Sensitivity Analysis (SA) is a systematic methodology used to quantify how the variation in the inputs of a model influences the variation in its outputs. This chapter presents a comprehensive sensitivity analysis of the EV2Gym simulation environment. The primary objectives are:

- To identify the most critical parameters that drive system performance, particularly economic profitability and grid stability.
- To understand how different control algorithms (heuristics, MPC, and RL) respond to variations in these parameters.
- To gain insights that can guide the development of more robust and generalist control agents.

The analysis was conducted using the custom-built **sensitivity_analysis.py** script, which automates the process of running large batches of simulations while systematically varying key input parameters.

4.1 Introduction to Global Sensitivity Analysis

In these sessions, the topic of global sensitivity analysis is discussed, which is presented as a very important method in uncertainty quantification.This method allows for a better understanding of the relationship between data, model, and prediction. There are many techniques for performing such global sensitivity analysis. However, before covering global sensitivity analysis, the speaker first discusses local sensitivity analysis.
Sensitivity analysis is defined as a study where uncertain input parameters are

varied to understand how that variation results in a change in some response of interest. This response of interest can be anything, such as a data variable or a prediction variable.
SA is the study of how variation of uncertain input parameters impacts the uncertainty of the response of interest. *Note: There is no unique way of doing sensitivity analysis.*
In this context, sensitivity analysis allows for the reduction of uncertainty in predictions, as it facilitates the identification of input parameters that affect those predictions. This, in turn, can help identify what data needs to be acquired to reduce uncertainty on those parameters.

- **SA can aid in reducing uncertainty in prediction** by identifying high-impact parameters. This can indicate which data to acquire to reduce uncertainty on said parameters.

- **SA can indicate which parameters have little influence on the prediction.** This may help in reducing the complexity of the problem by either removing those parameters or setting those with little influence to some fixed value.

- **SA can help in quantifying and understanding how the combined effect of parameters generates non-linear variations in responses (interactions).** When dealing with nonlinear systems or functions, as is common in uncertainty quantification problems and the forward models used, these nonlinear variations in responses are frequent. In such systems, a phenomenon known as "interactions" occurs. This concept will be covered in great detail.

As a side note, it is mentioned that there is no single, unique way of performing sensitivity analysis. Therefore, referring to "the" sensitivity analysis would actually point to a specific method. Each method allows for the study of different aspects, and it is important to understand which method is suitable for a given situation and application.

4.2 Different Classes of Sensitivity Analysis

There are many different classes of sensitivity analysis, but for the purposes of this course, the discussion focuses on local sensitivity analysis and global sensitivity analysis.

4.2.1 Local SA

In local sensitivity analysis, the process starts from a base case. This could be the mean of the parameters, or a single model can be built based on the available information. Once such a model or base case is established, the effect of making perturbations to that model is evaluated. Perturbations can be implemented in various ways; for instance, one can calculate the partial derivatives of the forward response function, given that the analysis is in the base case scenario. It is also possible to have multiple base case scenarios and evaluate sensitivity on each of them.

- Evaluates sensitivity for a single (deterministic) set of input parameters.
- Often based on partial derivatives of the response with respect to the parameters. The sensitivity measure is only informative for the particular set of parameters.
- Fast to compute ($N_p + 1$ model evaluations).

Algorithm 7 Local Sensitivity Analysis (LSA)

Input: model $f(\mathbf{x})$ with parameters $\mathbf{x} = [x_1, x_2, \dots, x_{N_p}]$, base case $\mathbf{x}_0$
Output: local sensitivity S_i for each parameter x_i
Build the base model: $y_0 = f(\mathbf{x}_0)$
for $i = 1$ to N_p **do**
    Perturb parameter i by a small Δx_i: $\mathbf{x}_i^+ = \mathbf{x}_0 + \Delta x_i \mathbf{e}_i$
    Calculate the answer: $y_i^+ = f(\mathbf{x}_i^+)$
    Calculate the local sensitivity:

$$S_i = \frac{y_i^+ - y_0}{\Delta x_i} \approx \left. \frac{\partial f}{\partial x_i} \right|_{\mathbf{x}_0}$$

end for
Return: $S_1, S_2, \dots, S_{N_p}$

4.2.2 Global SA (GSA)

In global sensitivity analysis, the base case scenario is disregarded. Instead, a Monte Carlo simulation is performed on the input parameters. The model is run many times with various input parameters drawn from a prior distribution. The models are then evaluated to understand the multivariate distribution of the output that is generated due to the variation of the input parameters.

- Assesses the effect of large changes in the input parameters, which is more appropriate for uncertainty quantification.
- Requires many model evaluations.

4.3 Challenges

The challenges (very common in geosciences) or uncertainty quantification are:

- **High dimensional problem:**
  - Many uncertain input parameters.

- Response may be high dimensional.

- **Large computational time:** Evaluating responses such as forward models and making predictions involves large computational times.
- **Non-linearity of the models:** The non-linearity of the problem is quite substantial.
- **Spatial aspect of the models:** There is a spatial aspect, such as the spatial distribution of hydraulic conductivity, permeability, porosity, or layering, which needs to be taken into account as it may have a considerable influence on the response. The problem is how to quantify the influence of spatial variability (which is not a simple variable) on some kind of response.

Screening Techniques

Two techniques are introduced: one-at-a-time (OAT) analysis and the Morris method. Both are simplified approaches. They are quite useful for "screening" because they require only a small number of model evaluations. However, it is very difficult to draw general conclusions from these methods, as there are several drawbacks when applying them to the kind of problems being addressed.

Algorithmic Description of OAT and Morris Methods

Algorithm 8 One-at-a-Time (OAT) Sensitivity Analysis

Input: model $f(\mathbf{x})$ with parameters $\mathbf{x} = [x_1, \dots, x_{N_p}]$, base case $\mathbf{x}_0$, perturbation factor Δ
Output: parameter influence ranking
Evaluate the base case: $y_0 = f(\mathbf{x}_0)$
for $i = 1$ to N_p **do**
    Perturb the i-th parameter: $\mathbf{x}_i^+ = \mathbf{x}_0 + \Delta x_i \mathbf{e}_i$
    Evaluate the model: $y_i^+ = f(\mathbf{x}_i^+)$
    Compute the effect of parameter i: $S_i = y_i^+ - y_0$
end for
Rank parameters based on $|S_i|$ to determine their influence
Note: Requires $2 \times N_p + 1$ model evaluations for full analysis.

Algorithm 9 Morris Method (Elementary Effects Method)

Input: model $f(\mathbf{x})$ with parameters $\mathbf{x} = [x_1, \ldots, x_{N_p}]$, number of trajectories L, perturbation Δ
Output: μ_n^*, σ_n for each parameter
for $l = 1$ to L **do**
    Generate a random starting point $\mathbf{x}^{(l)}$ in the parameter space
    for $n = 1$ to N_p **do**
        Perturb parameter n: $\mathbf{x}_n^{(l)} = \mathbf{x}^{(l)} + \Delta \mathbf{e}_n$
        Compute the elementary effect:

$$EE_n^{(l)} = \frac{f(\mathbf{x}_n^{(l)}) - f(\mathbf{x}^{(l)})}{\Delta}$$

    end for
end for
for $n = 1$ to N_p **do**
    Compute mean of absolute elementary effects:

$$\mu_n^* = \frac{1}{L} \sum_{l=1}^{L} |EE_n^{(l)}|$$

    Compute standard deviation of elementary effects:

$$\sigma_n = \sqrt{\frac{1}{L-1} \sum_{l=1}^{L} (EE_n^{(l)} - \overline{EE}_n)^2}$$

end for
Interpretation: Plot σ_n vs μ_n^* to classify parameters:

- Negligible effects: small μ_n^* and small σ_n
- Linear and additive effects: large μ_n^* and small σ_n
- Non-linear or interaction effects: large μ_n^* and large σ_n

Note on Interactions

Interactions are an often poorly understood concept. An example is given where a material (e.g., shale rock) is tested, and its strength (response Z) is evaluated based on two factors: temperature (X) and composition ratio (Y). A factorial design can be set up with high and low levels for each factor. An interaction means that the effect of one factor depends on the level of another factor. The main effects of X and Y, as well as the interaction effect XY, can be estimated. In many cases, the interaction effect (the combined effect of temperature and composition) can be the most significant factor, more so than either factor individually. This indicates a non-linear relationship where the whole is greater than the sum of its parts.

Digest of Screening Techniques

- Rapidly gives insight into parameter influence with a small number of model evaluations.
- Only provides qualitative/subjective measures of sensitivity.
- Does not evaluate or distinguish interactions from non-linear effects well.
- Requires only a single response.
- Cannot be applied to non-numerical input (e.g., scenarios).
- Requires continuity in the response, which means it cannot be applied with spatial uncertainty.
- As a precursor to GSA, the Morris method may identify parameters that can be removed from the study, simplifying the problem.

4.4 Methodology for Sensitivity and Robustness Analysis

To systematically investigate the V2G system's behavior and train robust control agents, this work employs a suite of increasingly sophisticated methodologies, all orchestrated through the **sensitivity_analysis.py** and **run_interactive.py** scripts. These methods range from classical sensitivity analysis techniques to advanced, dynamic training environments.

4.4.1 Parameter and Model-Output Space

The foundation for all analysis is a defined set of input parameters and output metrics. The input space consists of nine key parameters identified as having a potentially high impact on the system's operation. For each parameter, a set of discrete, realistic levels was established, as detailed in Table 4.1. The primary output metric, or Key Performance Indicator (KPI), used for evaluating sensitivity is the **Total Profit (€)** generated by a control agent over a full simulation run.

Table 4.1: Parameters and Levels for Sensitivity Analysis

Parameter	Baseline	Levels
Number of Charging Stations	25	5, 15, 25, 50, 100
Transformer Max Power (kW)	100	25, 50, 100, 200, 400
EV Spawn Multiplier	5	1, 3, 5, 7, 10
EV Charge Efficiency	0.95	0.80, 0.90, 0.95, 0.99
EV Discharge Efficiency	0.90	0.70, 0.85, 0.90, 0.98
EV Desired Capacity (%)	1.0	0.6, 0.75, 0.85, 0.95, 1.0
Discharge Price Factor	1.0	0.5, 0.8, 1.0, 1.5, 2.0
Inflexible Loads Forecast Mean	30%	0, 15, 30, 45, 60
Solar Power Forecast Mean	20%	0, 20, 40, 60, 80

4.4.2 One-at-a-Time (OaaT) Sensitivity Analysis

The OaaT method is the most fundamental approach to sensitivity analysis. It operates by varying a single input parameter across its predefined levels while keeping all other parameters fixed at their baseline values. For example, to analyze the impact of the transformer capacity, simulations are run with the 'Transformer Max Power (kW)' set to 25, 50, 100, 200, and 400, while the other eight parameters remain unchanged. The primary advantage of OaaT is its simplicity and the clarity of its results. It produces easy-to-interpret graphs showing the direct relationship between one input and one output. However, its major drawback is that it cannot detect interactions between parameters. The effect of changing the transformer power might be very different when the 'EV Spawn Multiplier' is high compared to when it is low, and OaaT cannot capture this type of synergistic effect.

4.4.3 Global Sensitivity Analysis: The Morris Method

To overcome the limitations of OaaT, we employ the Morris method, a global sensitivity analysis technique proposed by Max D. Morris in 1991 for efficient parameter screening. It is a "one-at-a-time" method, but it is executed at many different points in the input parameter space, and the results are then averaged.
The sampling process is organized into **trajectories**, where each trajectory represents a path through the high-dimensional input space. A key implementation choice is the use of **Optimal Trajectories**, a sophisticated sampling strategy designed to overcome the potential biases of purely random sampling.
The concept is analogous to surveying a large area: instead of taking random paths, which might accidentally cluster in one region and miss others entirely, an optimal approach involves pre-planning the paths to ensure the entire area is covered as uniformly as possible. In the context of the Morris method, 'SALib' selects a set of trajectories that are maximally "spread out" from each other. This ensures that the elementary effects are calculated in highly diverse regions of the parameter space (e.g., under conditions of both high and low solar power, high and low electricity prices, etc.). By guaranteeing a more systematic and uniform exploration, this optimal sampling strategy produces more robust and reliable

sensitivity indices (μ^* and σ) for a given number of simulations, reducing the risk of drawing conclusions from an unrepresentative sample of the model's behavior. The core of the method is the calculation of "elementary effects." For a given input parameter X_i, an elementary effect is the finite difference in the model output Y when X_i is perturbed by a fixed amount Δ, while all other inputs X_j ($j \neq i$) are held constant.

$$EE_i(\mathbf{X}) = \frac{Y(X_1, ..., X_i + \Delta, ..., X_k) - Y(\mathbf{X})}{\Delta} \tag{4.1}$$

The Morris method generates r different trajectories through the k-dimensional input space. For each trajectory, it computes the elementary effect for every parameter. The final output is a set of sensitivity indices for each parameter, derived from the distribution of these elementary effects:

- μ^* (**mu-star**): The mean of the absolute values of the elementary effects. It is a measure of the parameter's overall influence on the output. A high μ^* indicates an important parameter.
- σ (**sigma**): The standard deviation of the elementary effects. It measures the extent of non-linear effects or interactions with other parameters. A high σ suggests that the parameter's effect is not constant but depends on the values of other parameters in the model.

By plotting σ versus μ^*, we can visually classify parameters as either (a) having a large overall effect (high μ^*), (b) being involved in significant interactions (high σ), or (c) being relatively non-influential (low μ^* and σ). This provides a global perspective on the model's sensitivities.

Implementation with SALib

To conduct the global sensitivity analysis using the Morris method, the Python library **SALib** was utilized. The process implemented in the **sensitivity_analysis.py** script can be broken down into the following key steps:

1. **Problem Definition:** First, the sensitivity problem is defined. This is done by creating a Python dictionary that specifies the parameters to be analyzed, their names, and their bounds.
   This data structure, named **problem**, is a fundamental requirement for the **SALib** library.

```
problem = {
    'num_vars': len(selected_param_names),
    'names': selected_param_names,
    'bounds': [[min_val_1, max_val_1],
               [min_val_2, max_val_2],
               ...]
}
```

2. **Sampling:** Next, the Morris sampler provided by **SALib** is used to generate the input points to be evaluated.
   The function **morris_sampler.sample()** takes the **problem** dictionary, the number of trajectories to generate (**N**), and the number of grid levels (**num_levels**) as input.
   The function returns a NumPy array, **param_values**, where each row represents a combination of input parameters on which to run the model.

```
from SALib.sample import morris as morris_sampler

param_values = morris_sampler.sample(problem,
                                     N=num_trajectories,
                                     num_levels=num_levels)
```

3. **Model Execution:** The code then iterates over each row of the **param_values** array.
   For each combination of parameters, a model simulation is executed (**run_benchmark**), and the output of interest (e.g., **total_profits**) is recorded. These results are stored in a NumPy array **Y**.

4. **Analysis of Results:** Once the model has been run for all sampling points, the results are analyzed using the **morris_analyzer.analyze()** function. This function calculates the Morris sensitivity indices, primarily:

   - μ^* **(mu_star):** An estimate of the mean absolute effect of a parameter on the output. A high value indicates an influential parameter.
   - σ **(sigma):** The standard deviation of the elementary effects, which indicates non-linear effects or interactions with other parameters.

```
from SALib.analyze import morris as morris_analyzer

Si = morris_analyzer.analyze(problem,
                             param_values,
                             Y,
                             conf_level=0.95)
```

   The returned **Si** object contains the results of the analysis, which are then used to generate tables and plots to interpret the influence of each parameter.

Interpreting the $\mu^* - \sigma$ Plot

The $\mu^* - \sigma$ plot is the primary visualization tool for the Morris method, offering a clear way to understand the role of each input parameter. Parameters are represented as points on a 2D plane, where the x-axis is μ^* and the y-axis is σ.

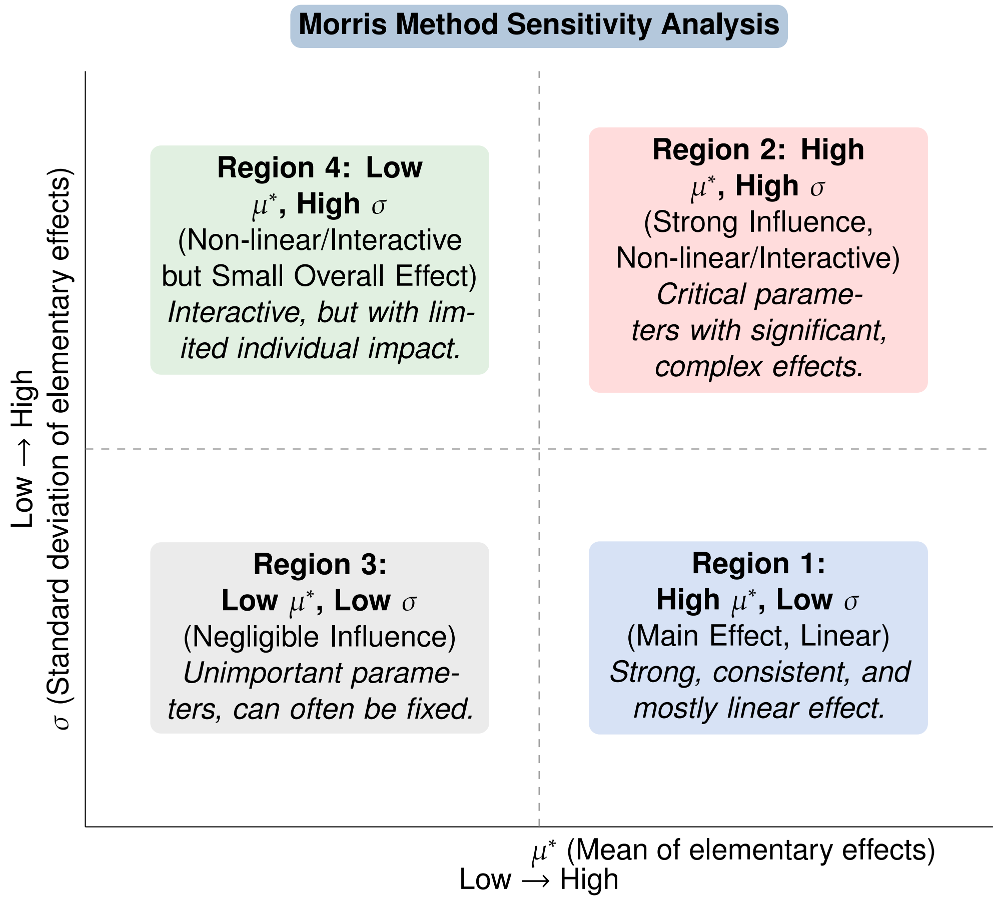


Figure 4.1: Conceptual Morris plot illustrating different parameter behaviors.

Figure 4.1 illustrates how parameters typically cluster in different regions of the plot, each indicating a distinct type of influence:

- **Region 1: High μ^*, Low σ (Main Effect, Linear)**: Parameters in this region have a strong, consistent, and mostly linear effect on the model output. Their influence is largely independent of the values of other parameters. Examples might include fundamental physical constants or primary drivers of the system.

- **Region 2: High μ^*, High σ (Strong Influence, Non-linear/Interactive)**: These are the most critical parameters. They have a significant overall impact, but their effect is highly dependent on the values of other parameters. This indicates strong non-linearities or interactions. **Discharge Price Factor** and **Transformer Max Power** often fall into this category in V2G systems, as their impact is heavily modulated by other conditions.

- **Region 3: Low μ^*, Low σ (Negligible Influence)**: Parameters located here have little to no effect on the model output. They are generally unimportant and can often be fixed without significantly altering the model's behavior. **EV Charge Efficiency** might be an example if its variation range is small compared to other factors.

- **Region 4: Low μ^*, High σ (Non-linear/Interactive but Small Overall Effect)**: These parameters exhibit strong interactions or non-linear effects, but their overall impact on the model output is relatively small. While they interact with other parameters, their individual contribution to the output variance is limited.

Common Patterns in $\mu^* - \sigma$ Plots

Beyond these general regions, specific patterns in the distribution of parameters can provide further insights into the model's structure:

- **Parameters along a Line (Linear Relationship)**: If several parameters align along a straight line originating from the origin, it often suggests a linear relationship between these parameters and the output, with varying degrees of influence. The slope of the line can indicate the strength of this linearity.
- **Parameters forming a Hyperbola (Non-linear, Interaction-driven)**: A hyperbolic distribution, where parameters with higher μ^* also tend to have higher σ, is characteristic of models with significant non-linearities and interaction effects. This is common in complex systems where the effect of one parameter is heavily modulated by others.
- **Cross-shaped Distribution (Mixed Effects)**: A cross-shaped pattern might emerge if some parameters primarily exhibit main effects (low σ) while others are dominated by interaction effects (high σ), with varying degrees of overall influence. This indicates a diverse set of parameter behaviors within the model.
- **Clustered Parameters (Similar Behavior)**: Parameters that cluster closely together in any region of the plot suggest that they behave similarly in terms of their influence and interaction effects on the model output.

The Morris analysis is invaluable for model simplification and for focusing control strategy development. It tells us which parameters must be modeled with high fidelity and which ones are less critical.

4.4.4 Stochastic Sampling for Robustness Training

To train a single, robust "generalist" agent, the agent must be exposed to a wide variety of scenarios during its training phase. The goal is to create a dataset of configuration files that adequately covers the entire parameter space defined in Table 4.1.

Grid Search and its Limitations

The most exhaustive approach is a Grid Search, which involves creating a scenario for every single possible combination of the parameter levels. As calculated previously, for the nine parameters, this would result in:

$$5 \times 5 \times 5 \times 4 \times 4 \times 5 \times 5 \times 5 \times 5 = 1,250,000 \tag{4.2}$$

Generating and storing over a million configuration files is computationally impractical and inefficient. Training on such a dataset would be extremely slow.

4.4.5 Dynamic Parameter Randomization

This is the most advanced and efficient methodology implemented in this work, selected as the final approach for training a robust agent. It completely eliminates the need for creating and storing static scenario files.
This method uses a custom Gymnasium environment, **RobustnessEnv**, which wraps the core **EV2Gym** simulator. At the beginning of every single training episode, when the **reset()** function is called, this wrapper performs the following actions in memory:

1. It randomly selects a value for each of the nine parameters from their respective lists of predefined levels.

2. It dynamically constructs a new configuration dictionary with this unique combination of parameters.

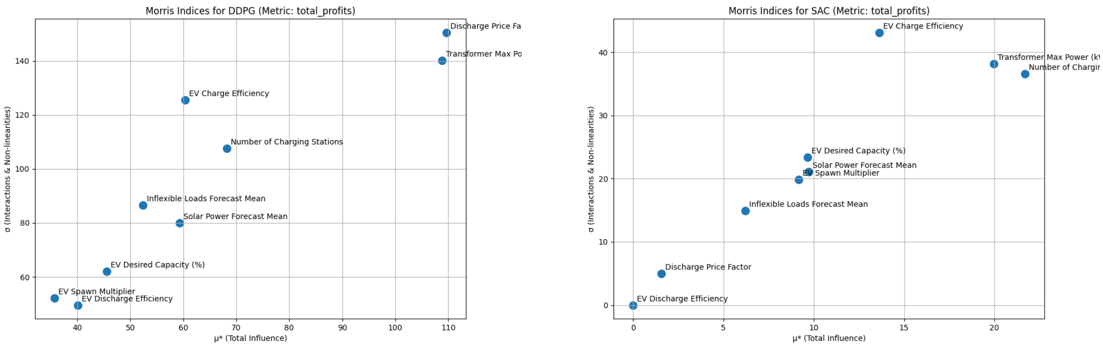
3. It initializes a new instance of the underlying **EV2Gym** environment for that episode using this dynamic configuration.

This "on-the-fly" scenario generation ensures that the agent is exposed to a near-infinite variety of conditions, as every episode is unique. It is the most efficient method in terms of both disk space and computational overhead, as it removes the file I/O bottleneck entirely. The neural network architecture is fixed beforehand to handle the maximum possible dimensions (e.g., 100 charging stations), and padding is used dynamically for any episode that uses a smaller dimension, as explained in the previous chapter. This approach represents a state-of-the-art method for training highly robust and generalizable control agents.

4.5 Analysis of Results

4.5.1 Morris Method Global Analysis Results

A global sensitivity analysis using the Morris method was performed to identify the most influential parameters on the total profit metric for both the DDPG and SAC agents. This analysis is crucial for understanding which factors drive agent performance and for revealing differences in their learned policies.



(a) Morris Analysis for DDPG

(b) Morris Analysis for SAC

Figure 4.2: Morris sensitivity analysis plots for DDPG and SAC. The comparison reveals both shared sensitivities and critical differences in their learned strategies.

Critical Interpretation and Comparison

A detailed examination of the plots in Figure 4.2 reveals key differences in what each agent learned to prioritize.

- **Shared Dominant Factor:** For both DDPG and SAC, the **Transformer Max Power** stands out with a very high mean effect (μ^*) and a high standard deviation (σ). This is an expected and reassuring result, confirming that both agents correctly identified the Trasformer Power as a key element to consider.

- **Divergence in Secondary Factors:** The most interesting insights come from the secondary parameters. For the **DDPG** agent, the **Discarge price factor** is the next most influential parameters. This suggests that DDPG's deterministic policy is highly focused on price spread.

- In stark contrast, the **SAC** agent displays a high sensitivity to **EV Charge Efficiency** , **Number of Charging Stations** and Maximum Transformer Power (this last one is enforced by the OAT analysis see 4.3) all of these have high μ^* and σ. The sensitivity to charge efficiency is particularly noteworthy and was not observed in the DDPG agent. This suggests that SAC's stochastic and more exploratory policy learns to exploit the subtle, non-linear effects of charging efficiency, perhaps by identifying specific conditions where small efficiency gains lead to disproportionately large profit increases.

4.5.2 One-at-a-Time (OaaT) Results

The OaaT analysis further clarifies the direct relationships between key parameters and system outputs.

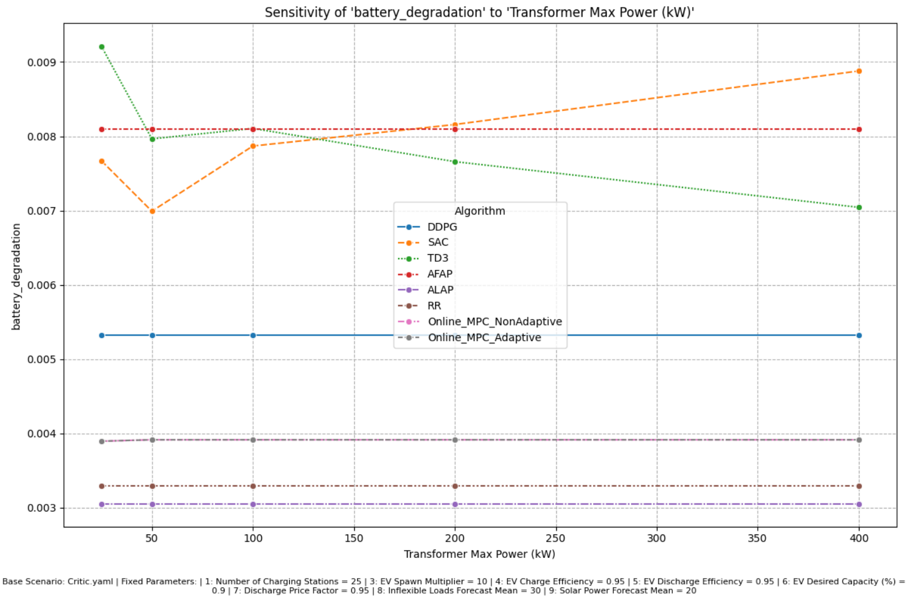


Figure 4.3: Impact of maximum transformer power on battery degradation for the SAC algorithm.

Figure 4.3 provides further confirmation of the significant influence of **Maximum Transformer Power** on battery degradation for the SAC agent, as identified by Morris' analysis. This highlights how adequate transformer sizing is crucial to mitigate battery degradation and optimize the useful life of SAC-managed EVs.

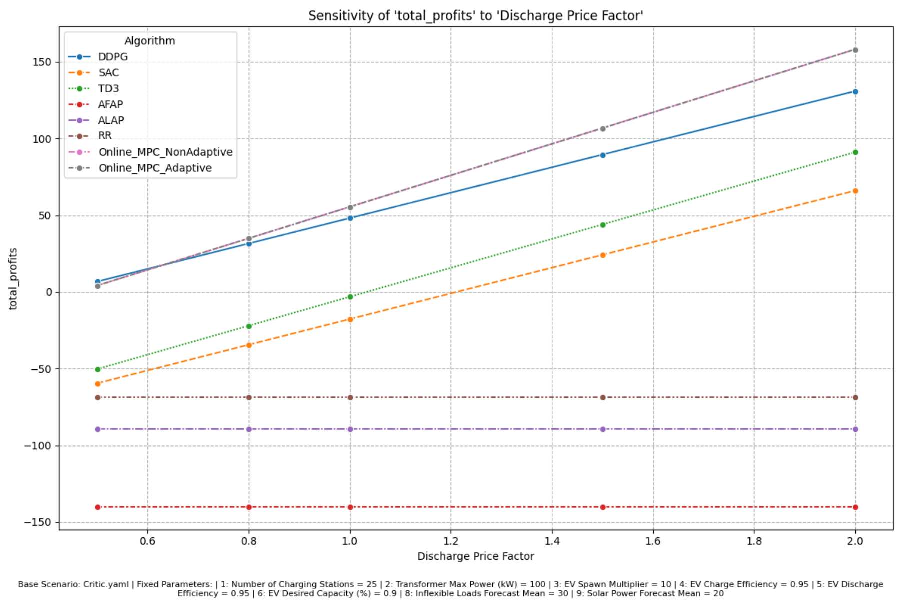


Figure 4.4: Impact of Discharge Price Factor on Total Profits for various algorithms.

As shown in Figure 4.4, for all V2G-capable algorithms (SAC, DDPG, MPC), profit increases substantially as the **Discharge Price Factor** rises above 1.0. This confirms the finding from the Morris analysis that this is the most critical economic lever. Heuristic algorithms that do not discharge (AFAP, ALAP) are, as expected, unaffected.

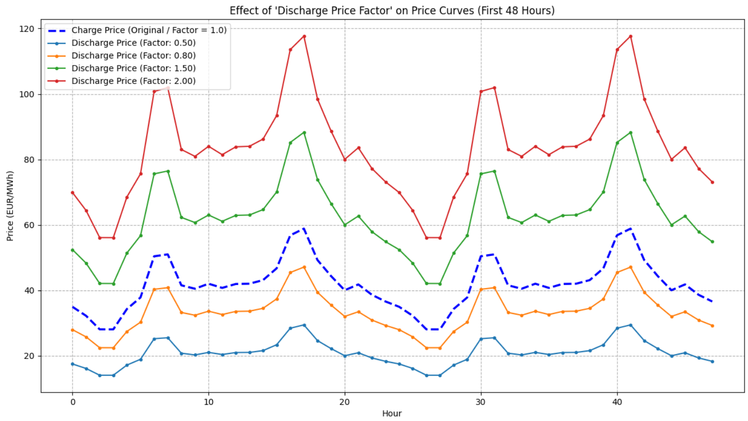


Figure 4.5: Illustration of how the discharge price (selling price) changes relative to the charge price (buying price) based on the discharge price factor.

Figure **??** provides a visual explanation for the results in the previous plot, showing how a factor greater than 1.0 makes the selling price consistently higher than the buying price, creating clear arbitrage opportunities.

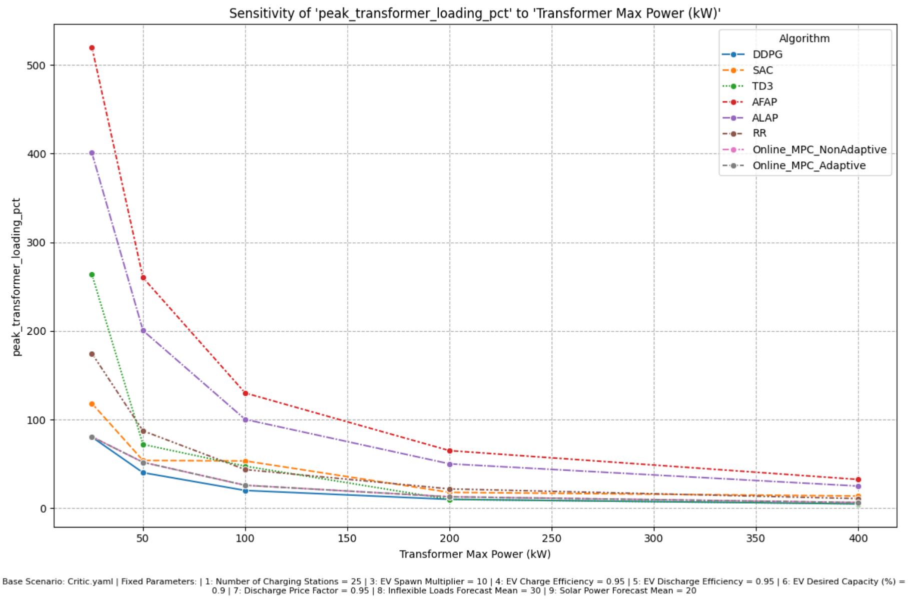


Figure 4.6: Impact of Transformer Max Power on Peak Transformer Loading.

Figure 4.6 illustrates the direct impact of transformer capacity on grid stress. For unintelligent algorithms like AFAP, peak loading remains dangerously high until the transformer capacity is significantly oversized. In contrast, proactive controllers like DDPG, and MPC adapt their behavior to respect the transformer's limit, keeping peak load consistently below 100% across a wide range of capacities. This demonstrates their crucial role in maintaining grid stability.However TD3 shows that without proper optimization cannot go well then DDPG even though it's a DDPG improvement.

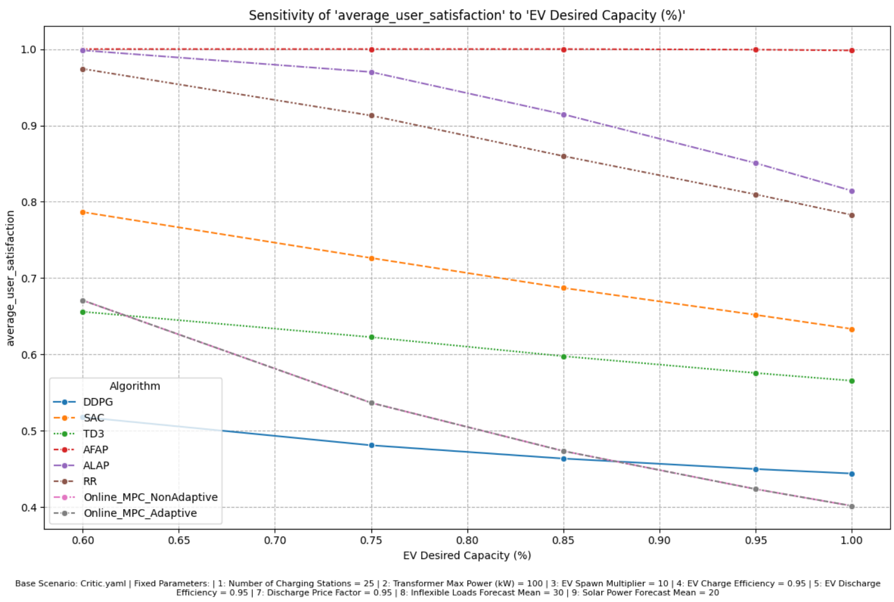


Figure 4.7: Impact of EV Desired Capacity on Average User Satisfaction.

Figure 4.7 shows that as the users' desired state of charge increases, it becomes more challenging for all algorithms to achieve high satisfaction. Notably, heuristic algorithms like AFAP and ALAP maintain near-perfect satisfaction as they have a singular focus on charging, whereas the more intelligent agents must balance this goal against others like profit maximization and grid stability.

4.5.3 Reward Function and Curriculum Learning Analysis

A key contribution of this work was the development of the **FastProfitAdaptiveReward** function and a Curriculum Learning (CL) strategy to improve agent training.

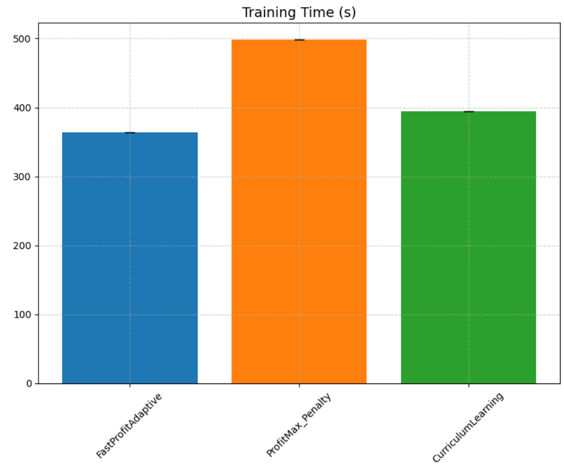


Figure 4.8: Training time performance comparison between the original reward function (**ProfitMax_Penalty**) and the new adaptive function (**FastProfitAdaptive**), with and without Curriculum Learning.

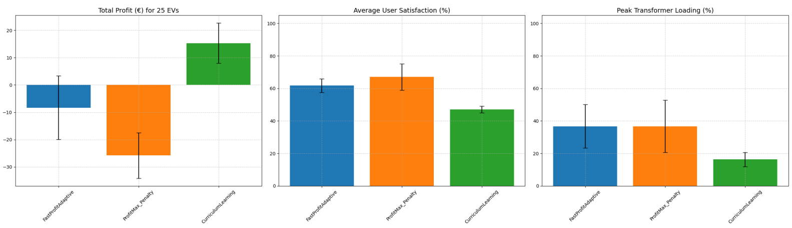
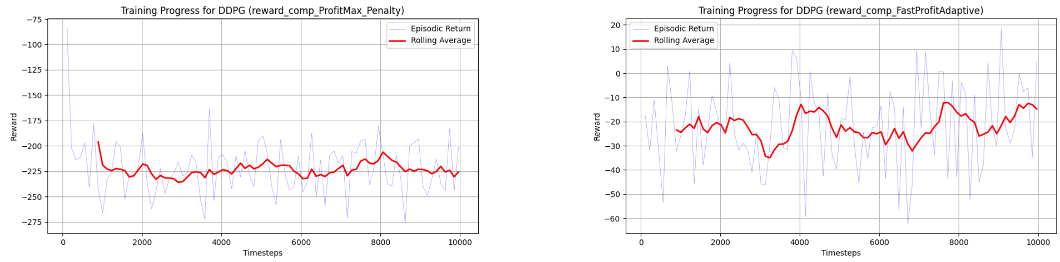


Figure 4.9: Performance comparison on the KPI between the original reward function (**ProfitMax_Penalty**) and the new adaptive function (**FastProfitAdaptive**), with and without Curriculum Learning.

As summarized in Figure 4.9, the DDPG agent trained with the CL method significantly outperforms the one trained with the original, achieving higher profits and better overall performance because it uses both the advantage of the two reward funtions.



(a) Original Reward

(b) New Adaptive Reward

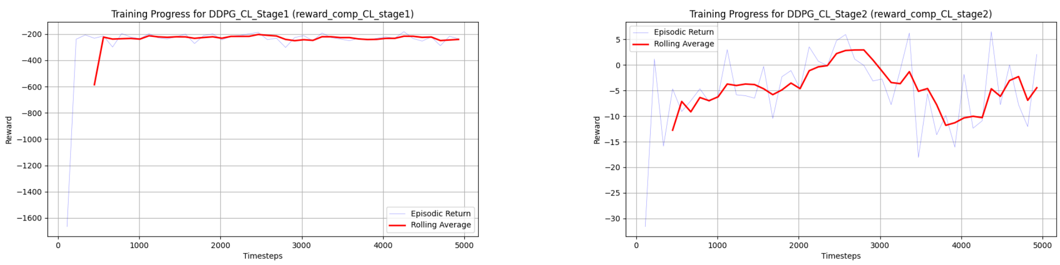
Figure 4.10: Comparison of training progress for DDPG with the original vs. the new adaptive reward function. The new function leads to a more stable and consistently positive reward signal during training.

Curriculum Learning Stages

The CL strategy involved two stages:

1. **Stage 1 (First 5000 steps):** The agent is trained with the original **ProfitMax_Penalty** reward, forcing it to first learn basic constraint satisfaction.

2. **Stage 2 (After 5000 steps):** The reward function switches to the new **FastProfitAdaptiveReward**, encouraging the agent to build upon its foundation of safe behavior to explore more profitable strategies.



(a) CL Stage 1: Learning basic constraints.

(b) CL Stage 2: Optimizing for profit.

Figure 4.11: Training progress during the two stages of Curriculum Learning. A clear and significant improvement in reward is visible upon switching to the adaptive reward function in Stage 2, demonstrating the effectiveness of the CL strategy.

Figure 4.11 clearly shows the success of this approach. In Stage 2, the agent's reward immediately and consistently climbs to a much higher level than what was achievable with either reward function alone. This confirms that by first establishing a baseline of safe behavior and then optimizing for profit, the CL strategy enables the agent to discover superior policies.

Chapter 5

Conclusions and Future Work

This thesis has undertaken a comprehensive investigation into the complex, multi-objective problem of intelligent EV charging and V2G management. Through the enhancement of the EV2Gym simulation framework and a rigorous comparative analysis of heuristic, model-based (MPC), and DRL control strategies, this work has yielded several key insights into the challenges and opportunities of integrating EVs into the power grid.

5.1 Summary of Key Findings

Our experimental campaign demonstrated a clear hierarchy of performance among the different control paradigms:

- **Heuristic Baselines (AFAP, ALAP):** These simple, rule-based methods consistently failed to navigate the complexities of the V2G environment. Their inability to respond to price signals or respect grid constraints resulted in significant economic losses and severe transformer overloads, confirming their inadequacy for real-world smart charging applications.
- **MPC:** The MPC controllers, particularly the adaptive horizon variant, proved to be highly effective and reliable. They demonstrated a strong ability to enforce hard constraints, successfully avoiding transformer overloads while achieving respectable economic performance. However, their performance is fundamentally tied to the accuracy of forecasts, and their computational cost remains a significant consideration for large-scale deployments. Moreover, if the problem becomes infeasible and the algorithm is not properly tuned—especially in terms of the prediction and control horizons (N_p and N_c)—the controller may fail to converge or even stall, compromising overall system operability.
- **DRL:** The DRL agents, especially SAC and DDPG, showcased the highest potential for profitability by effectively learning to exploit price differentials through V2G operations. The results indicated that while more advanced algorithms such as TD3 or TQC are theoretically designed to outperform DDPG by reducing overestimation bias and improving stability, in practice

they can yield suboptimal solutions if not carefully tuned and optimized. This highlights that the true potential of DRL lies not only in algorithm selection but, most importantly, in the proper design and calibration of the learning process itself.

5.2 The Critical Role of System Parameters and Training Methodology

The sensitivity analysis conducted in Chapter 4 provided definitive evidence on the parameters that govern V2G system performance. The **Discharge Price Factor** was identified as the single most dominant driver of profitability, with its influence exhibiting strong non-linearities and interactions. Similarly, the **Transformer Max Power** emerged as a critical bottleneck, whose impact is heavily modulated by the EV demand (spawn multiplier). This underscores the necessity for control agents to be robust to variations in these key environmental factors. However, it should be noted that the **Morris sensitivity analysis**, while useful for identifying dominant parameters, presents certain limitations in this context. Since it is not inherently designed for evaluating near-optimal trajectories and relies on random sampling, its reliability can be affected when applied to highly non-linear, optimization-based systems. Therefore, to obtain more consistent and meaningful results, the analysis would require either a larger number of trajectories or a finer discretization of the input levels, ideally calibrated around optimal or near-optimal operating regions. Perhaps the most significant contribution of this thesis is the demonstrated success of advanced DRL training techniques. The development of the **FastProfitAdaptiveReward** function, which dynamically balances profit-seeking with constraint satisfaction based on historical performance, led to agents that learned more stable and profitable policies than those trained with a static reward function. Furthermore, the implementation of a **two-stage CL strategy** proved to be exceptionally effective. By first training the agent to master basic constraint satisfaction before exposing it to the full profit maximization objective, the CL approach enabled the agent to discover superior policies that were otherwise inaccessible. The marked increase in performance during the second stage of the curriculum provides strong evidence that guiding the learning process is crucial for solving complex, multi-objective real-world problems.

5.3 Future Work

While Qiu et al. (2023) survey RL applications for EV dispatch (G2V, V2H, V2G) with an emphasis on operational and dispatch problems, their treatment is predominantly technical (states/actions/rewards, ancillary services, network and communication concerns) rather than primarily framed around profit-maximization in market settings [34]. Nevertheless, several technical contributions and identified gaps in their review can be taken as concrete starting points to strengthen profit-oriented DRL controllers:

– **Adopt network-aware state representations.** Use the richer state vectors proposed for V2G (time, SoC, nodal price, local RES, nodal demand, voltage/frequency deviations, carbon-price signals) when training profit-seeking agents, so that economic actions remain feasible with respect to physical network constraints. This reduces the risk of profit-seeking policies that violate grid limits.

– **Reward design: multi-objective and constrained formulations.** Qiu et al.[34] show that common practice mixes economic reward with constraint penalties; for profit-driven V2G, prefer either multi-objective RL (Pareto fronts) or explicit constrained-RL formulations (e.g., constrained policy optimization, Lagrangian/CPO) so that revenue objectives do not compromise safety. Include sensitivity analyses of penalty weights in any ablation study.

– **Hybrid DRL–MPC (safety layer) and runtime "shielding".** To reconcile economic optimization with hard grid constraints, implement a hierarchical control where a DRL agent proposes market-optimal setpoints and an MPC (or other certifying controller) enforces feasibility (voltage, transformer limits, thermal limits). Qiu et al.[34] identify this co-design requirement between power and control modules as key for realistic dispatch.

– **Environment realism, co-optimization and cyber considerations.** Extend price-driven environments with alternating current power-flow constraints, transformer aging/capacity models, and communication-channel models (latency/packet-loss and potential cyber risks) so that revenue-seeking policies remain robust under realistic operational and adversarial conditions.

– **Sim-to-real via Hardware in the loop (HIL), transfer learning and curated datasets.** Follow their recommendation to validate algorithms using HIL tests and real-world traces (prices, arrival/departure, measurement noise); publish anonymized datasets and training checkpoints so that market-aware DRL methods can be replicated and stress-tested under real uncertainty.

Taken together, these technical building blocks provide principled ways to convert market/profit objectives into *operationally safe* DRL controllers: richer state/action modelling, constrained or multi-objective reward design, hybrid safety layers, scalable MARL architectures, realistic co-optimized environments, and sim-to-real validation.
In conclusion, while this thesis has primarily addressed the economic dimension of V2G control, the technical foundations discussed by Qiu et al.[34] including safety, scalability, and robustness—represent complementary directions that can strengthen the reliability and real-world feasibility

of profit-driven control frameworks. Combining these two perspectives will be essential for developing intelligent agents that are not only economically optimal but also operationally secure and grid-compliant.

Bibliography

[1] Feyijimi Adegbohun et al. “A Review of Bidirectional Charging Grid Support Applications and Battery Degradation Considerations”. In: *Energies* 17.6 (2024).

[2] Fayiz Alfaverh, Mouloud Denai, and Yichuang Sun. “Optimal vehicle-to-grid control for supplementary frequency regulation using deep reinforcement learning”. In: *Electric Power Systems Research* 214 (2023), p. 108949. ISSN: 0378-7796.

[3] Dou An, Feifei Cui, and Xun Kang. “Optimal scheduling for charging and discharging of electric vehicles based on deep reinforcement learning”. In: *Frontiers in Energy Research* Volume 11 - 2023 (2023).

[4] A. Bemporad and C. Filippi. “Suboptimal explicit MPC via approximate multiparametric quadratic programming”. In: 5 (2001), 4851–4856 vol.5.

[5] Alberto Bemporad. “Explicit Model Predictive Control”. In: (2013).

[6] Christoph R. Birkl et al. “Degradation diagnostics for lithium ion cells”. In: *Journal of Power Sources* 341 (2017), pp. 373–386.

[7] Francesca Bisetto. “La Duck Curve e la scomposizione della stagionalità dei consumi elettrici. Aspetti teorici e modelli statistici”. Dipartimento di Scienze Statistiche, Correlatore: Luigi Grossi. Tesi di laurea magistrale. Padova: Università degli Studi di Padova, 2023.

[8] Francesco Borrelli, Alberto Bemporad, and Manfred Morari. *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.

[9] Greg Brockman et al. “OpenAI Gym”. In: (2016). arXiv: `1606.01540 [cs.LG]`.

[10] E.F. Camacho and C. Bordons. *Model Predictive Control*. Springer, 2013.

[11] Viorica Rozina Chifu et al. “A Deep Q-Learning based Smart Scheduling of EVs for Demand Response in Smart Grids”. In: (2024). arXiv: `2401.02653 [cs.LG]`.

[12] European Commission. *Fit for 55: Delivering the EU’s 2030 Climate Target*. 2021.

[13] C. R. Cutler and B. L. Ramaker. "Dynamic Matrix Control – A Computer Control Algorithm". In: *Proceedings of the Joint Automatic Control Conference*. Vol. 1. San Francisco, USA, 1980.

[14] Stefan Englberger et al. "Electric vehicle multi-use: Optimizing multiple value streams using mobile storage systems in a vehicle-to-grid context". In: *Applied Energy* 304 (2021), p. 117862. ISSN: 0306-2619.

[15] Kilian Freitag et al. "Curriculum Reinforcement Learning for Complex Reward Functions". In: (2025). arXiv: 2410.16790 [cs.LG].

[16] Scott Fujimoto, Herke van Hoof, and David Meger. "Addressing Function Approximation Error in Actor-Critic Methods". In: 2018. arXiv: 1802.09477 [cs.AI].

[17] Shuifu Gu, Kejun Qian, and Yongbiao Yang. "Optimization of Electric Vehicle Charging and Discharging Strategies Considering Battery Health State: A Safe Reinforcement Learning Approach". In: *World Electric Vehicle Journal* 16.5 (2025).

[18] Tuomas Haarnoja et al. *Soft Actor-Critic Algorithms and Applications*. 2019. arXiv: 1812.05905 [cs.LG].

[19] Tuomas Haarnoja et al. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: 2018. arXiv: 1801.01290 [cs.LG].

[20] Sinan Ibrahim et al. "Comprehensive Overview of Reward Engineering and Shaping in Advancing Reinforcement Learning Applications". In: (2024). arXiv: 2408.10215 [cs.LG].

[21] Benjamin Karg and Sergio Lucia. "Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning". In: *IEEE Transactions on Cybernetics* 50.9 (2020). ISSN: 2168-2275.

[22] Arthur J. Krener. "Adaptive Horizon Model Predictive Control". In: (2016). arXiv: 1602.08619 [math.OC].

[23] Arsenii Kuznetsov et al. "Controlling Overestimation Bias with Truncated Mixture of Continuous Distributional Quantile Critics". In: 2020. arXiv: 2005.04269 [cs.LG].

[24] Donghe Li et al. "An electrical vehicle-assisted demand response management system: A reinforcement learning method". In: *Frontiers in Energy Research* Volume 10 - 2022 (2023).

[25] Timothy P. Lillicrap et al. "Continuous control with deep reinforcement learning". In: (2019). arXiv: 1509.02971 [cs.LG].

[26] Ding Liu et al. "Deep Reinforcement Learning for Charging Scheduling of Electric Vehicles Considering Distribution Network Voltage Stability". In: *Sensors* 23.3 (2023).

[27] Horia Mania, Aurelia Guy, and Benjamin Recht. "Simple random search provides a competitive approach to reinforcement learning". In: (2018). arXiv: 1803.07055 [cs.LG].

[28] D.Q. Mayne et al. “Constrained model predictive control: Stability and optimality”. In: *Automatica* 36.6 (2000), pp. 789–814.

[29] Camila Minchala-Ávila, Paul Arévalo, and Danny Ochoa-Correa. “A Systematic Review of Model Predictive Control for Robust and Efficient Energy Management in Electric Vehicle Integration and V2G Applications”. In: *Modelling* 6.1 (2025).

[30] Volodymyr Mnih et al. “Asynchronous Methods for Deep Reinforcement Learning”. In: 2016. arXiv: `1602.01783 [cs.LG]`.

[31] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping”. In: *Proceedings of the Sixteenth International Conference on Machine Learning*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, pp. 278–287. ISBN: 1558606122.

[32] Stylianos Orfanoudakis et al. “EV2Gym: A Flexible V2G Simulator for EV Smart Charging Research and Benchmarking”. In: *arXiv preprint arXiv:2404.01849* (2024).

[33] Rey Pocius et al. “Comparing Reward Shaping, Visual Hints, and Curriculum Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32 (Apr. 2018).

[34] Dawei Qiu et al. “Reinforcement learning for electric vehicle applications in power systems:A critical review”. In: *Renewable and Sustainable Energy Reviews* 173 (2023), p. 113052. ISSN: 1364-0321.

[35] J. Richalet et al. “Model predictive heuristic control: Applications to industrial processes”. In: *Automatica* 14.5 (1978), pp. 413–428. ISSN: 0005-1098.

[36] Mohammad Sadeghi. “Cost and power loss aware coalitions under uncertainty in transactive energy systems”. In: *Université d’Ottawa / University of Ottawa* (2022). PhD Thesis.

[37] Gabriel Salvatti et al. “Electric Vehicles Energy Management with V2G/G2V Multifactor Optimization of Smart Grids”. In: *Energies* 13 (Mar. 2020), p. 1191.

[38] Tom Schaul et al. “Prioritized Experience Replay”. In: (Nov. 2015).

[39] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: 2017. arXiv: `1707.06347 [cs.LG]`.

[40] Minho Shin et al. “Building an Interoperability Test System for Electric Vehicle Chargers Based on ISO/IEC 15118 and IEC 61850 Standards”. In: *Applied Sciences* 6.6 (2016).

[41] G. Srihari et al. “Integration of electric vehicle into smart grid: a meta heuristic algorithm for energy management between V2G and G2V”. In: *Frontiers in Energy Research* Volume 12 - 2024 (2024).

[42] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition, in progress. A Bradford Book, c. 2014, 2015. Cambridge, Massachusetts; London, England: The MIT Press, 2015.

[43] Ahmad Tavakoli et al. "Impacts of Grid integration of Solar PV andElectric Vehicle on Grid Stability, PowerQuality and Energy Economics: A Review". In: *IET Energy Systems Integration* 2 (Sept. 2020).

[44] J. Vetter et al. "Ageing mechanisms in lithium-ion batteries". In: *Journal of Power Sources* 147.1 (2005), pp. 269–281.

[45] David Silver et. al Volodymyr Mnih Koray Kavukcuoglu. "Human-level control through deep reinforcement learning". In: *Nature* 518.7540 (2015), pp. 529–533.

[46] Muning Wen et al. "Multi-Agent Reinforcement Learning is a Sequence Modeling Problem". In: (2022). arXiv: `2205.14953 [cs.MA]`.

[47] Corey D. White and K. Max Zhang. "Using vehicle-to-grid technology for frequency regulation and peak-load reduction". In: *Journal of Power Sources* 196.8 (2011), pp. 3972–3980.

[48] Hongbin Xie et al. "Reinforcement learning for vehicle-to-grid: A review". In: *Advances in Applied Energy* 17 (2025), p. 100214. ISSN: 2666-7924.

[49] Na Xu et al. "A Review of Smart Grid Evolution and Reinforcement Learning: Applications, Challenges and Future Directions". In: *Energies* 18.7 (2025).

[50] Yubao Zhang, Xin Chen, and Yuchen Zhang. "Transfer Deep Reinforcement Learning-based Large-scale V2G Continuous Charging Coordination with Renewable Energy Sources". In: (2022). arXiv: `2210.07013 [eess.SY]`.